



US012407812B2

(12) **United States Patent**
Zhang et al.

(10) **Patent No.:** **US 12,407,812 B2**

(45) **Date of Patent:** **Sep. 2, 2025**

(54) **DERIVING REFERENCE SAMPLE
POSITIONS IN VIDEO CODING**

(71) Applicants: **Beijing Bytedance Network
Technology Co., Ltd.**, Beijing (CN);
Bytedance Inc., Los Angeles, CA (US)

(72) Inventors: **Kai Zhang**, San Diego, CA (US); **Li
Zhang**, San Diego, CA (US); **Hongbin
Liu**, Beijing (CN); **Zhipin Deng**,
Beijing (CN); **Jizheng Xu**, San Diego,
CA (US); **Yue Wang**, Beijing (CN)

(73) Assignees: **BEIJING BYTEDANCE NETWORK
TECHNOLOGY CO., LTD.**, Beijing
(CN); **BYTEDANCE INC.**, Los
Angeles, CA (US)

(*) Notice: Subject to any disclaimer, the term of this
patent is extended or adjusted under 35
U.S.C. 154(b) by 0 days.

(21) Appl. No.: **18/452,736**

(22) Filed: **Aug. 21, 2023**

(65) **Prior Publication Data**

US 2023/0396759 A1 Dec. 7, 2023

Related U.S. Application Data

(63) Continuation of application No. 17/699,021, filed on
Mar. 18, 2022, now Pat. No. 11,743,454, which is a
(Continued)

(30) **Foreign Application Priority Data**

Sep. 19, 2019 (WO) PCT/CN2019/106671
Sep. 20, 2019 (WO) PCT/CN2019/107082
Oct. 5, 2019 (WO) PCT/CN2019/109837

(51) **Int. Cl.**

H04N 19/105 (2014.01)

H04N 19/132 (2014.01)

(Continued)

(52) **U.S. Cl.**

CPC **H04N 19/105** (2014.11); **H04N 19/132**
(2014.11); **H04N 19/159** (2014.11);
(Continued)

(58) **Field of Classification Search**

CPC .. **H04N 19/105**; **H04N 19/132**; **H04N 19/159**;
H04N 19/186; **H04N 19/59**; **H04N 19/70**;
H04N 19/176

See application file for complete search history.

(56)

References Cited

U.S. PATENT DOCUMENTS

5,602,593 A 2/1997 Katto
6,804,403 B1 10/2004 Wang et al.
(Continued)

FOREIGN PATENT DOCUMENTS

CN 101309409 A 11/2008
CN 104169971 A 11/2014
(Continued)

OTHER PUBLICATIONS

“Slice Header Prediction for Depth Maps Bit Reduction”—Yang et
al., 2012 5th International Congress on Image and Signal Processing
(CISP 2012) (Year: 2012).*

(Continued)

Primary Examiner — Mainul Hasan

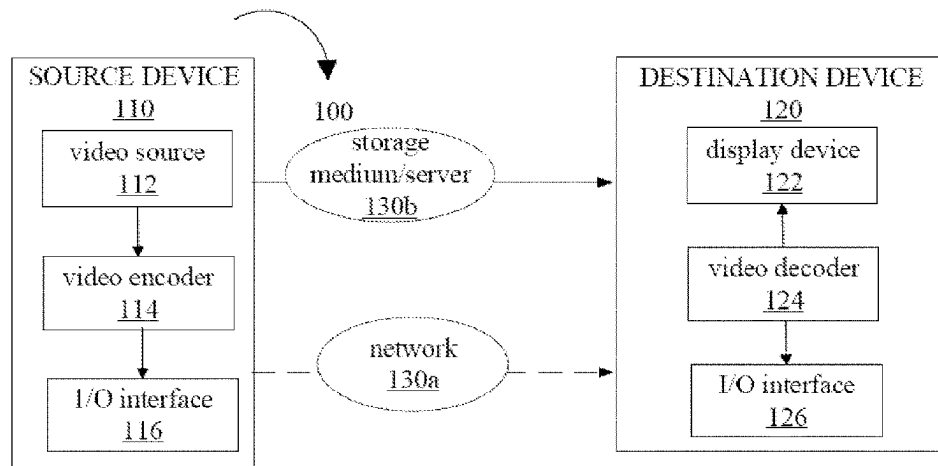
(74) *Attorney, Agent, or Firm* — Conley Rose, P.C.

(57)

ABSTRACT

An example method of video processing includes determin-
ing, for a conversion between a current picture of a video
and a coded representation of the video, a position of a
reference sample in a reference picture that is associated
with the current picture based on a top-left position of a
window of a picture. The picture includes at least the current
picture or the reference picture, and the window is subject to

(Continued)



a processing rule during the conversion. The method also includes performing the conversion based on the determining.

20 Claims, 17 Drawing Sheets

Related U.S. Application Data

continuation of application No. PCT/CN2020/116469, filed on Sep. 21, 2020.

(51) Int. Cl.

H04N 19/159 (2014.01)

H04N 19/176 (2014.01)

H04N 19/186 (2014.01)

H04N 19/70 (2014.01)

(52) U.S. Cl.

CPC **H04N 19/176** (2014.11); **H04N 19/186** (2014.11); **H04N 19/70** (2014.11)

(56) References Cited

U.S. PATENT DOCUMENTS

7,627,037 B2 12/2009 Li et al.
8,442,109 B2 5/2013 Wang et al.
9,294,777 B2 3/2016 Wang
9,485,508 B2 11/2016 Wang et al.
9,509,999 B2 11/2016 Seregin et al.
9,521,425 B2 12/2016 Chen et al.
9,578,328 B2 2/2017 Rapaka
9,667,996 B2 5/2017 Chen et al.
9,712,871 B2 7/2017 Ramasubramonian et al.
9,762,927 B2 9/2017 Chen et al.
9,788,007 B2 10/2017 Wang et al.
9,918,091 B2 3/2018 Wang et al.
9,992,493 B2 6/2018 Chen et al.
9,992,494 B2 6/2018 Zhang et al.
10,116,937 B2 10/2018 Li et al.
10,148,981 B2 12/2018 Zhu
10,158,884 B2 12/2018 Zhang
10,212,411 B2 2/2019 Zhang et al.
10,306,240 B2 5/2019 Xiu et al.
10,306,269 B2 5/2019 Hendry et al.
10,382,781 B2 8/2019 Zhao
10,402,689 B1 9/2019 Bogdanovych et al.
10,523,964 B2 12/2019 Chuang
10,567,769 B2 2/2020 Li et al.
10,764,576 B2 9/2020 Li
10,776,663 B1 9/2020 Bogdanovych et al.
10,779,002 B2 9/2020 Chen
10,904,565 B2 1/2021 Chuang
11,039,165 B2 6/2021 Liu
11,128,884 B2 9/2021 Liu et al.
11,233,988 B2 1/2022 Seregin
11,240,531 B2 2/2022 Liu
11,310,508 B2 4/2022 Liu et al.
11,330,294 B2 5/2022 Zheng
11,425,417 B2 8/2022 Liu
11,503,342 B2 11/2022 Deshpande
11,611,780 B2 3/2023 Zhang
11,641,464 B2 5/2023 Zhang et al.
11,711,547 B2 7/2023 Zhang et al.
11,722,660 B2 8/2023 Zhang et al.
11,736,734 B2 8/2023 Wang et al.
11,743,454 B2 8/2023 Zhang et al.
11,831,901 B2 11/2023 Liu et al.
12,015,795 B2 6/2024 Zhang et al.
2006/0013308 A1 1/2006 Kim
2008/0253448 A1 10/2008 Lin et al.
2011/0310976 A1 12/2011 Wang
2012/0082218 A1 4/2012 Misra
2012/0189049 A1 7/2012 Coban

2013/0177084 A1 7/2013 Wang et al.
2013/0259446 A1 10/2013 Sathish
2014/0003493 A1 1/2014 Chen
2014/0192887 A1 7/2014 Hong
2014/0328398 A1 11/2014 Chen
2014/0362910 A1 12/2014 Seregin
2015/0023409 A1 1/2015 Schierl
2015/0195577 A1* 7/2015 Hannuksela H04N 19/70
375/240.25
2015/0201204 A1 7/2015 Chen
2015/0264348 A1* 9/2015 Zou H04N 19/593
375/240.02
2015/0304666 A1 10/2015 Seregin
2015/0319447 A1 11/2015 Minoo
2015/0355459 A1 12/2015 Shen
2015/0358631 A1 12/2015 Zhang
2016/0100168 A1 4/2016 Rapaka
2016/0100189 A1 4/2016 Pang et al.
2016/0191931 A1 6/2016 Hannuksela
2016/0277762 A1 9/2016 Zhang
2016/0360210 A1 12/2016 Xiu
2017/0064339 A1* 3/2017 Van der Auwera .. H04N 19/117
2017/0094277 A1* 3/2017 Chen H04N 19/124
2017/0150156 A1 5/2017 Zhang
2017/0150176 A1 5/2017 Zhang
2017/0201769 A1 7/2017 Chon
2017/0295380 A1 10/2017 Huang
2018/0199057 A1 7/2018 Chuang et al.
2018/0220138 A1 8/2018 He
2018/0241998 A1 8/2018 Chen
2018/0270502 A1 9/2018 Mukherjee
2018/0367814 A1 12/2018 Seregin
2019/0110058 A1 4/2019 Chien
2019/0110064 A1 4/2019 Zhang
2019/0132606 A1 5/2019 Su et al.
2019/0141333 A1 5/2019 Lee
2019/0166370 A1 5/2019 Xiu
2019/0215537 A1 7/2019 Poirier
2019/0238864 A1 8/2019 Xiu
2019/0260990 A1 8/2019 Lim
2019/0273919 A1 9/2019 Lim
2019/0306502 A1 10/2019 Gadde et al.
2020/0260070 A1* 8/2020 Yoo H04N 19/103
2020/0359016 A1 11/2020 Li
2020/0359017 A1 11/2020 Li
2020/0359018 A1 11/2020 Li
2020/0382795 A1 12/2020 Zhang et al.
2020/0382807 A1 12/2020 Liu et al.
2020/0396453 A1 12/2020 Zhang et al.
2020/0396455 A1* 12/2020 Liu H04N 19/176
2021/0021812 A1 1/2021 Zheng
2021/0029362 A1 1/2021 Liu et al.
2021/0029378 A1 1/2021 He
2021/0051335 A1 2/2021 Liao
2021/0051339 A1 2/2021 Liu et al.
2021/0051341 A1 2/2021 Seregin
2021/0067791 A1 3/2021 Ye
2021/0067796 A1 3/2021 Chen
2021/0076063 A1 3/2021 Liu et al.
2021/0084311 A1 3/2021 Seregin
2021/0084315 A1 3/2021 Chen
2021/0092372 A1 3/2021 Misra et al.
2021/0092435 A1 3/2021 Liu et al.
2021/0105499 A1 4/2021 Lee
2021/0126222 A1 4/2021 Liu et al.
2021/0127112 A1 4/2021 Choi
2021/0136419 A1 5/2021 Hsiang
2021/0160511 A1 5/2021 Zhang et al.
2021/0168366 A1 6/2021 Li et al.
2021/0185347 A1 6/2021 Liu et al.
2021/0195179 A1 6/2021 Coban
2021/0227211 A1 7/2021 Liu et al.
2021/0227245 A1 7/2021 Liu et al.
2021/0227246 A1 7/2021 Liu et al.
2021/0227250 A1 7/2021 Liu et al.
2021/0235075 A1 7/2021 Lee
2021/0235083 A1 7/2021 Liu et al.
2021/0250592 A1 8/2021 Xiu
2021/0250622 A1 8/2021 Jung

(56)

References Cited**U.S. PATENT DOCUMENTS**

2021/0266530	A1	8/2021	Liu et al.	
2021/0266585	A1	8/2021	Liu et al.	
2021/0274201	A1	9/2021	Xu	
2021/0274202	A1	9/2021	Xu	
2021/0274211	A1	9/2021	Liu et al.	
2021/0274212	A1	9/2021	Liu et al.	
2021/0274213	A1	9/2021	Xiu	
2021/0281865	A1	9/2021	Liu et al.	
2021/0281876	A1	9/2021	Zhang et al.	
2021/0297674	A1	9/2021	Xu	
2021/0297705	A1*	9/2021	Sjöberg	H04N 19/174
2021/0306666	A1	9/2021	Lee	
2021/0314630	A1	10/2021	Misra et al.	
2021/0352279	A1	11/2021	Xu	
2021/0360270	A1	11/2021	Xu	
2021/0368164	A1	11/2021	Xu	
2021/0368178	A1	11/2021	Xu	
2021/0368181	A1	11/2021	Liu et al.	
2021/0368187	A1	11/2021	Zhang et al.	
2021/0368198	A1	11/2021	Zhang et al.	
2021/0368199	A1	11/2021	Zhang et al.	
2021/0368203	A1	11/2021	Zhang et al.	
2021/0377559	A1	12/2021	Liu et al.	
2021/0385437	A1	12/2021	Xu	
2021/0385481	A1	12/2021	Liu et al.	
2021/0385482	A1	12/2021	Liu et al.	
2021/0400299	A1	12/2021	Zhu et al.	
2022/0007053	A1	1/2022	Hanhart	
2022/0060735	A1	2/2022	Chen	
2022/0060743	A1	2/2022	Bordes	
2022/0150490	A1	5/2022	Kuo	
2022/0174303	A1	6/2022	Jang	
2022/0217328	A1	7/2022	Zhang	
2022/0232207	A1	7/2022	Zhang et al.	
2022/0239950	A1	7/2022	Zhang	
2022/0256197	A1	8/2022	Zhang	
2022/0264086	A1	8/2022	Zhang	
2022/0264137	A1	8/2022	Zhang	
2022/0272353	A1	8/2022	Zhang	
2022/0272378	A1*	8/2022	Samuelsson	H04N 19/70
2022/0353536	A1	11/2022	Samuelsson	
2022/0353537	A1	11/2022	Deshpande	
2022/0394255	A1	12/2022	Kang	
2023/0007264	A1	1/2023	Koo	
2023/0046994	A1	2/2023	Zhang et al.	

FOREIGN PATENT DOCUMENTS

CN	104365105	A	2/2015
CN	104604223	A	5/2015
CN	105120271	A	12/2015
CN	105122804	A	12/2015
CN	105144719	A	12/2015
CN	105191315	A	12/2015
CN	105230018	A	1/2016
CN	105308965	A	2/2016
CN	105519118	A	4/2016
CN	105721880	A	6/2016
CN	106105227	A	11/2016
CN	106416255	A	2/2017
CN	106664425	A	5/2017
CN	106797476	A	5/2017
CN	107409209	A	11/2017
CN	107431818	A	12/2017
CN	107483934	A	12/2017
CN	107646195	A	1/2018
CN	108781282	A	11/2018
CN	109076221	A	12/2018
CN	109255750	A	1/2019
CN	109792539	A	5/2019
CN	110115032	A	8/2019
CN	110169061	A	8/2019
CN	110169073	A	8/2019
CN	111201793	A	5/2020
CN	114556937	A	5/2022

CN	114556937	B	5/2022
CN	114556955	B	4/2024
CN	114503588	B	10/2024
CN	114503574	B	1/2025
EP	2753079	A1	7/2014
EP	3177018	A1	6/2017
EP	3939311	A1	1/2022
GB	2590634	A	7/2021
IN	547978		8/2024
IN	552065		10/2024
IN	554104		11/2024
JP	2020017970	A	1/2020
JP	2022505470	A	1/2022
JP	2022537064	A	8/2022
JP	2022544164	A	10/2022
JP	2022547599	A	11/2022
JP	2022548704	A	11/2022
JP	7639073	B2	3/2025
KR	102737934	B1	11/2024
KR	102746666	B1	12/2024
KR	102758910	B1	1/2025
MX	411735		4/2024
RU	2619888	C2	5/2017
RU	2824188	C2	8/2024
WO	2009051719	A2	4/2009
WO	2013002589	A2	1/2013
WO	2013128010	A2	9/2013
WO	2013128010	A9	7/2014
WO	2014167178	A1	10/2014
WO	2015038877	A1	3/2015
WO	2015053697	A1	4/2015
WO	2015103224	A1	7/2015
WO	2015196117	A1	12/2015
WO	2016196043	A1	12/2016
WO	2017133661	A1	8/2017
WO	2017151343	A1	9/2017
WO	2017180766	A1	10/2017
WO	2017195554	A1	11/2017
WO	2017214420	A1	12/2017
WO	2018012933	A1	1/2018
WO	2018053293	A1	3/2018
WO	2018084339	A1	5/2018
WO	2020009434	A1	1/2020
WO	2020096755	A1	5/2020
WO	2020185879	A1	9/2020
WO	2020247255	A1	12/2020
WO	2020253858	A1	12/2020
WO	2020262286	A1	12/2020
WO	2021026255	A1	2/2021
WO	2021026363	A1	2/2021
WO	2021050234	A1	3/2021
WO	2021068956	A1	4/2021
WO	2021123326	A1	6/2021

OTHER PUBLICATIONS

Document: JVET-M1001-v6, Bross, B., et al., "Versatile Video Coding (Draft 4)," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 13th Meeting: Marrakech, MA, Jan. 9-18, 2019, 299 pages.

Document: JVET-P0409-v3, Chen, J., et al., "CE1-related: Enable PROF and RPR," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 16th Meeting: Geneva, CH, Oct. 1-11, 2019, 8 pages.

Document: JVET-N0118-v3, Hendry, "AHG19: Adaptive resolution change (ARC) support in VVC," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 14th Meeting: Geneva, CH, Mar. 19-27, 2019, 8 pages.

Document: JVET-P0239-v1, Wan, W., et al., "AHG17: Picture Header," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 16th Meeting: Geneva, CH, Oct. 1-11, 2019, 9 pages.

Document: JVET-P0068-v2, Gao, H., et al., "CE4: CE4-1.1, CE4-1.2 and CE4-1.14: Geometric Merge Mode (GEO)," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG11 16th Meeting: Geneva, CH, Oct. 1-11, 2019, 7 pages.

(56)

References Cited**OTHER PUBLICATIONS**

Russian Office Action from Russian Application No. 2022107828 dated Jan. 30, 2024, 12 pages.

Chinese Notice of Allowance from Chinese Patent Application No. 202080070273.7 dated Jul. 17, 2024, 6 pages.

Document: JVET-Q2001-v2, Bross, B., et al., "Versatile Video Coding (Draft 8)," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, 492 pages.

"Resampling Process of the Scalable High Efficiency Video Coding"—Chen et al., IEEE 2015 Data Compression Conference; Date of Conference: Apr. 7-9, 2015. (Year: 2015).

Suehring et al. "AHG17: Conformance Window," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 14th Meeting, Geneva, CH, Mar. 19-27, 2019, document JVET-N0352, 2019.

Hendry et al. "AHG8: Support for Reference Picture Resampling—Handling of Picture Size Signalling, Conformance Windows, and DPB Management," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 15th Meeting, Gothenburg SE, Jul. 3-12, 2019, document JVET-00133, 2019.

Bross et al. "Versatile Video Coding (Draft 10)," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 19th Meeting, by teleconference, Jun. 22-Jul. 1, 2020, document JVET-S2001, 2020.

Notice of Allowance from U.S. Appl. No. 17/713,063 dated Apr. 27, 2023.

Extended European Search Report from European Patent No. 20866342.7 dated Oct. 31, 2022, (12 pages).

Extended European Search Report from European Patent No. 20866741.0 dated Nov. 7, 2022 (11 pages).

Extended European Search Report from European Patent No. 20874792.3 dated Nov. 3, 2022 (9 pages).

Extended European Search Report from European Patent No. 20876825.9 dated Oct. 31, 2022 (10 pages).

Extended European Search Report from European Patent No. 20874449.0 dated Dec. 21, 2022 (12 pages).

Extended European Search Report from European Patent No. 20906207.4 dated Dec. 20, 2022 (12 pages).

Final Office Action from U.S. Appl. No. 17/713,063 dated Nov. 25, 2022.

Notice of Allowance from U.S. Appl. No. 17/718,785 dated Dec. 7, 2022.

Notice of Allowance from U.S. Appl. No. 17/719,708 dated Dec. 23, 2022.

Notice of Allowance from U.S. Appl. No. 17/849,293 dated Jan. 20, 2023.

Examination Report from Indian Patent Application No. 202247020262 dated Aug. 25, 2022 (5 pages).

Non Final Office Action from U.S. Appl. No. 17/699,014 dated Jun. 24, 2022.

Non Final Office Action from U.S. Appl. No. 17/713,063 dated Jul. 15, 2022.

Non Final Office Action from U.S. Appl. No. 17/718,880 dated Jul. 19, 2022.

Non Final Office Action from U.S. Appl. No. 17/713,104 dated Jul. 22, 2022.

Non Final Office Action from U.S. Appl. No. 17/718,785 dated Aug. 4, 2022.

Non Final Office Action from U.S. Appl. No. 17/719,708 dated Aug. 12, 2022.

International Search Report and Written Opinion from International Patent Application No. PCT/CN2020/116468 dated Nov. 25, 2020 (10 pages).

International Search Report and Written Opinion from International Patent Application No. PCT/CN2020/116469 dated Dec. 4, 2020 (13 pages).

International Search Report and Written Opinion from International Patent Application No. PCT/CN2020/119912 dated Jan. 12, 2021 (12 pages).

International Search Report and Written Opinion from International Patent Application No. PCT/CN2020/119913 dated Jan. 14, 2021 (10 pages).

International Search Report and Written Opinion from International Patent Application No. PCT/CN2020/120288 dated Dec. 30, 2020 (10 pages).

International Search Report and Written Opinion from International Patent Application No. PCT/CN2020/120289 dated Dec. 31, 2020 (8 pages).

International Search Report and Written Opinion from International Patent Application No. PCT/CN2020/120554 dated Jan. 12, 2021 (12 pages).

International Search Report and Written Opinion from International Patent Application No. PCT/CN2020/140044 dated Mar. 31, 2021 (10 pages).

Non Final Office Action from U.S. Appl. No. 17/699,021 dated Jul. 1, 2022.

Final Office Action from U.S. Appl. No. 17/699,021 dated Nov. 15, 2022.

Notice of Allowance from U.S. Appl. No. 17/699,021 dated Mar. 29, 2023.

Wan et al. "AHG17: Picture Header," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 16th Meeting: Geneva, CH, Oct. 1-11, 2019, document JVET-P0239, 2019. JVET-P0239.spec_text-v1.

Bross et al. "Versatile Video Coding (Draft 6)," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 15th Meeting: Gothenburg, SE, Jul. 3-12, 2019, document JVET-O2001, 2019.

https://vcgit.hhi.fraunhofer.de/jvet/VVCSsoftware_VTM/tags/VTM-6.0, Jun. 9, 2022.

Hendry et al. "On Adaptive Resolution Change (ARC) for WC," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 13th Meeting: Marrakech, MA, Jan. 9-18, 2019. document JVET-M0135, 2019.

Hannuksela et al. "Use Cases and Proposed Design Choices for Adaptive Resolution Changing (ARC)," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 13th Meeting: Marrakech, MA, Jan. 9-8, 2019, document JVET-M0259, 2019.

Hannuksela et al. "AHG19: On Adaptive Resolution Changing," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 14th Meeting: Geneva, CH, Mar. 19-27, 2019, document JVET-N0048, 2019.

Wenger et al. "[AHG19] On Signaling of Adaptive Resolution Change," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 14th Meeting: Geneva, CH, Mar. 19-27, 2019, document JVET-N0052, 2019.

Hong et al. AHG19: "Adaptive Resolution Change (ARC) Support in WC," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 14th Meeting: Geneva, CH, Mar. 19-27, 2019, document JVET-N0118, 2019.

Chen et al. "AHG 19: Adaptive Resolution Change," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 14th Meeting: Geneva, CH, Mar. 19-27, 2019, document JVET-N0279, 2019.

Luo et al. "CE2-Related: Prediction Refinement with Optical Flow for Affine Mode," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 14th Meeting: Geneva, CH, Mar. 19-27, 2019, document JVET-N0236, 2019.

Zhang et al. "CE4-related: Interweaved Prediction for Affine Motion Compensation," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 11th Meeting: Ljubljana, SI, Jul. 10-18, 2018, document JVET-K0102, 2018.

"High Efficiency Video Coding," Series H: Audiovisual and Multimedia Systems: Infrastructure of Audiovisual Services Coding of Moving Video, ITU-T Telecommunication Standardization Sector of ITU, H.265, Feb. 2018.

Rosewarne et al. "High Efficiency Video Coding {HEVC} Test Model 16 (HM 16) Improved Encoder Description Update 7," Joint Collaborative Team on Video Coding (JCT-VC) of ITU-T SG16 WP3 and ISO/IEC JTC1/SC29/WG11 25th Meeting: Chengdu, CN, Oct. 14-21, 2016, document JCTVC-Y1002, 2016.

(56)

References Cited

OTHER PUBLICATIONS

- Chen et al. "Algorithm Description of Joint Exploration Test Model 7 (JEM 7)," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 7th Meeting: Torino, IT, Jul. 13-21, 2017, document JVET-G1001, 2017.
- JEM-7.0: https://jvet.hhi.fraunhofer.de/svn/svn_HMJEMSoftware/tags/HM-16.6-JEM-7.0, Jun. 9, 2022.
- Chen et al. "CE4: Affine Merge Enhancement with Simplification (Test 4.2.2)," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 12th Meeting: Macao, CN, Oct. 3-12, 2018, document JVET-L0368, 2018.
- Zhang et al. "CE2-5.5: History-Parameter-based Affine Model Inheritance," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 14th Meeting: Geneva, CH, Mar. 19-27, 2019, document JVET-N0263, 2019.
- Wang et al. "AHG17: On Access Unit Delimiter and Picture Header," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 16th Meeting: Geneva, CH, Oct. 1-11, 2019, document JVET-P0120, 2019.
- Wan et al. "AHG17: Picture Header," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 16th Meeting: Geneva, CH, Oct. 1-11, 2019, document JVET-P0239, 2019.
- Sanchez et al. "AHG17: On HRD for Open GOP and RAP," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 15th Meeting: Gothenburg, SE, Jul. 3-12, 2019, document JVET-O0569, 2019.
- Nalci et al. "Non-CE6: LFNST Signaling at the TU Level," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 15th Meeting: Gothenburg, SE, Jul. 3-12, 2019, document JVET-O0569, 2019.
- Galpin et al. "Non-CE4: Temporal Merge Modes Simplification," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 15th Meeting: Gothenburg, SE, Jul. 3-12, 2019, document JVET-O0501, 2019.
- Wan et al. "AHG17: Text for Picture Header," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 16th Meeting: Geneva, CH, Oct. 1-11, 2019, document JVET-P1006, 2019.
- Seregin et al. "AHG8: Scaling Window for Scaling Ratio Derivation," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 16th Meeting: Geneva, CH, Oct. 1-11, 2019, document JVET-P0590, 2019.
- Bross et al. "Versatile Video Coding (Draft 7)," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 16th Meeting: Geneva, CH, Oct. 1-11, 2019, document JVET-P2001, 2019.
- Bross et al. "Versatile Video Coding (Draft 8)," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, document JVET-Q2001, 2020.
- Wang et al. "AHG9: Cleanups on Parameter Sets," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, document JVET-Q0117, 2020.
- Abe et al. "AHG9: On JCCR Signaling," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, document JVET-Q0147, 2020.
- Helmrich et al. "CE7-Related: Joint Chroma Residual Coding with Multiple Modes," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 14th Meeting: Geneva, CH, Mar. 19-27, 2019, document JVET-N0282, 2019.
- Lainema et al. "CE7-Related: Joint Coding of Chrominance Residuals," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 13th Meeting: Marrakech, MA, Jan. 9-18, 2019, document JVET-M0305, 2019.
- Meng et al. "Non-CE4: Harmonization of PROF, BDOF and DMVR Syntax," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 1 16th Meeting: Geneva, CH, Oct. 1-11, 2019, document JVET-P0314, 2019.
- Wang et al. "CE4-Related: Remove Redundancy between TMVP and ATMVP," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 13th Meeting: Marrakech, MA, Jan. 9-18, 2019, document JVET-M0345, 2019.
- Xu, et al., "CE8-related: A SPS Level Flag for BDPCM and JCCR," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 15th Meeting: Gothenburg, SE, Jul. 3-12, 2019, document JVET-O0376, 2019.
- Yang, H., et al., "Subblock-Based Motion Derivation and Inter Prediction Renement in the Versatile Video Coding Standard," IEEE Transactions on Circuits and Systems for Video Technology 31.10 (2021): 3862-3877.
- Chen et al., "CE1-Related: Enable PROF for RPR," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 16th Meeting: Geneva, CH, Oct. 1-11, 2019, document JVET-P0409, 2019.
- Zhang et al., "Non-CE1: A Fix on Reference Sample Offset for RPR," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 16th Meeting: Geneva, CH, Oct. 1-11, 2019, JVET-P0381, 2019.
- Sullivan, et al., "Meeting Report of the 15th Meeting of the Joint Video Experts Team (JVET) Gothenburg, SE, Jul. 3-12, 2019," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 15th Meeting: Gothenburg, SE, Jul. 3-12, 2019, document JVET-O2000, 2019.
- "Information technology—Generic Coding of Moving Pictures and Associated Audio Information—Part 2: Video," ISO/IEC 13818-2: 2016, IEC 3. Rue De Varembe, Geneva, CH, Sep. 27, 2013, pp. 1-225, XP082007972.
- Chen et al. "Decoder-Side Motion Vector Refinement Based on Bilateral Template Matching," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 4th Meeting, Chengdu, CN, Oct. 15-21, 2016, document JVET-D0029, 2016.
- Zhou et al., "JVET AHG Report: Implementation Studies (AHG 16)," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 16th Meeting: Geneva, CH, Oct. 1-11, 2019, document JVET-P0016, 2019.
- Wang, et al., "AHG9: Cleanups on Signaling for CC-ALF, BDPCM, ACT and Palette," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, document JVET-Q0520, 2020.
- Deng et al., "AHG9: PH and SH Syntax Cleanups," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting, Brussels, BE, Jan. 7-17, 2020, document JVET-Q0116, 2020.
- Xu et al., "AHG9: Picture Header Syntax Cleanups," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting, Brussels, BE, Jan. 7-17, 2020, Document JVET-Q0428, 2020.
- Document: JVET-M0246, Liu, H., et al., "CE2: Adaptive Motion Vector Resolution for Affine Inter Mode (Test 2.1.2)," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 13th Meeting: Marrakech, MA, Jan. 9-18, 2019, 2 pages. Advanced Video Coding for Generic Audiovisual Services, "Series H: Audiovisual and Multimedia Systems, Infrastructure of Audiovisual Services—Coding of Moving Video," ITU-T Telecommunication Standardization Sector of ITU-T H.264 Jan. 2012, pp. 19,20,42,43,72. Table 6-1; 8 lines just after Table 6-1; p. 72, lines 32-47.
- Wan et al. "AHG17: Text for Picture Header," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 16th Meeting: Geneva, CH, Oct. 1-11, 2019, document JVET-P1006, 2019. JVET-P1006_SpecText-v2 pp. 1, 37-48.
- Communication pursuant to Article 94(3) EPC for European Application No. 20874792.3, mailed Oct. 10, 2024, 5 pages.
- Notice of Reasons for Refusal for Japanese Application No. 2023-204308, mailed Oct. 8, 2024, 10 pages.
- Notice of Reasons for Refusal for Japanese Application No. 2023-207340, mailed Oct. 15, 2024, 15 pages.
- Deng Z., et al. "Non-CE4: Alignment of Luma and Chroma Weights Calculation for TPM Blending," Joint Video Experts Team (JVET)

(56)

References Cited**OTHER PUBLICATIONS**

of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 16th Meeting: Geneva, CH, Oct. 1-11, 2019, Document JVET-P0530, 6 Pages.

Document: JVET-P0305-v2, Liao, R., et al., "CE4-related: Unification of triangle partition mode and geometric merge mode," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 16th Meeting: Geneva, CH, Oct. 1-11, 2019, 7 pages, Sep. 29, 2019.

Hendry, et al., "AHG19: Adaptive Resolution Change (ARC) Support in VVC," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 14th Meeting: Geneva, CH, Mar. 19-27, 2019, Document: JVET-N0118, 8 Pages.

Liao R-L., et al., "CE4-Related: Simplification of Blending Weights and Motion Field Storage in Geometric Merge Mode," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 16th Meeting: Geneva, CH, Oct. 1-11, 2019, Document: JVET-P0304-v2, 7 Pages.

Non-Final Office Action for U.S. Appl. No. 18/360,608, mailed Nov. 4, 2024, 21 pages.

Notice of Allowance for Chinese Patent Application No. 202080066280.X dated Oct. 22, 2024, 10 pages.

Notice of Reasons for Refusal for Japanese Patent Application No. 2023-189119 dated Oct. 22, 2024, 28 pages.

Written Decision on Registration for Korean Application No. 10-2022-7009731, mailed Oct. 25, 2024, 6 pages.

Bross B., et al., "Versatile Video Coding (Draft 10)," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 19th Meeting: by Teleconference, Jun. 22, 2020-Jul. 1, 2020, Document: JVET-S2001-v5, 538 Pages.

Bross B., et al., "Versatile Video Coding (Draft 7)," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 16th Meeting: Geneva, CH, Oct. 1-11, 2019, Document: JVET-P2001, 464 Pages.

Bross B., et al., "Versatile Video Coding (Draft 7)," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 16th Meeting: Geneva, CH, Oct. 1-11, 2019, Document: JVET-P2001-v9, 495 Pages.

Bross B., et al., "Versatile Video Coding (Draft 8)," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 17th Meeting: Brussels, BE, Jan. 7-17, 2020, Document: JVET-Q2001-vD, 511 Pages.

Bross et al., "Versatile Video Coding (Draft 7)," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 16th Meeting: Geneva, CH, Oct. 1-11, 2019, Document: JVET-P2001-v4, 1951 Pages.

Document: JVET-O0182r1, He, Y., et al. "AHG12: On Picture and Sub-Picture Signaling," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 15th Meeting: Gothenburg, SE, 312 Jul. 2019, 6 pages.

Document: JVET-P0590-v2, Seregin, V., et al., "AHG8: Scaling window for scaling ratio derivation," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 16th Meeting: Geneva, CH, 111 Oct. 2019, 4 pages.

Document: JVET-P2001-v2, Bross, B., et al., "Versatile Video Coding (Draft 7)," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC29/WG 11 16th Meeting: Geneva, CH, Oct. 1-11, 2019, 464 pages.

European Office Action from European Patent Application No. 20866741.0 dated Sep. 30, 2024, 7 pages.

Extended European Search Report for European Application No. 20873315.4, mailed Nov. 7, 2022, 11 pages.

Gao H., et al., "CE4: CE4-1.1, CE4-1.2 and CE4-1.14: Geometric Merge Mode (GEO)," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 16th Meeting: Geneva, CH, Oct. 1-11, 2019, Document: JVET-P0068-v1, 508 Pages.

Hannuksela M.M., "Level Limits for No. of Luma Samples, Tiles, and Subpictures," Joint Video Exploration Team of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 16th Meeting: Geneva, Oct. 1-11, 2019, Document: JVET-P0099, 2 Pages, XP030216237, Sep. 24, 2019.

Ikeda M., et al., "Level Restrictions on Maximum Tile Width for Line Buffer Reduction," The Joint Video Exploration Team of ISO/IEC JTC1/SC29/WG11 and ITU-T SG.16, 16. JVET Meeting: Geneva, Oct. 1-11, 2019, Document: JVET-P0295-v2, 3 pages, XP030216851, (Sep. 24, 2019).

International Search Report and Written Opinion for International Application No. PCT/CN2020/140045, mailed Mar. 25, 2021, 8 Pages.

Liu H., et al., "CE2: Adaptive Motion Vector Resolution for Affine Inter Mode (Test 2.1.2)," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 13th Meeting: Marrakech, MA, Jan. 9-18, 2019, Document: JVET-M0246 r1, 13 Pages.

Luo J.D., "CE2-Related: Prediction Refinement with Optical Flow for Affine Mode," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 14th Meeting: Geneva, CH, Mar. 19-27, 2019, Document: JVET-N0236, 64 Pages.

Vietnamese Office Action from Vietnamese Patent Application No. 1-2022-02117 dated Sep. 9, 2024, 4 pages.

Zhang K., et al., "Non-CE1: A Fix on Reference Sample Offset for RPR," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 16th Meeting: Geneva, CH, Oct. 1-11, 2019, Document: JVET-P0381-v1, 5 Pages.

Bross B., et al., "Versatile Video Coding (Draft 10)," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, JVET-S2001 (version 17), 19th Meeting: by Teleconference, Sep. 4, 2020, pp. 45-48, 116-123, 287-295.

Bross B., et al., "Versatile Video Coding (Draft 6)," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 15th Meeting: Gothenburg, SE, Jul. 3-12, 2019, Document: JVET-O2001, 456 Page, Retrieved from URL: http://phenix.it-sudparis.eu/jvet/doc_end_user/documents/15_Gothenburg/wg11/JVET-O2001-v14.zip.

Notice of Reasons for Refusal for Japanese Application No. 2023-127153, mailed Oct. 1, 2024, 5 pages.

Vietnamese Office Action from Vietnamese Patent Application No. 1-2022-02350 dated Sep. 9, 2024, 4 pages.

Vietnamese Office Action from Vietnamese Patent Application No. 1-2022-04112 dated Aug. 28, 2024, 4 pages.

Vietnamese Office Action from Vietnamese Patent Application No. 1-2022-01792 dated Aug. 30, 2024, 4 pages.

Communication Pursuant to Article 94(3) for European Application No. 20906207.4, mailed Feb. 12, 2025, 3 Pages.

First Office Action for Chinese Application No. 2020800071694.1, mailed Dec. 30, 2024, 21 pages.

First Office Action for Chinese Application No. 202080071644.3, mailed Dec. 31, 2024, 20 pages.

Hearing Notice for Indian Patent Application No. 202247015422, mailed Dec. 27, 2024, 2 pages.

Hendry, et al., AHG19: "Adaptive Resolution Change (ARC) Support in VVC," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 14th Meeting: Geneva, CH, Mar. 19-27, 2019, Document: JVET-N0118-v3, 8 Pages.

Non-Final Office Action for U.S. Appl. No. 18/312,732, mailed Jan. 31, 2025, 79 Pages.

Notice of Allowance for Vietnamese Patent Application No. 1-2022-01792, dated Feb. 4, 2025, 2 page.

Notice of Final Rejection for Korean Application No. 10-2022-7006966, mailed Jan. 24, 2025, 9 Pages.

Yang et al., "Sub-block-based Motion Derivation and Inter Prediction Refinement in Versatile Video Coding Standard," IEEE Transaction on Circuits and Systems for Video Technology, vol. 31, No. 10, Oct. 2021, 16 pages.

* cited by examiner

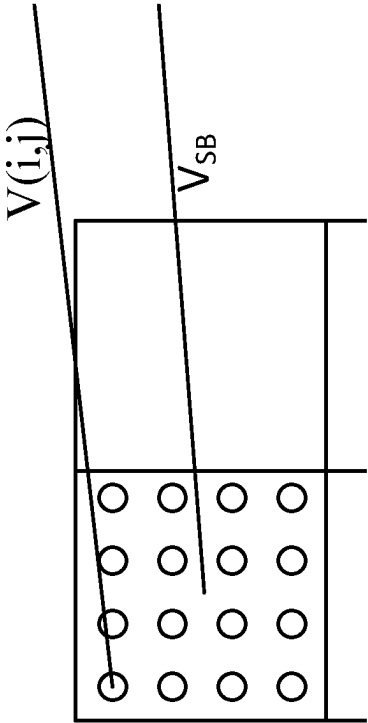


FIG. 1

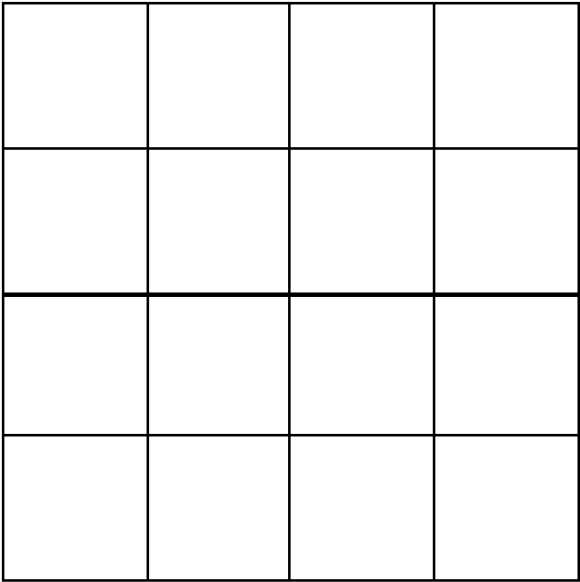


FIG. 2

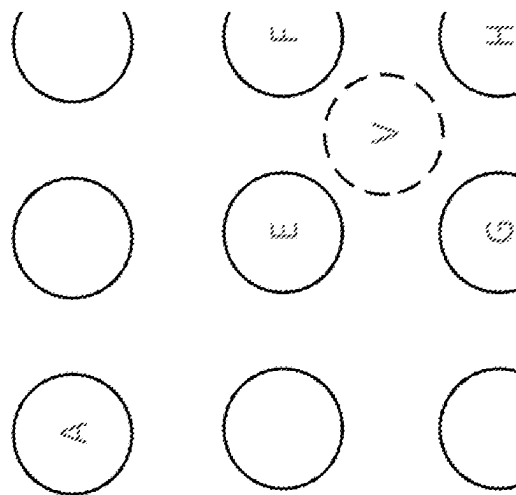


FIG. 3A

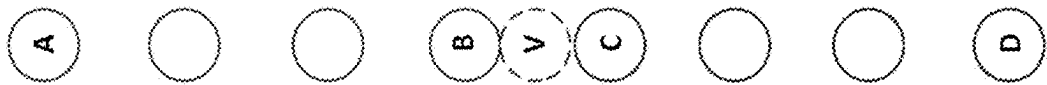


FIG. 3C



FIG. 3B

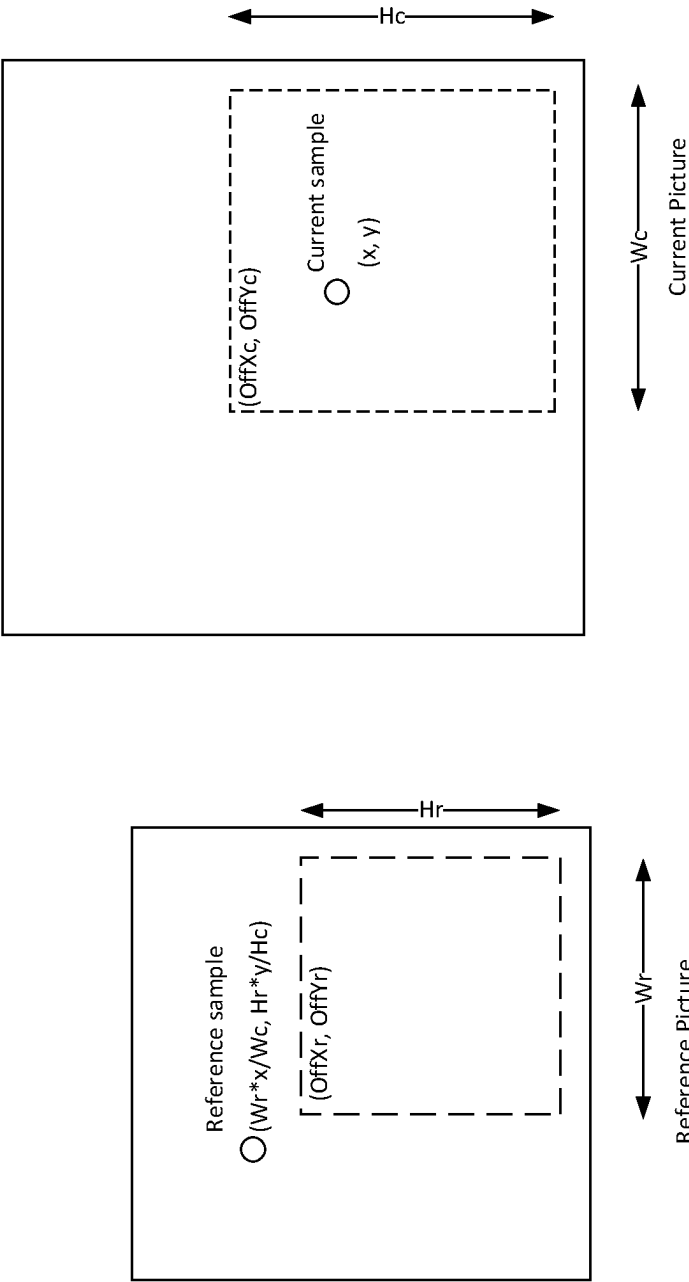


FIG. 4A

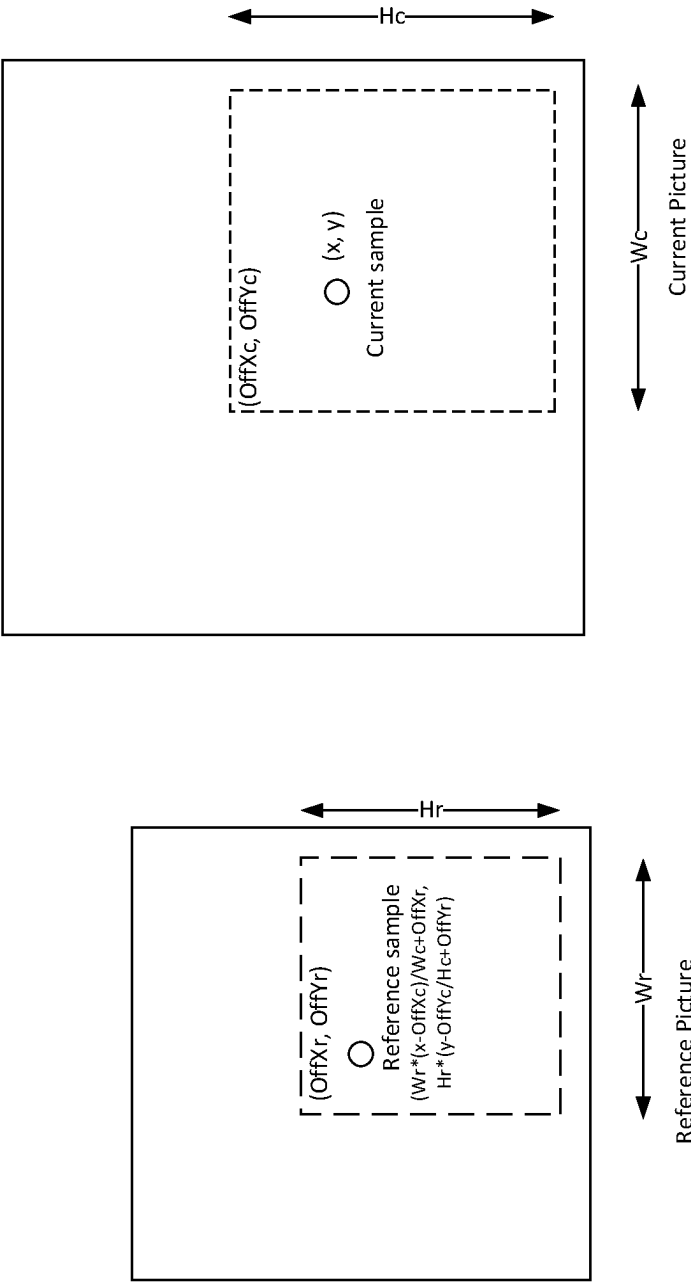


FIG. 4B

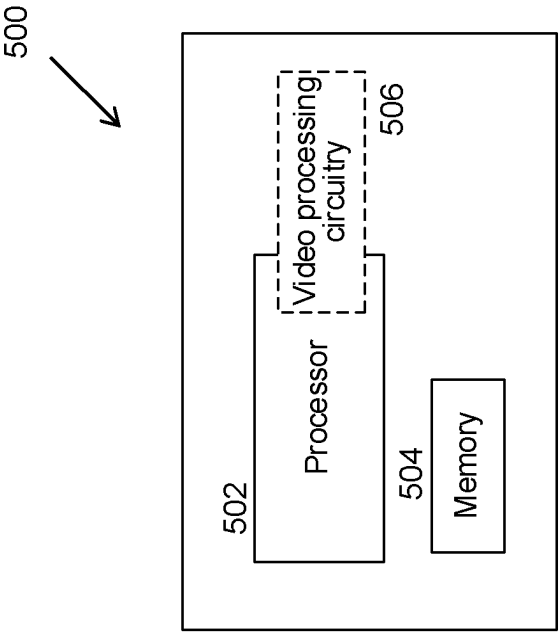


FIG. 5

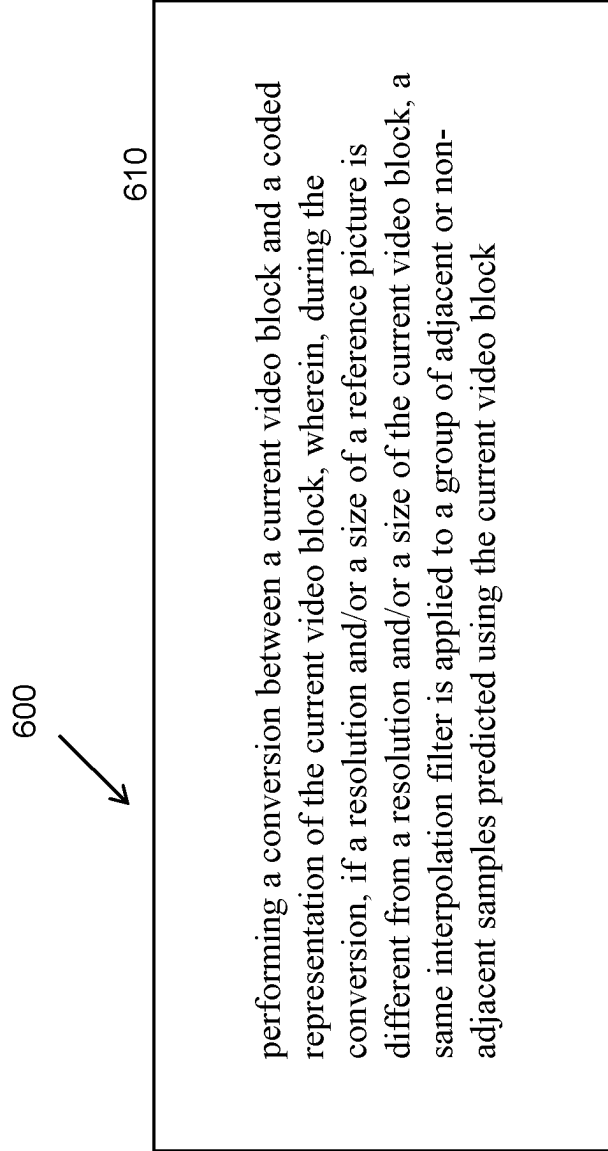


FIG. 6

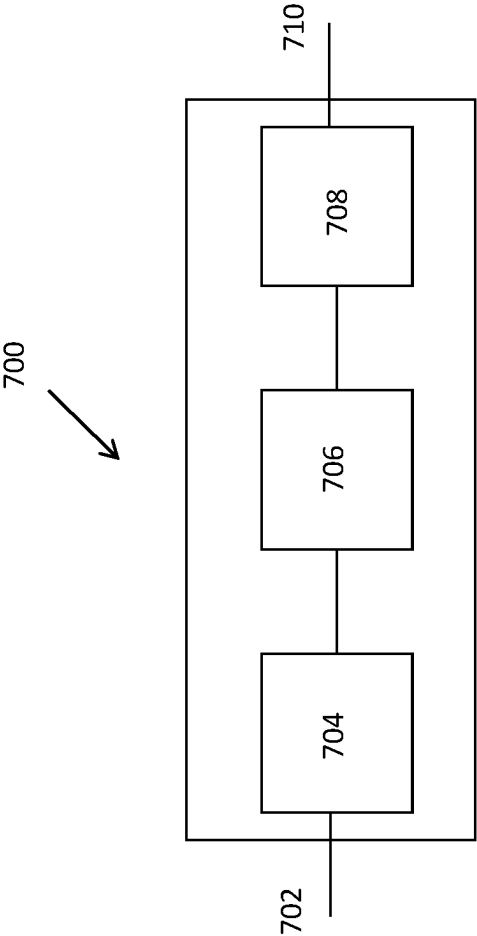


FIG. 7

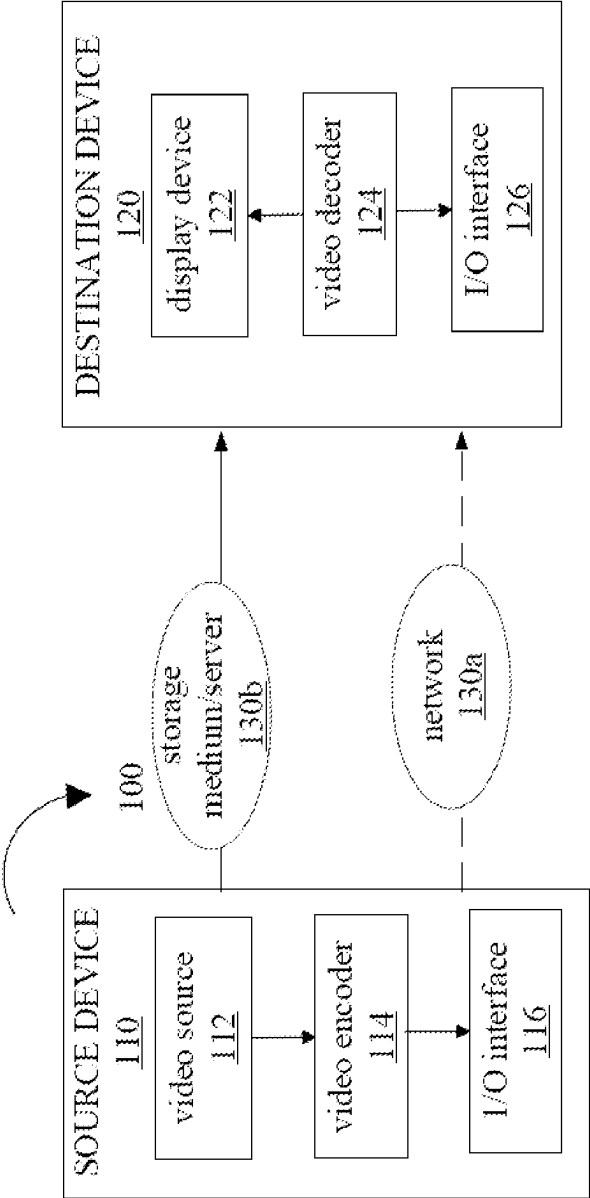


FIG. 8

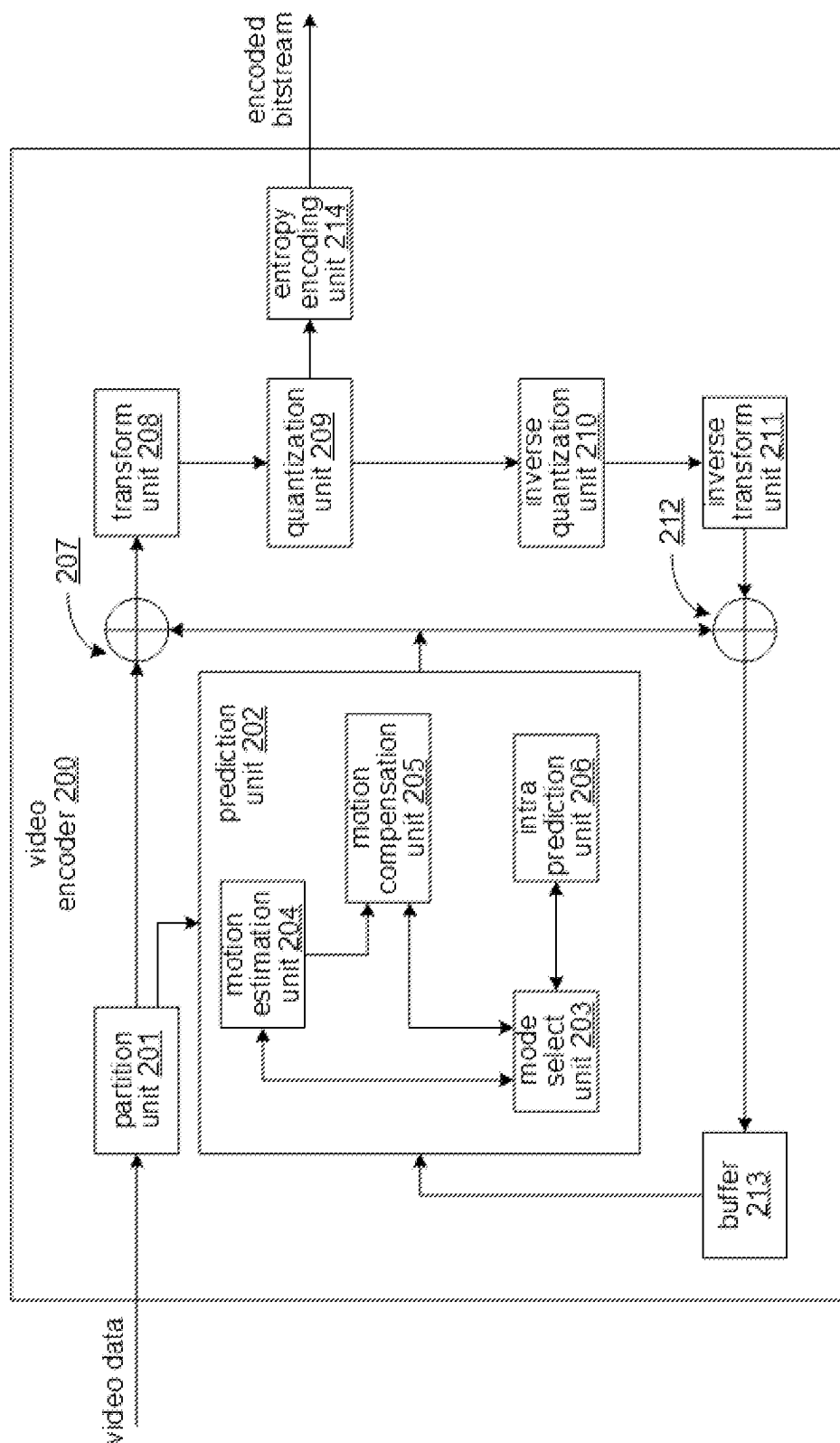


FIG. 9

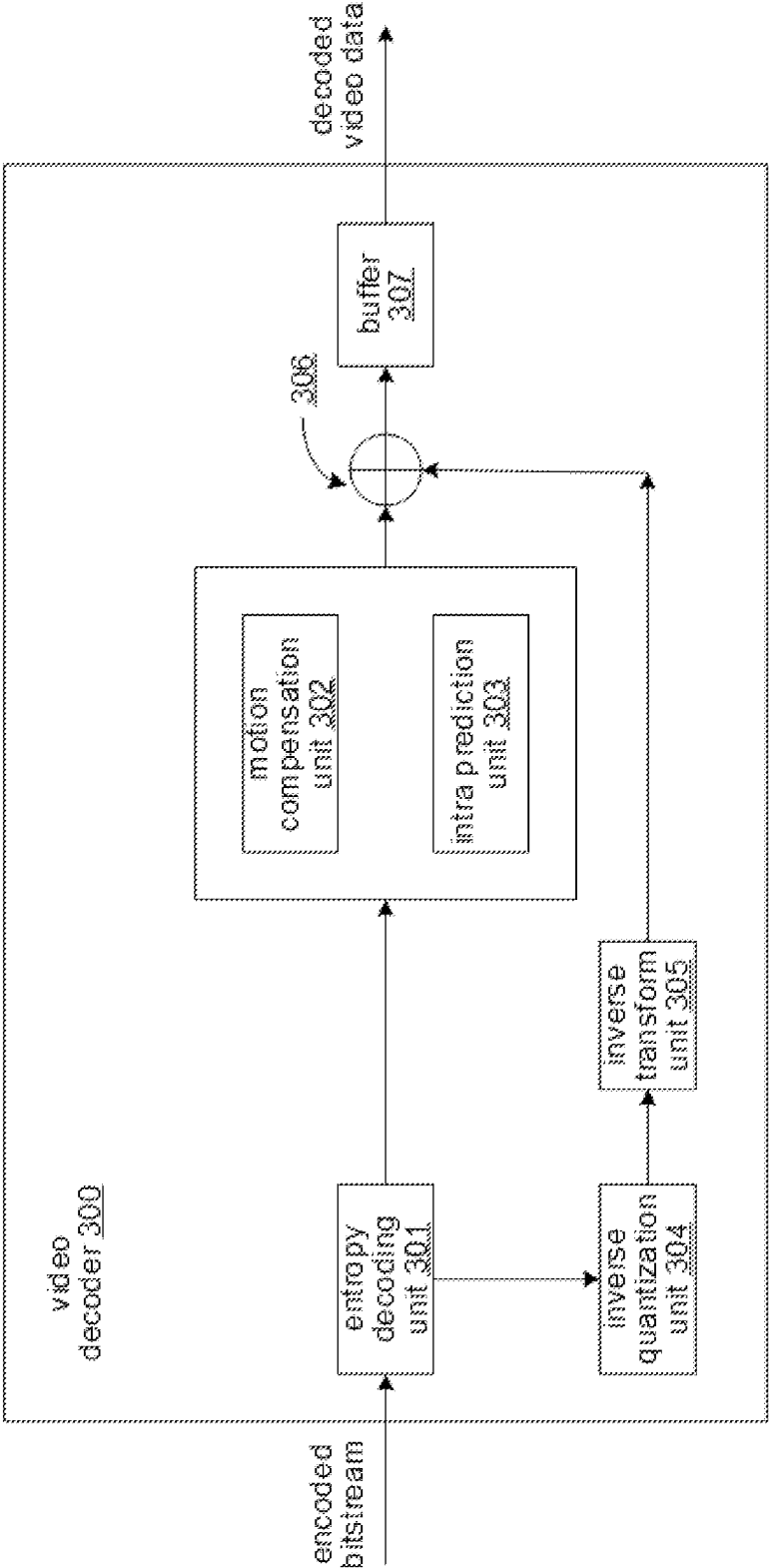
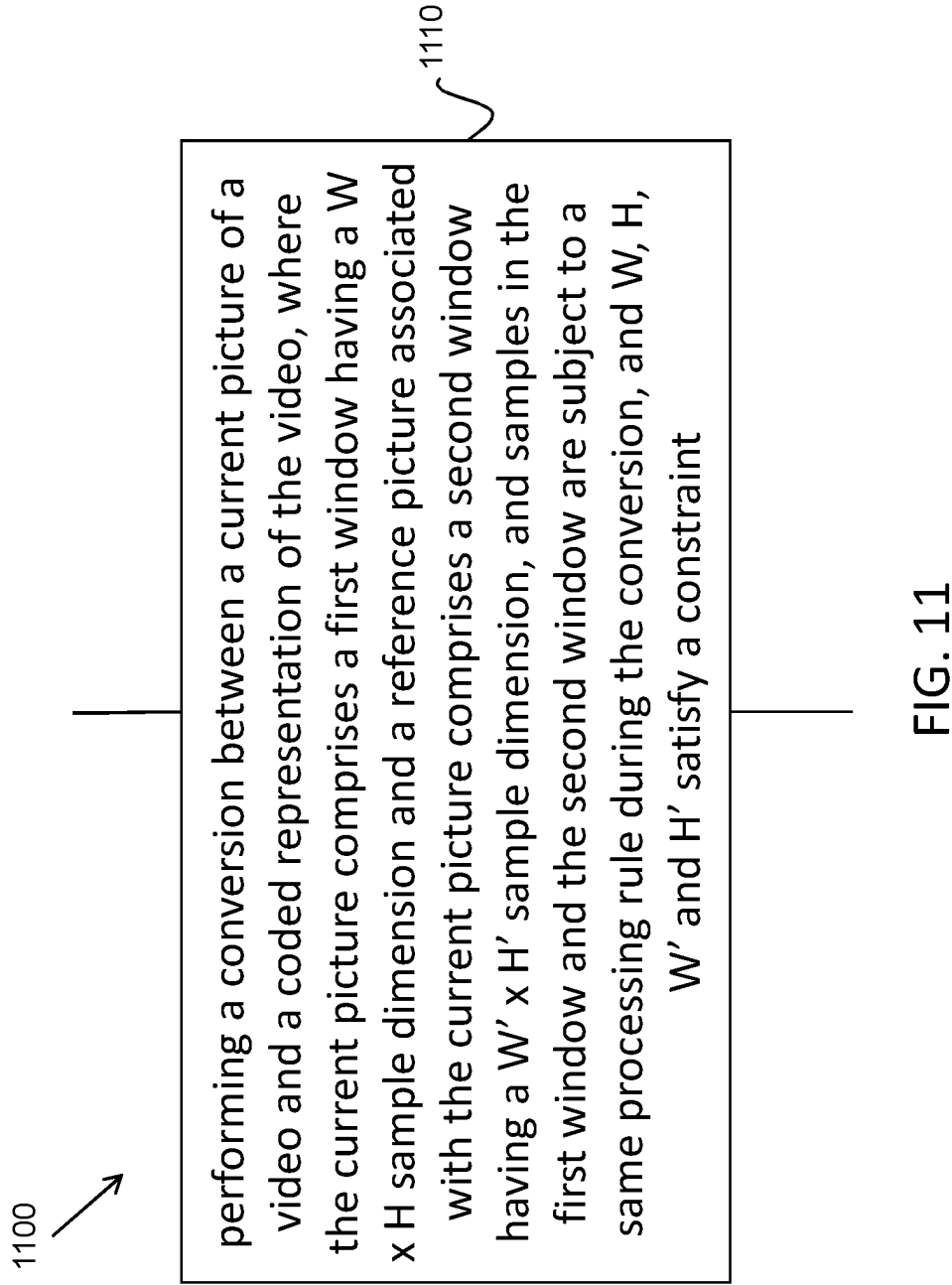


FIG. 10



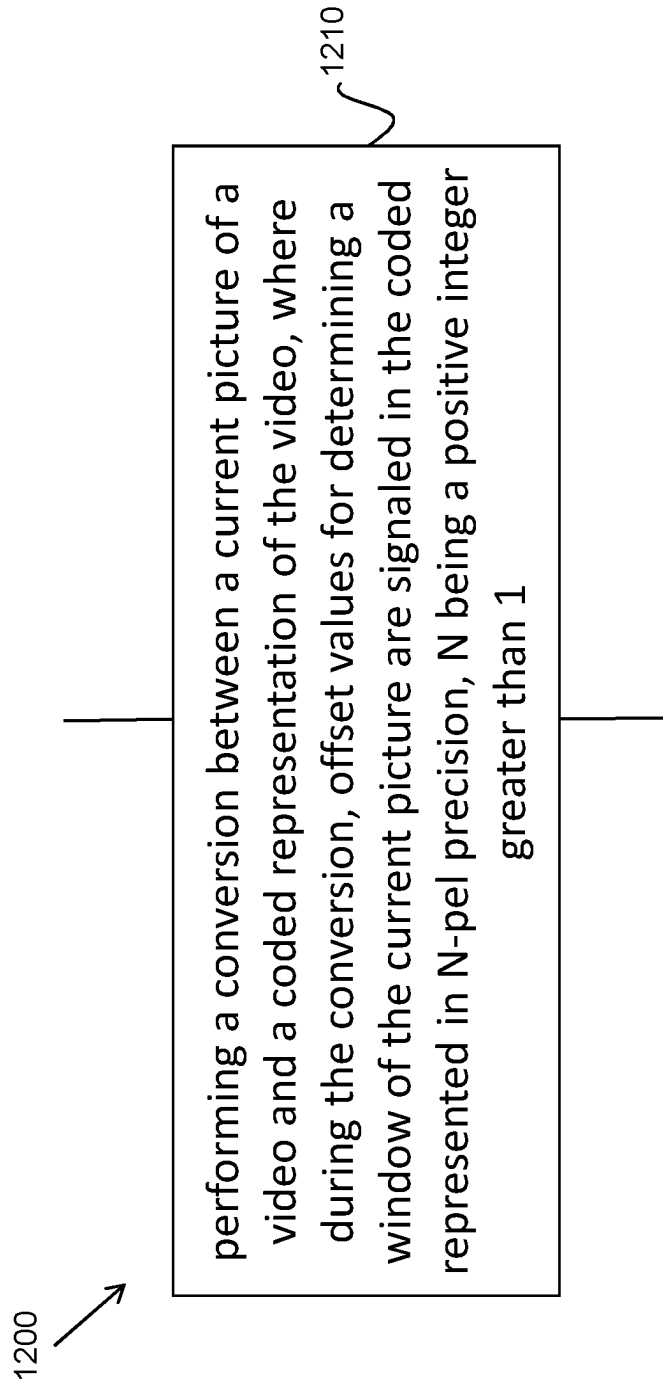
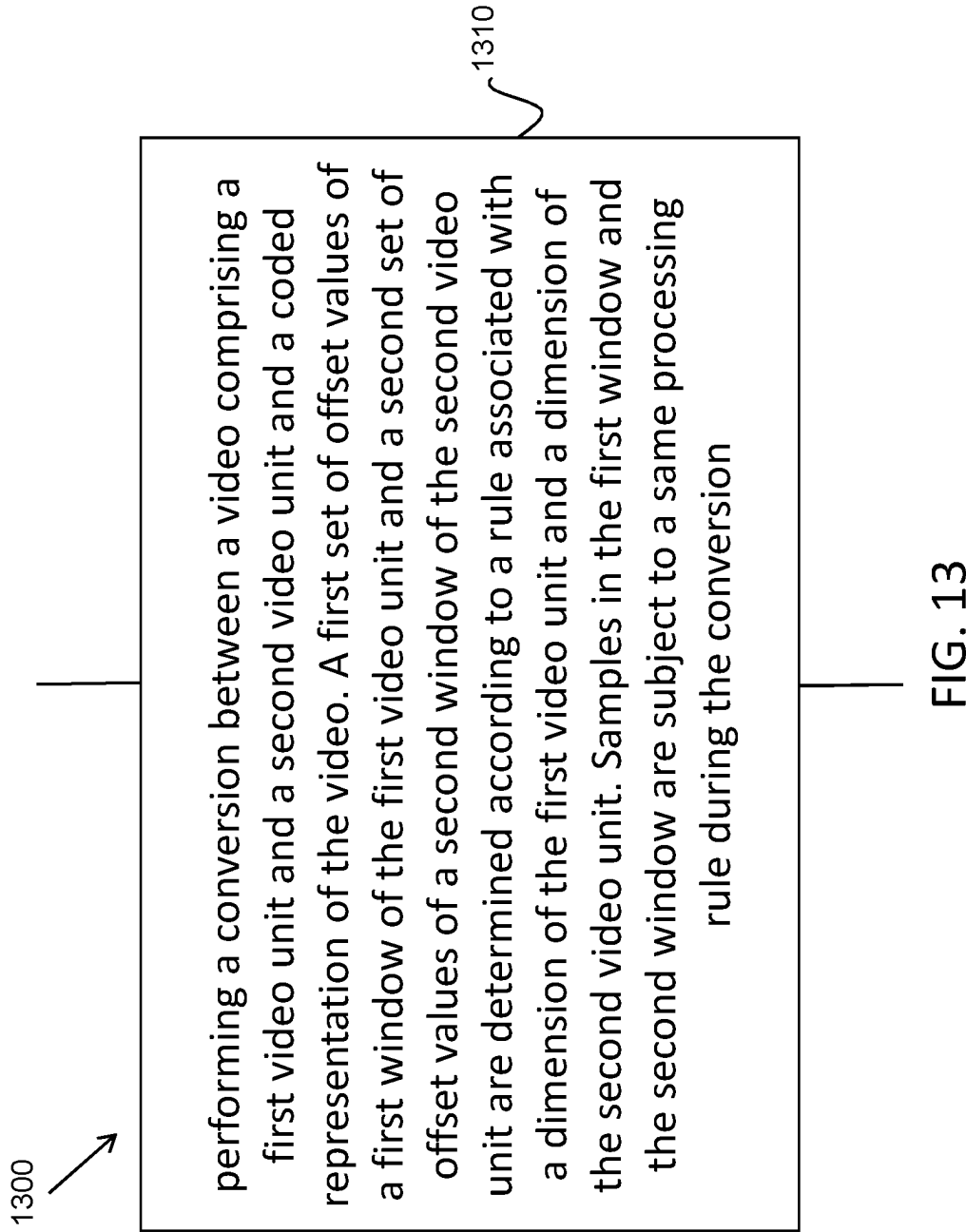


FIG. 12



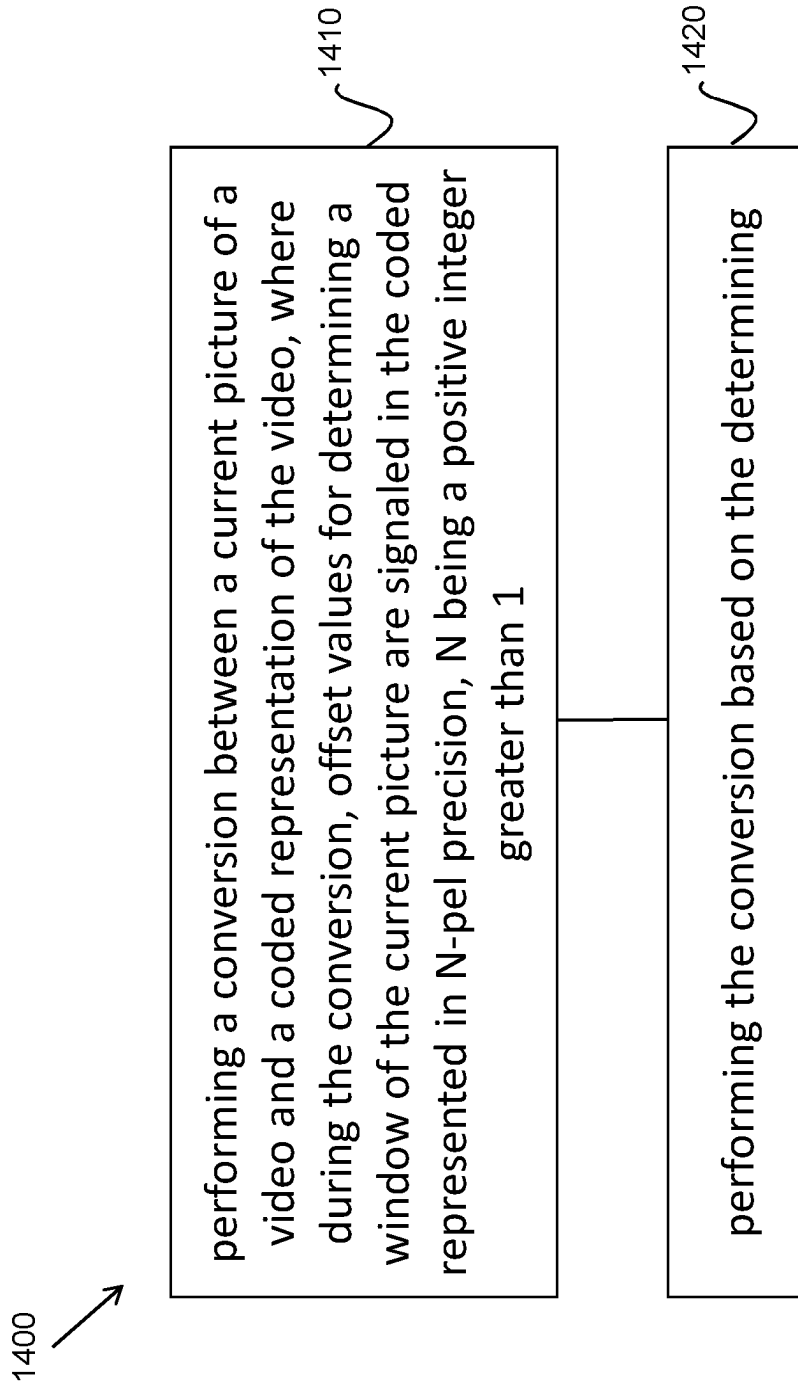


FIG. 14

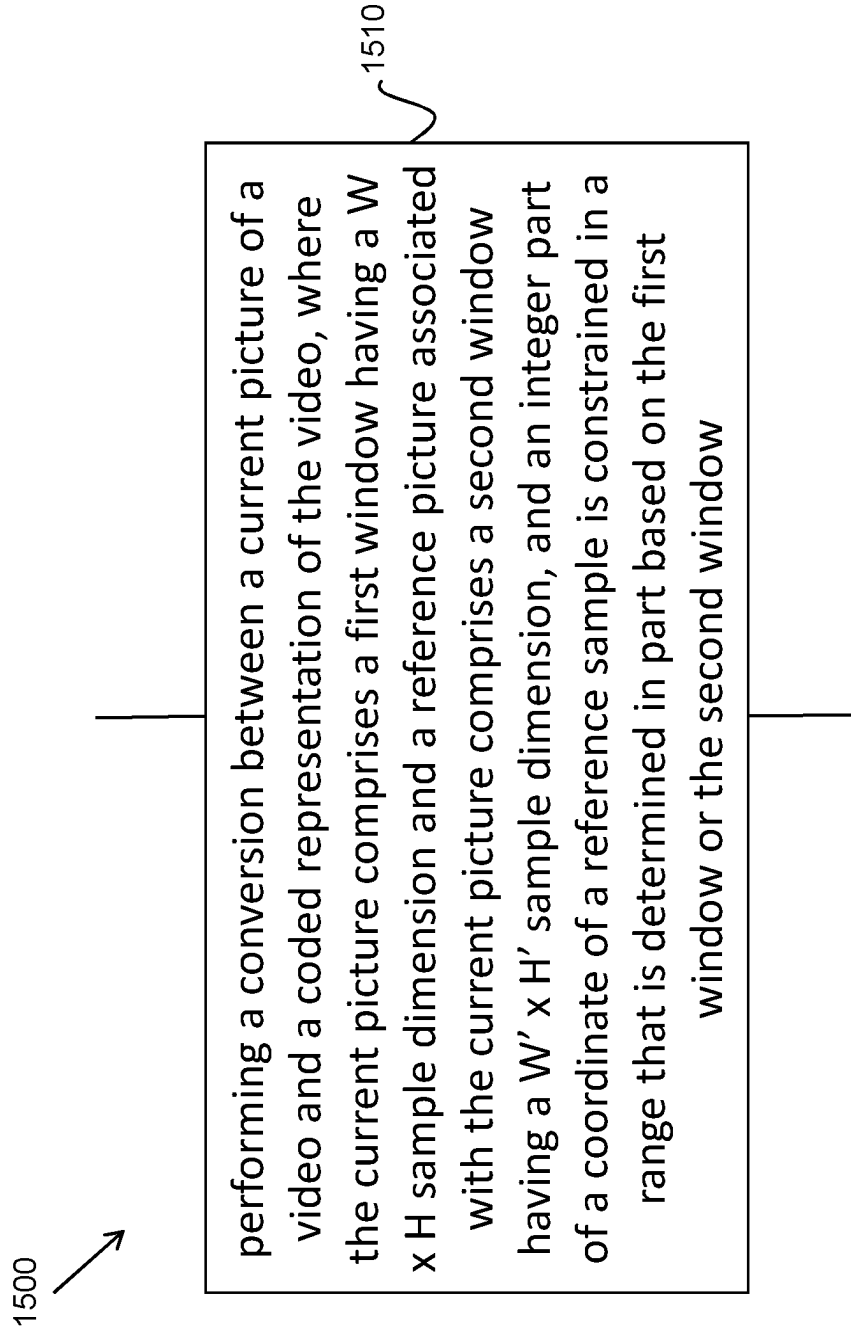


FIG. 15

1

DERIVING REFERENCE SAMPLE POSITIONS IN VIDEO CODING

CROSS REFERENCE TO RELATED APPLICATIONS

This application is a continuation of U.S. application Ser. No. 17/699,021, filed on Mar. 18, 2022, which is a continuation of International Patent Application No. PCT/CN2020/116469, filed on Sep. 21, 2020, which claims the priority to and benefits of International Patent Application No. PCT/CN2019/106671, filed on Sep. 19, 2019, International Patent Application No. PCT/CN2019/107082, filed on Sep. 20, 2019 and International Patent Application No. PCT/CN2019/109837, filed on Oct. 5, 2019. All the aforementioned patent applications are hereby incorporated by reference in their entireties.

TECHNICAL FIELD

The present disclosure relates to video coding techniques, devices and systems.

BACKGROUND

Currently, efforts are underway to improve the performance of current video codec technologies to provide better compression ratios or provide video coding and decoding schemes that allow for lower complexity or parallelized implementations. Industry experts have recently proposed several new video coding tools and tests are currently underway for determining their effectivity.

SUMMARY

Devices, systems, and methods related to digital video coding, and specifically, to management of motion vectors are described. The described methods may be applied to existing video coding standards (e.g., High Efficiency Video Coding (HEVC) or Versatile Video Coding (VVC)) and future video coding standards or video codecs.

In one representative aspect, the disclosed technology may be used to provide a method for video processing. This method includes performing a conversion between a current picture of a video and a coded representation of the video. The current picture comprises a first window having a $W \times H$ sample dimension, where W and H are integers. A reference picture associated with the current picture comprises a second window having a $W' \times H'$ sample dimension, where W' and H' are integers. Samples in the first window and the second window are subject to a same processing rule during the conversion and W , H , W' and H' satisfy a constraint.

In another representative aspect, the disclosed technology may be used to provide a method for video processing. This method includes performing a conversion between a current picture of a video and a coded representation of the video. During the conversion, offset values for determining a window of the current picture are signaled in the coded represented in N-pel precision, N being a positive integer greater than 1.

In another representative aspect, the disclosed technology may be used to provide a method for video processing. This method includes performing a conversion between a video comprising a first video unit and a second video unit and a coded representation of the video. A first set of offset values of a first window of the first video unit and a second set of offset values of a second window of the second video unit

2

are determined according to a rule associated with a dimension of the first video unit and a dimension of the second video unit, where samples in the first window and the second window are subject to a same processing rule during the conversion.

In another representative aspect, the disclosed technology may be used to provide a method for video processing. This method includes determining, for a conversion between a current picture of a video and a coded representation of the video, a position of a reference sample in a reference picture that is associated with the current picture based on a top-left position of a window of a picture, where the picture comprises at least the current picture or the reference picture. The window is subject to a processing rule during the conversion. The method also includes performing the conversion based on the determining.

In another representative aspect, the disclosed technology may be used to provide a method for video processing. This method includes performing a conversion between a current picture of a video and a coded representation of the video. The current picture comprises a first window having a $W \times H$ sample dimension, where W and H are integers. A reference picture associated with the current picture comprises a second window having a $W' \times H'$ sample dimension, where W' and H' are integers and a top-left position of the second window is denoted as (X_0, Y_0) . Samples in the first window and the second window are subject to a same processing rule during the conversion. An integer part of a coordinate of a reference sample is constrained in a range that is determined in part based on the first window or the second window.

In another representative aspect, the disclosed technology may be used to provide a method for video processing. This method includes performing a conversion between a current video block and a coded representation of the current video block, wherein, during the conversion, if a resolution and/or a size of a reference picture is different from a resolution and/or a size of the current video block, a same interpolation filter is applied to a group of adjacent or non-adjacent samples predicted using the current video block.

In another representative aspect, the disclosed technology may be used to provide another method for video processing. This method includes performing a conversion between a current video block and a coded representation of the current video block, wherein, during the conversion, if a resolution and/or a size of a reference picture is different from a resolution and/or a size of the current video block, wherein blocks predicted using the current video block are only allowed to use integer-valued motion information related to the current block.

In another representative aspect, the disclosed technology may be used to provide another method for video processing. This method includes performing a conversion between a current video block and a coded representation of the current video block, wherein, during the conversion, if a resolution and/or a size of a reference picture is different from a resolution and/or a size of the current video block, an interpolation filter is applied to derive blocks predicted using the current video block, and wherein the interpolation filter is selected based on a rule.

In another representative aspect, the disclosed technology may be used to provide another method for video processing. This method includes performing a conversion between a current video block and a coded representation of the current video block, wherein, during the conversion, if a resolution and/or a size of a reference picture is different from a resolution and/or a size of the current video block, selectively applying a deblocking filter, wherein a strength

of the deblocking filter set in accordance with a rule related to the resolution and/or the size of the reference picture relative to the resolution and/or the size of the current video block.

In another representative aspect, the disclosed technology may be used to provide another method for video processing. This method includes performing a conversion between a current video block and a coded representation of the current video block, wherein, during the conversion, a reference picture of the current video block is resampled in accordance with a rule based on dimensions of the current video block.

In another representative aspect, the disclosed technology may be used to provide another method for video processing. This method includes performing a conversion between a current video block and a coded representation of the current video block, wherein, during the conversion, a use of a coding tool on the current video block is selectively enabled or disabled depending on a resolution/size of a reference picture of the current video block relative to a resolution/size of the current video block.

In another representative aspect, the disclosed technology may be used to provide another method for video processing. This method includes performing a conversion between multiple video blocks and coded representations of the multiple video blocks, wherein, during the conversion, a first conformance window is defined for a first video block and a second conformance window for a second video block, and wherein a ratio of a width and/or a height of the first conformance window to the second conformance window is in accordance with a rule based at least on a conformance bitstream.

In another representative aspect, the disclosed technology may be used to provide another method for video processing. This method includes performing a conversion between multiple video blocks and coded representations of the multiple video blocks, wherein, during the conversion, a first conformance window is defined for a first video block and a second conformance window for a second video block, and wherein a ratio of a width and/or a height of the first conformance window to the second conformance window is in accordance with a rule based at least on a conformance bitstream.

Further, in a representative aspect, an apparatus in a video system comprising a processor and a non-transitory memory with instructions thereon is disclosed. The instructions upon execution by the processor, cause the processor to implement any one or more of the disclosed methods.

In one representative aspect, a video decoding apparatus comprising a processor configured to implement a method recited herein is disclosed.

In one representative aspect, a video encoding apparatus comprising a processor configured to implement a method recited herein is disclosed.

Also, a computer program product stored on a non-transitory computer readable media, the computer program product including program code for carrying out any one or more of the disclosed methods is disclosed.

The above and other aspects and features of the disclosed technology are described in greater detail in the drawings, the description and the claims.

BRIEF DESCRIPTION OF THE DRAWINGS

FIG. 1 shows an example of sub-block motion vector (VSB) and motion vector difference.

FIG. 2 shows an example of a 16×16 video block divided into 16 4×4 regions.

FIG. 3A shows an example of a specific position in a sample.

FIG. 3B shows another example of a specific position in a sample.

FIG. 3C shows yet another example of a specific position in a sample.

FIG. 4A shows an example of positions of the current sample and its reference sample.

FIG. 4B shows another example of positions of the current sample and its reference sample.

FIG. 5 is a block diagram of an example of a hardware platform for implementing a visual media decoding or a visual media encoding technique described in the present disclosure.

FIG. 6 shows a flowchart of an example method for video coding.

FIG. 7 is a block diagram of an example video processing system in which disclosed techniques may be implemented.

FIG. 8 is a block diagram that illustrates an example video coding system.

FIG. 9 is a block diagram that illustrates an encoder in accordance with some embodiments of the present disclosure.

FIG. 10 is a block diagram that illustrates a decoder in accordance with some embodiments of the present disclosure.

FIG. 11 is a flowchart representation of a method of video processing in accordance with the present technology.

FIG. 12 is a flowchart representation of another method of video processing in accordance with the present technology.

FIG. 13 is a flowchart representation of another method of video processing in accordance with the present technology.

FIG. 14 is a flowchart representation of another method of video processing in accordance with the present technology.

FIG. 15 is a flowchart representation of another method of video processing in accordance with the present technology.

DETAILED DESCRIPTION

1. Video Coding in HEVC/H.265

Video coding standards have evolved primarily through the development of the well-known International Telecommunication Union (ITU) telecommunication standardization sector (ITU-T) and International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC) standards. The ITU-T produced H.261 and H.263, ISO/IEC produced Moving Picture Experts Group (MPEG)-1 and MPEG-4 Visual, and the two organizations jointly produced the H.262/MPEG-2 Video and H.264/MPEG-4 Advanced Video Coding (AVC) and H.265/HEVC standards. Since H.262, the video coding standards are based on the hybrid video coding structure wherein temporal prediction plus transform coding are utilized. To explore the future video coding technologies beyond HEVC, Joint Video Exploration Team (JVET) was founded by Video Coding Experts Group (VCEG) and MPEG jointly in 2015. Since then, many new methods have been adopted by JVET and put into the reference software named Joint Exploration Model (JEM). In April 2018, the Joint Video Expert Team (JVET) between VCEG (Q6/16) and ISO/IEC Joint Technical Committee (JTC1) SC29/WG11 (MPEG) was created

to work on the VVC standard targeting at 50% bitrate reduction compared to HEVC.

2. Overview

2.1. Adaptive Resolution Change (ARC)

AVC and HEVC does not have the ability to change resolution without having to introduce an instantaneous decoding refresh (IDR) or intra random access point (IRAP) picture; such ability can be referred to as adaptive resolution change (ARC). There are use cases or application scenarios that would benefit from an ARC feature, including the following:

Rate adaption in video telephony and conferencing: For adapting the coded video to the changing network conditions, when the network condition gets worse so that available bandwidth becomes lower, the encoder may adapt to it by encoding smaller resolution pictures. Currently, changing picture resolution can be done only after an IRAP picture; this has several issues. An IRAP picture at reasonable quality will be much larger than an inter-coded picture and will be correspondingly more complex to decode: this costs time and resource. This is a problem if the resolution change is requested by the decoder for loading reasons. It can also break low-latency buffer conditions, forcing an audio re-sync, and the end-to-end delay of the stream will increase, at least temporarily. This can give a poor user experience.

Active speaker changes in multi-party video conferencing: For multi-party video conferencing, it is common that the active speaker is shown in bigger video size than the video for the rest of conference participants. When the active speaker changes, picture resolution for each participant may also need to be adjusted. The need to have ARC feature becomes more important when such change in active speaker happens frequently.

Fast start in streaming: For streaming application, it is common that the application would buffer up to certain length of decoded picture before start displaying. Starting the bitstream with smaller resolution would allow the application to have enough pictures in the buffer to start displaying faster.

Adaptive stream switching in streaming: The Dynamic Adaptive Streaming over Hypertext Transfer Protocol (HTTP) (DASH) specification includes a feature named @mediaStreamStructureId. This enables switching between different representations at open-group of pictures (GOP) random access points with non-decodable leading pictures, e.g., clean random access (CRA) pictures with associated random access skipped leading (RASL) pictures in HEVC. When two different representations of the same video have different bitrates but the same spatial resolution while they have the same value of @mediaStreamStructureId, switching between the two representations at a CRA picture with associated RASL pictures can be performed, and the RASL pictures associated with the switching-at CRA pictures can be decoded with acceptable quality hence enabling seamless switching. With ARC, the @mediaStreamStructureId feature would also be usable for switching between DASH representations with different spatial resolutions.

ARC is also known as Dynamic resolution conversion.

ARC may also be regarded as a special case of Reference Picture Resampling (RPR) such as H.263 Annex P.

2.2. Reference Picture Resampling in H.263 Annex P

This mode describes an algorithm to warp the reference picture prior to its use for prediction. It can be useful for resampling a reference picture having a different source

format than the picture being predicted. It can also be used for global motion estimation, or estimation of rotating motion, by warping the shape, size, and location of the reference picture. The syntax includes warping parameters to be used as well as a resampling algorithm. The simplest level of operation for the reference picture resampling mode is an implicit factor of 4 resampling as only a finite impulse response (FIR) filter needs to be applied for the upsampling and downsampling processes. In this case, no additional signaling overhead is required as its use is understood when the size of a new picture (indicated in the picture header) is different from that of the previous picture.

2.3. Conformance Window in VVC

Conformance window in VVC defines a rectangle. Samples inside the conformance window belongs to the image of interest. Samples outside the conformance window may be discarded when output.

When conformance window is applied, the scaling ration in RPR is derived based on conformance windows.

Picture Parameter Set RBSP Syntax

pic_parameter_set_rbsp() {	Descriptor
pps_pic_parameter_set_id	ue(v)
pps_seq_parameter_set_id	ue(v)
pic_width_in_luma_samples	ue(v)
pic_height_in_luma_samples	ue(v)
<u>conformance_window_flag</u>	<u>u(1)</u>
<u>if(conformance_window_flag) {</u>	
<u>conf_win_left_offset</u>	<u>ue(v)</u>
<u>conf_win_right_offset</u>	<u>ue(v)</u>
<u>conf_win_top_offset</u>	<u>ue(v)</u>
<u>conf_win_bottom_offset</u>	<u>ue(v)</u>
}	

pic_width_in_luma_samples specifies the width of each decoded picture referring to the PPS in units of luma samples. pic_width_in_luma_samples shall not be equal to 0, shall be an integer multiple of Max(8, MinCbSizeY), and shall be less than or equal to pic_width_max_in_luma_samples.

When subpics_present_flag is equal to 1, the value of pic_width_in_luma_samples shall be equal to pic_width_max_in_luma_samples.

pic_height_in_luma_samples specifies the height of each decoded picture referring to the PPS in units of luma samples. pic_height_in_luma_samples shall not be equal to 0 and shall be an integer multiple of Max(8, MinCbSizeY), and shall be less than or equal to pic_height_max_in_luma_samples.

When subpics_present_flag is equal to 1, the value of pic_height_in_luma_samples shall be equal to pic_height_max_in_luma_samples.

Let refPicWidthInLumaSamples and refPicHeightInLumaSamples be the pic_width_in_luma_samples and pic_height_in_luma_samples, respectively, of a reference picture of a current picture referring to this PPS. Is a requirement of bitstream conformance that all of the following conditions are satisfied:

- pic_width_in_luma_samples * 2 shall be greater than or equal to refPicWidthInLumaSamples.
- pic_height_in_luma_samples * 2 shall be greater than or equal to refPicHeightInLumaSamples.

- *pic_width_in_luma_samples shall be less than or equal to refPicWidthInLumaSamples * 8.*
- *pic_height_in_luma_samples shall be less than or equal to refPicHeightInLumaSamples * 8.*

conformance_window_flag equal to 1 indicates that the conformance cropping window offset parameters follow next in the SPS. conformance_window_flag equal to 0 indicates that the conformance cropping window offset parameters are not present.

conf_win_left_offset, conf_win_right_offset, conf_win_top_offset, and conf_win_bottom_offset specify the samples of the pictures in the CVS that are output from the decoding process, in terms of a rectangular region specified in picture coordinates for output. When conformance_window_flag is equal to 0, the values of conf_win_left_offset, conf_win_right_offset, conf_win_top_offset, and conf_win_bottom_offset are inferred to be equal to 0.

The conformance cropping window contains the luma samples with horizontal picture coordinates from SubWidthC*conf_win_left_offset to pic_width_in_luma_samples-(SubWidthC*conf_win_right_offset+1) and vertical picture coordinates from SubHeightC*conf_win_top_offset to pic_height_in_luma_samples-(SubHeightC*conf_win_bottom_offset+1), inclusive.

The value of SubWidthC*(conf_win_left_offset+conf_win_right_offset) shall be less than pic_width_in_luma_samples, and the value of SubHeightC*(conf_win_top_offset+conf_win_bottom_offset) shall be less than pic_height_in_luma_samples.

The variables PicOutputWidthL and PicOutputHeightL are derived as follows:

$$\text{PicOutputWidthL} = \text{pic_width_in_luma_samples} - \text{SubWidthC} * (\text{conf_win_right_offset} + \text{conf_win_left_offset}) \quad (7-43)$$

$$\text{PicOutputHeightL} = \text{pic_height_in_pic_size_units} - \text{SubHeightC} * (\text{conf_win_bottom_offset} + \text{conf_win_top_offset}) \quad (7-44)$$

When ChromaArrayType is not equal to 0, the corresponding specified samples of the two chroma arrays are the samples having picture coordinates (x/SubWidthC, y/SubHeightC), where (x, y) are the picture coordinates of the specified luma samples.

NOTE – The conformance cropping window offset parameters are only applied at the output. All internal decoding processes are applied to the uncropped picture size.

Let ppsA and ppsB be any two PPSs referring to the same SPS. It is a requirement of bitstream conformance that, when ppsA and ppsB have the same the values of pic_width_in_luma_samples and pic_height_in_luma_samples, respectively, ppsA and ppsB shall have the same values of conf_win_left_offset, conf_win_right_offset, conf_win_top_offset, and conf_win_bottom_offset, respectively.

2.4. Reference Picture Resampling (RPR)

In some embodiments, ARC is also known as Reference Picture Resampling (RPR). With RPR, temporal motion vector prediction (TMVP) is disabled if the collocated picture has a different resolution to the current picture. Besides, Bi-directional Optical Flow (BDOF) and Decoder-Side Motion Vector Refinement (DMVR) are disabled when the reference picture has a different resolution to the current picture.

To handle the normal motion compensation (MC) when the reference picture has a different resolution than the current picture, the interpolation section is defined as below:

8.5.6.3 Fractional Sample Interpolation Process

8.5.6.3.1 General

Inputs to this process are:

- a luma location (xSb, ySb) specifying the top-left sample of the current coding subblock relative to the top-left luma sample of the current picture,
- a variable sbWidth specifying the width of the current coding subblock,
- a variable sbHeight specifying the height of the current coding subblock,
- a motion vector offset mvOffset,
- a refined motion vector refMvLX,
- the selected reference picture sample array refPicLX,
- the half sample interpolation filter index hpellIdx,
- the bi-directional optical flow flag bdofFlag,
- a variable cldx specifying the colour component index of the current block.

Outputs of this process are:

- an (sbWidth+brdExtSize)×(sbHeight+brdExtSize) array predSamplesLX of prediction sample values.

The prediction block border extension size brdExtSize is derived as follows:

$$\text{brdExtSize} = (\text{bdofFlag} | (\text{inter_affine_flag}[\text{xSb}][\text{ySb}] \& \& \text{sps_affine_prof_enabled_flag})) ? 2 : 0 \quad (8-752)$$

The variable fRefWidth is set equal to the PicOutputWidthL of the reference picture in luma samples.

The variable fRefHeight is set equal to PicOutputHeightL of the reference picture in luma samples.

The motion vector mvLX is set equal to (refMvLX-mvOffset).

If cldx is equal to 0, the following applies:

The scaling factors and their fixed-point representations are defined as

$$\text{hori_scale_fp} = ((\text{fRefWidth} < 14) + (\text{PicOutputWidthL} > 1)) / \text{PicOutputWidthL} \quad (8-753)$$

$$\text{vert_scale_fp} = ((\text{fRefHeight} < 14) + (\text{PicOutputHeightL} > 1)) / \text{PicOutputHeightL} \quad (8-754)$$

Let (xIntL, yIntL) be a luma location given in full-sample units and (xFracL, yFracL) be an offset given in 1/16-sample units. These variables are used only in this clause for specifying fractional-sample locations inside the reference sample arrays refPicLX.

The top-left coordinate of the bounding block for reference sample padding (xSbIntL, ySbIntL) is set equal to (xSb+(mvLX[0]>>4), ySb+(mvLX[1]>>4)).

For each luma sample location (xL=0 . . . sbWidth-1+brdExtSize, yL=0 . . . sbHeight-1+brdExtSize) inside the prediction luma sample array predSamplesLX, the corresponding prediction luma sample value predSamplesLX[xL][yL] is derived as follows: Let (refxSbL, refySbL) and (refxL, refyL) be luma locations pointed to by a motion vector (refMvLX[0], refMvLX[1]) given in 1/16-sample units. The variables refxSbL, refxL, refySbL, and refyL are derived as follows:

$$\text{refxSbL} = ((\text{xSb} < 4) + \text{refMvLX}[0]) * \text{hori_scale_fp} \quad (8-755)$$

$$\text{refxL} = ((\text{Sign}(\text{refxSb}) * ((\text{Abs}(\text{refxSb}) + 128) > 8) + \text{xL} * ((\text{hori_scale_fp} + 8) > 4)) + 32) > 6 \quad (8-756)$$

9

$\text{refySb}_L = ((ySb < 4) + \text{refMvLX}[1]) * \text{vert_scale_fp}$ (8-757)

$\text{refyL} = ((\text{Sign}(\text{refySb}) * ((\text{Abs}(\text{refySb}) + 128) >> 8) + yL * ((\text{vert_scale_fp} + 8) >> 4)) + 32) >> 6$ (8-758)

The variables $x\text{Int}_L$, $y\text{Int}_L$, $x\text{Frac}_L$ and $y\text{Frac}_L$ are derived as follows:

$x\text{Int}_L = \text{refx}_L >> 4$ (8-759)

$y\text{Int}_L = \text{refy}_L >> 4$ (8-760)

$x\text{Frac}_L = \text{refx}_L \& 15$ (8-761)

$y\text{Frac}_L = \text{refy}_L \& 15$ (8-762)

If bdoFlag is equal to TRUE or ($\text{sps_affine_prof_enabled_flag}$ is equal to TRUE and $\text{inter_affine_flag}[xSb][ySb]$ is equal to TRUE), and one or more of the following conditions are true, the prediction luma sample value $\text{predSamplesLX}[x_L][y_L]$ is derived by invoking the luma integer sample fetching process as specified in clause 8.5.6.3.3 with $(x\text{Int}_L + (x\text{Frac}_L >> 3) - 1)$, $y\text{Int}_L + (y\text{Frac}_L >> 3) - 1$ and refPicLX as inputs.

x_L is equal to 0.

x_L is equal to $\text{sbWidth} + 1$.

y_L is equal to 0.

y_L is equal to $\text{sbHeight} + 1$.

Otherwise, the prediction luma sample value $\text{predSamplesLX}[x_L][y_L]$ is derived by invoking the luma sample 8-tap interpolation filtering process as specified in clause 8.5.6.3.2 with $(x\text{Int}_L - (\text{brdExtSize} > 0 ? 1 : 0))$, $y\text{Int}_L - (\text{brdExtSize} > 0 ? 1 : 0))$, $(x\text{Frac}_L, y\text{Frac}_L)$, $(x\text{SbInt}_L, y\text{SbInt}_L)$, refPicLX , hpellIdx , sbWidth , sbHeight and (xSb, ySb) as inputs.

Otherwise (cIdx is not equal to 0), the following applies:

Let $(x\text{IntC}, y\text{IntC})$ be a chroma location given in full-sample units and $(x\text{FracC}, y\text{FracC})$ be an offset given in $1/32$ sample units. These variables are used only in this clause for specifying general fractional-sample locations inside the reference sample arrays refPicLX .

The top-left coordinate of the bounding block for reference sample padding $(x\text{SbIntC}, y\text{SbIntC})$ is set equal to $((x\text{Sb}/\text{SubWidthC}) + (\text{mvLX}[0] >> 5))$, $(y\text{Sb}/\text{SubHeightC}) + (\text{mvLX}[1] >> 5))$.

For each chroma sample location $(xC = 0 \dots \text{sbWidth} - 1, yC = 0 \dots \text{sbHeight} - 1)$ inside the prediction chroma sample arrays predSamplesLX , the corresponding prediction chroma sample value $\text{predSamplesLX}[xC][yC]$ is derived as follows:

Let $(\text{refxSb}_C, \text{refySb}_C)$ and $(\text{refx}_C, \text{refy}_C)$ be chroma locations pointed to by a motion vector $(\text{mvLX}[0], \text{mvLX}[1])$ given in $1/32$ -sample units. The variables refxSb_C , refySb_C , refx_C and refy_C are derived as follows:

$\text{refxSb}_C = ((xSb/\text{SubWidthC} < 5) + \text{mvLX}[0]) * \text{hori_scale_fp}$ (8-763)

$\text{refxC} = ((\text{Sign}(\text{refxSb}_C) * ((\text{Abs}(\text{refxSb}_C) + 256) >> 9) + xC * ((\text{hori_scale_fp} + 8) >> 4)) + 16) >> 5$ (8-764)

$\text{refySb}_C = ((ySb/\text{SubHeightC} < 5) + \text{mvLX}[1]) * \text{vert_scale_fp}$ (8-765)

$\text{refyC} = ((\text{Sign}(\text{refySb}_C) * ((\text{Abs}(\text{refySb}_C) + 256) >> 9) + yC * ((\text{vert_scale_fp} + 8) >> 4)) + 16) >> 5$ (8-766)

10

The variables $x\text{Int}_C$, $y\text{Int}_C$, $x\text{Frac}_C$ and $y\text{Frac}_C$ are derived as follows:

$x\text{Int}_C = \text{refx}_C >> 5$ (8-767)

$y\text{Int}_C = \text{refy}_C >> 5$ (8-768)

$x\text{Frac}_C = \text{refx}_C \& 31$ (8-769)

$y\text{Frac}_C = \text{refy}_C \& 31$ (8-770)

The prediction sample value $\text{predSamplesLX}[xC][yC]$ is derived by invoking the process specified in clause 8.5.6.3.4 with $(x\text{IntC}, y\text{IntC})$, $(x\text{FracC}, y\text{FracC})$, $(x\text{SbIntC}, y\text{SbIntC})$, sbWidth , sbHeight and refPicLX as inputs.

8.5.6.3.2 Luma Sample Interpolation Filtering Process

Inputs to this process are:

a luma location in full-sample units $(x\text{Int}_L, y\text{Int}_L)$,

a luma location in fractional-sample units $(x\text{Frac}_L, y\text{Frac}_L)$,

a luma location in full-sample units $(x\text{SbInt}_L, y\text{SbInt}_L)$ specifying the top-left sample of the bounding block for reference sample padding relative to the top-left luma sample of the reference picture,

the luma reference sample array refPicLX_L ,

the half sample interpolation filter index hpellIdx ,

a variable sbWidth specifying the width of the current subblock,

a variable sbHeight specifying the height of the current subblock,

a luma location (xSb, ySb) specifying the top-left sample of the current subblock relative to the top-left luma sample of the current picture,

Output of this process is a predicted luma sample value predSampleLX_L

The variables shift1 , shift2 and shift3 are derived as follows:

The variable shift1 is set equal to $\text{Min}(4, \text{BitDepth}_Y - 8)$, the variable shift2 is set equal to 6 and the variable shift3 is set equal to $\text{Max}(2, 14 - \text{BitDepth}_Y)$.

The variable picW is set equal to $\text{pic_width_in_luma_samples}$ and the variable picH is set equal to $\text{pic_height_in_luma_samples}$.

The luma interpolation filter coefficients $f_L[p]$ for each $1/16$ fractional sample position p equal to $x\text{Frac}_L$ or $y\text{Frac}_L$ are derived as follows:

If $\text{MotionModelIdx}[xSb][ySb]$ is greater than 0, and sbWidth and sbHeight are both equal to 4, the luma interpolation filter coefficients $f_L[p]$ are specified in Table 2.

Otherwise, the luma interpolation filter coefficients $f_L[p]$ are specified in Table 1 depending on hpellIdx .

The luma locations in full-sample units $(x\text{Int}_i, y\text{Int}_i)$ are derived as follows for $i = 0 \dots 7$:

If $\text{subpic_treated_as_pic_flag}[\text{SubPicIdx}]$ is equal to 1, the following applies:

$x\text{Int}_i = \text{Clip3}(\text{SubPicLeftBoundaryPos}, \text{SubPicRightBoundaryPos}, x\text{Int}_L + i - 3)$ (8-771)

$y\text{Int}_i = \text{Clip3}(\text{SubPicTopBoundaryPos}, \text{SubPicBottomBoundaryPos}, y\text{Int}_L + i - 3)$ (8-772)

Otherwise ($\text{subpic_treated_as_pic_flag}[\text{SubPicIdx}]$ is equal to 0), the following applies:

$x\text{Int}_i = \text{Clip3}(0, \text{picW} - 1, \text{sps_ref_wraparound_enabled_flag} ? \text{ClipH}((\text{sps_ref_wraparound_offset_minus1} + 1) * \text{MinCbSizeY}, \text{picW}, x\text{Int}_L + i - 3) : x\text{Int}_L + i - 3)$ (8-773)

$y\text{Int}_i = \text{Clip3}(0, \text{picH} - 1, y\text{Int}_L + i - 3)$ (8-774)

11

The luma locations in full-sample units are further modified as follows for $i=0 \dots 7$:

$$xInt_i = \text{Clip3}(xSbInt_L - 3, xSbInt_L + sbWidth + 4, xInt_i) \quad (8-775)$$

$$yInt_i = \text{Clip3}(ySbInt_L - 3, ySbInt_L + sbHeight + 4, yInt_i) \quad (8-776) \quad 5$$

The predicted luma sample value predSampleLX_L is derived as follows:

If both $x\text{Frac}_L$ and $y\text{Frac}_L$ are equal to 0, the value of predSampleLX_L is derived as follows:

$$\text{predSampleLX}_L = \text{refPicLX}_L[xInt_3][yInt_3] << \text{shift3} \quad (8-777) \quad 10$$

Otherwise, if $x\text{Frac}_L$ is not equal to 0 and $y\text{Frac}_L$ is equal to 0, the value of predSampleLX_L is derived as follows:

$$\text{predSampleLX}_L = (\sum_{i=0}^7 f_L[x\text{Frac}_L][i] * \text{refPicLX}_L[xInt_i][yInt_3]) >> \text{shift1} \quad (8-778)$$

12

Otherwise, if $x\text{Frac}_L$ is equal to 0 and $y\text{Frac}_L$ is not equal to 0, the value of predSampleLX_L is derived as follows:

$$\text{predSampleLX}_L = (\sum_{i=0}^7 f_L[y\text{Frac}_L][i] * \text{refPicLX}_L[xInt_3][yInt_i]) >> \text{shift1} \quad (8-779)$$

Otherwise, if $x\text{Frac}_L$ is not equal to 0 and $y\text{Frac}_L$ is not equal to 0, the value of predSampleLX_L is derived as follows:

The sample array $\text{temp}[n]$ with $n=0 \dots 7$, is derived as follows:

$$\text{temp}[n] = (\sum_{i=0}^7 f_L[x\text{Frac}_L][i] * \text{refPicLX}_L[xInt_i][yInt_n]) >> \text{shift1} \quad (8-780)$$

The predicted luma sample value predSampleLX_L is derived as follows:

$$\text{predSampleLX}_L = (\sum_{i=0}^7 f_L[y\text{Frac}_L][i] * \text{temp}[i]) >> \text{shift2} \quad (8-781)$$

TABLE 8-11

Specification of the luma interpolation filter coefficients $f_L[p]$ for each 1/16 fractional sample position p.									
Fractional sample	interpolation filter coefficients								
position p	$f_L[p][0]$	$f_L[p][1]$	$f_L[p][2]$	$f_L[p][3]$	$f_L[p][4]$	$f_L[p][5]$	$f_L[p][6]$	$f_L[p][7]$	
1	0	1	-3	63	4	-2	1	0	
2	-1	2	-5	62	8	-3	1	0	
3	-1	3	-8	60	13	-4	1	0	
4	-1	4	-10	58	17	-5	1	0	
5	-1	4	-11	52	26	-8	3	-1	
6	-1	3	-9	47	31	-10	4	-1	
7	-1	4	-11	45	34	-10	4	-1	
8	-1	4	-11	40	40	-11	4	-1	
(hpelldfx = 0)									
8	0	3	9	20	20	9	3	0	
(hpelldfx = 1)									
9	-1	4	-10	34	45	-11	4	-1	
10	-1	4	-10	31	47	-9	3	-1	
11	-1	3	-8	26	52	-11	4	-1	
12	0	1	-5	17	58	-10	4	-1	
13	0	1	-4	13	60	-8	3	-1	
14	0	1	-3	8	62	-5	2	-1	
15	0	1	-2	4	63	-3	1	0	

TABLE 8-12

Specification of the luma interpolation filter coefficients $f_L[p]$ for each 1/16 fractional sample position p for affine motion mode.									
Fractional sample	interpolation filter coefficients								
position p	$f_L[p][0]$	$f_L[p][1]$	$f_L[p][2]$	$f_L[p][3]$	$f_L[p][4]$	$f_L[p][5]$	$f_L[p][6]$	$f_L[p][7]$	
1	0	1	-3	63	4	-2	1	0	
2	0	1	-5	62	8	-3	1	0	
3	0	2	-8	60	13	-4	1	0	
4	0	3	-10	58	17	-5	1	0	
5	0	3	-11	52	26	-8	2	0	
6	0	2	-9	47	31	-10	3	0	
7	0	3	-11	45	34	-10	3	0	
8	0	3	-11	40	40	-11	3	0	
9	0	3	-10	34	45	-11	3	0	
10	0	3	-10	31	47	-9	2	0	
11	0	2	-8	26	52	-11	3	0	
12	0	1	-5	17	58	-10	3	0	
13	0	1	-4	13	60	-8	2	0	
14	0	1	-3	8	62	-5	1	0	
15	0	1	-2	4	63	-3	1	0	

13

8.5.6.3.3 Luma Integer Sample Fetching Process

Inputs to this process are:

- a luma location in full-sample units ($xInt_L$, $yInt_L$),
- the luma reference sample array $refPicLX_L$,

Output of this process is a predicted luma sample value $predSampleLX_L$

The variable shift is set equal to $\text{Max}(2, 14-\text{BitDepth}_Y)$.

The variable $picW$ is set equal to $pic_width_in_luma_samples$ and the variable $picH$ is set equal to $pic_height_in_luma_samples$.

The luma locations in full-sample units ($xInt$, $yInt$) are derived as follows:

$$xInt = \text{Clip3}(0, picW-1, sps_ref_wraparound_enabled_flag ? \text{ClipH}((sps_ref_wraparound_offset_minus1+1) * \text{MinCbSizeY}, picW, xInt_L) : xInt_L) \quad (8-782)$$

$$yInt = \text{Clip3}(0, picH-1, yInt_L) \quad (8-783)$$

The predicted luma sample value $predSampleLX_L$ is derived as follows:

$$predSampleLX_L = refPicLX_L[xInt][yInt] << shift3 \quad (8-784)$$

8.5.6.3.4 Chroma Sample Interpolation Process

Inputs to this process are:

- a chroma location in full-sample units ($xInt_C$, $yInt_C$),
- a chroma location in $1/32$ fractional-sample units ($xFrac_C$, $yFrac_C$),
- a chroma location in full-sample units ($xSbIntC$, $ySbIntC$) specifying the top-left sample of the bounding block for reference sample padding relative to the top-left chroma sample of the reference picture,
- a variable $sbWidth$ specifying the width of the current subblock,
- a variable $sbHeight$ specifying the height of the current subblock,
- the chroma reference sample array $refPicLX_C$.

Output of this process is a predicted chroma sample value $predSampleLX_C$

The variables $shift1$, $shift2$ and $shift3$ are derived as follows:

- The variable $shift1$ is set equal to $\text{Min}(4, \text{BitDepth}_C-8)$,
- the variable $shift2$ is set equal to 6 and the variable $shift3$ is set equal to $\text{Max}(2, 14-\text{BitDepth}_C)$.

The variable $picW_C$ is set equal to $pic_width_in_luma_samples/\text{SubWidthC}$ and the variable $picH_C$ is set equal to $pic_height_in_luma_samples/\text{SubHeightC}$.

The chroma interpolation filter coefficients $f_C[p]$ for each $1/32$ fractional sample position p equal to $xFrac_C$ or $yFrac_C$ are specified in Table 3.

The variable $xOffset$ is set equal to $(sps_ref_wraparound_offset_minus1+1) * \text{MinCbSizeY} / \text{SubWidthC}$.

The chroma locations in full-sample units ($xInt_i$, $yInt_i$) are derived as follows for $i=0 \dots 3$:

If $subpic_treated_as_pic_flag[\text{SubPicIdx}]$ is equal to 1, the following applies:

$$xInt_i = \text{Clip3}(\text{SubPicLeftBoundaryPos}/\text{SubWidthC}, \text{SubPicRightBoundaryPos}/\text{SubWidthC}, xInt_L + i) \quad (8-785)$$

$$yInt_i = \text{Clip3}(\text{SubPicTopBoundaryPos}/\text{SubHeightC}, \text{SubPicBotBoundaryPos}/\text{SubHeightC}, yInt_L + i) \quad (8-786)$$

Otherwise ($subpic_treated_as_pic_flag[\text{SubPicIdx}]$ is equal to 0), the following applies:

$$xInt_i = \text{Clip3}(0, picW_C-1, sps_ref_wraparound_enabled_flag ? \text{ClipH}(xOffset, picW_C, xInt_C + i-1) : xInt_C + i-1) \quad (8-787)$$

$$yInt_i = \text{Clip3}(0, picH_C-1, yInt_C + i-1) \quad (8-788)$$

14

The chroma locations in full-sample units ($xInt_i$, $yInt_i$) are further modified as follows for $i=0 \dots 3$:

$$xInt_i = \text{Clip3}(xSbIntC-1, xSbIntC+sbWidth+2, xInt_i) \quad (8-789)$$

$$yInt_i = \text{Clip3}(ySbIntC-1, ySbIntC+sbHeight+2, yInt_i) \quad (8-790)$$

The predicted chroma sample value $predSampleLX_C$ is derived as follows:

If both $xFrac_C$ and $yFrac_C$ are equal to 0, the value of $predSampleLX_C$ is derived as follows:

$$predSampleLX_C = refPicLX_C[xInt_i][yInt_i] << shift3 \quad (8-791)$$

Otherwise, if $xFrac_C$ is not equal to 0 and $yFrac_C$ is equal to 0, the value of $predSampleLX_C$ is derived as follows:

$$predSampleLX_C = (\sum_{i=0}^3 f_C[xFrac_C][i] * refPicLX_C[xInt_i][yInt_i]) >> shift1 \quad (8-792)$$

Otherwise, if $xFrac_C$ is equal to 0 and $yFrac_C$ is not equal to 0, the value of $predSampleLX_C$ is derived as follows:

$$predSampleLX_C = (\sum_{i=0}^3 f_C[yFrac_C][i] * refPicLX_C[xInt_i][yInt_i]) >> shift1 \quad (8-793)$$

Otherwise, if $xFrac_C$ is not equal to 0 and $yFrac_C$ is not equal to 0, the value of $predSampleLX_C$ is derived as follows:

The sample array $temp[n]$ with $n=0,3$, is derived as follows:

$$temp[n] = (\sum_{i=0}^3 f_C[xFrac_C][i] * refPicLX_C[xInt_i][yInt_i]) >> shift1 \quad (8-794)$$

The predicted chroma sample value $predSampleLX_C$ is derived as follows:

$$yFrac_C yFrac_C yFrac_C yFrac_C predSampleLX_C = (f_C[0] * temp[0] + yFrac_C yFrac_C yFrac_C f_C[1] * temp[1] + yFrac_C yFrac_C yFrac_C f_C[2] * temp[2] + yFrac_C yFrac_C yFrac_C f_C[3] * temp[3]) >> shift2 \quad (8-795)$$

TABLE 8-13

Specification of the chroma interpolation filter coefficients $f_C[p]$ for each $1/32$ fractional sample position p .				
Fractional sample position p	interpolation filter coefficients			
	$f_C[p][0]$	$f_C[p][1]$	$f_C[p][2]$	$f_C[p][3]$
1	-1	63	2	0
2	-2	62	4	0
3	-2	60	7	-1
4	-2	58	10	-2
5	-3	57	12	-2
6	-4	56	14	-2
7	-4	55	15	-2
8	-4	54	16	-2
9	-5	53	18	-2
10	-6	52	20	-2
11	-6	49	24	-3
12	-6	46	28	-4
13	-5	44	29	-4
14	-4	42	30	-4
15	-4	39	33	-4
16	-4	36	36	-4
17	-4	33	39	-4
18	-4	30	42	-4
19	-4	29	44	-5
20	-4	28	46	-6
21	-3	24	49	-6
22	-2	20	52	-6
23	-2	18	53	-5
24	-2	16	54	-4
25	-2	15	55	-4
26	-2	14	56	-4
27	-2	12	57	-3

TABLE 8-13-continued

Specification of the chroma interpolation filter coefficients $fc[p]$ for each $1/32$ fractional sample position p .				
Fractional sample	interpolation filter coefficients			
position p	$fc[p][0]$	$fc[p][1]$	$fc[p][2]$	$fc[p][3]$
28	-2	10	58	-2
29	-1	7	60	-2
30	0	4	62	-2
31	0	2	63	-1

2.5. Refined Sub-Block Based Affine Motion Compensated Prediction

The techniques disclosed herein include a method to refine the sub-block based affine motion compensated prediction with optical flow. After the sub-block based affine motion compensation is performed, prediction sample is refined by adding a difference derived by the optical flow equation, which is referred as prediction refinement with optical flow (PROF). The proposed method can achieve inter prediction in pixel level granularity without increasing the memory access bandwidth.

To achieve a finer granularity of motion compensation, this contribution proposes a method to refine the sub-block based affine motion compensated prediction with optical flow. After the sub-block based affine motion compensation is performed, luma prediction sample is refined by adding a difference derived by the optical flow equation. The proposed PROF (prediction refinement with optical flow) is described as following four steps.

Step 1) The sub-block-based affine motion compensation is performed to generate sub-block prediction $I(i,j)$.

Step 2) The spatial gradients $g_x(i,j)$ and $g_y(i,j)$ of the sub-block prediction are calculated at each sample location using a 3-tap filter $[-1, 0, 1]$.

$$g_x(i,j) = I(i+1,j) - I(i-1,j)$$

$$g_y(i,j) = I(i,j+1) - I(i,j-1)$$

The sub-block prediction is extended by one pixel on each side for the gradient calculation. To reduce the memory bandwidth and complexity, the pixels on the extended borders are copied from the nearest integer pixel position in the reference picture. Therefore, additional interpolation for padding region is avoided.

Step 3) The luma prediction refinement (denoted ΔI) as is calculated by the optical flow equation.

$$\Delta I(i,j) = g_x(i,j) * \Delta v_x(i,j) + g_y(i,j) * \Delta v_y(i,j)$$

where the delta MV (denoted as $\Delta v(i,j)$) is the difference between pixel motion vector (MV) computed for sample location (i,j) , denoted by $v(i,j)$, and the sub-block MV of the sub-block to which pixel (i,j) belongs, as shown in FIG. 1.

Since the affine model parameters and the pixel location relative to the sub-block center are not changed from sub-block to sub-block, $\Delta v(i,j)$ can be calculated for the first sub-block, and reused for other sub-blocks in the same coding unit (CU). Let x and y be the horizontal and vertical offset from the pixel location to the center of the sub-block, $\Delta v(x,y)$ can be derived by the following equation,

$$\begin{cases} \Delta v_x(x, y) = c * x + d * y \\ \Delta v_y(x, y) = e * x + f * y \end{cases}$$

For 4-parameter affine model,

$$\begin{cases} c = f = \frac{v_{1x} - v_{0x}}{w} \\ e = -d = \frac{v_{1y} - v_{0y}}{w} \end{cases}$$

For 6-parameter affine model,

$$\begin{cases} c = \frac{v_{1x} - v_{0x}}{w} \\ d = \frac{v_{2x} - v_{0x}}{h} \\ e = \frac{v_{1y} - v_{0y}}{w} \\ f = \frac{v_{2y} - v_{0y}}{h} \end{cases}$$

where (v_{0x}, v_{0y}) , (v_{1x}, v_{1y}) , (v_{2x}, v_{2y}) are the top-left, top-right and bottom-left control point motion vectors, w and h are the width and height of the CU.

Step 4) Finally, the luma prediction refinement is added to the sub-block prediction $I(i,j)$. The final prediction I' is generated as the following equation.

$$I'(i,j) = I(i,j) + \Delta I(i,j)$$

Some details are described below:

a) How to Derive the Gradients for PROF

In some embodiments, the gradients are calculated for each sub-block (4x4 sub-block in VTM-4.0) for each reference list. For each sub-block, the nearest integer samples of the reference block are fetched to pad the four side outer lines of samples.

Suppose the MV for the current sub-block is (MV_x, MV_y) . Then the fractional part is calculated as $(FracX, FracY) = (MV_x \& 15, MV_y \& 15)$. The integer part is calculated as $(IntX, IntY) = (MV_x >> 4, MV_y >> 4)$. The offsets $(OffsetX, OffsetY)$ are derived as:

$$OffsetX = FracX > 7 ? 1 : 0;$$

$$OffsetY = FracY > 7 ? 1 : 0;$$

Suppose the top-left coordinate of the current sub-block is (x_{Cur}, y_{Cur}) and the dimensions of the current sub-block is $W \times H$. Then (x_{Cor0}, y_{Cor0}) , (x_{Cor1}, y_{Cor1}) , (x_{Cor2}, y_{Cor2}) and (x_{Cor3}, y_{Cor3}) are calculated as:

$$(x_{Cor0}, y_{Cor0}) = (x_{Cur} + IntX + OffsetX - 1, y_{Cur} + IntY + OffsetY - 1);$$

$$(x_{Cor1}, y_{Cor1}) = (x_{Cur} + IntX + OffsetX - 1, y_{Cur} + IntY + OffsetY + H);$$

$$(x_{Cor2}, y_{Cor2}) = (x_{Cur} + IntX + OffsetX - 1, y_{Cur} + IntY + OffsetY);$$

$$(x_{Cor3}, y_{Cor3}) = (x_{Cur} + IntX + OffsetX + W, y_{Cur} + IntY + OffsetY);$$

Suppose $PredSample[x][y]$ with $x=0 \dots W-1$, $y=0 \dots H-1$ stores the prediction samples for the sub-block. Then the padding samples are derived as:

$$PredSample[x][y-1] = (Ref(x_{Cor0} + x, y_{Cor0}) < < Shift0) - \text{Rounding, for } x = -1 \dots W;$$

$$PredSample[x][H] = (Ref(x_{Cor1} + x, y_{Cor1}) < < Shift0) - \text{Rounding, for } x = -1 \dots W;$$

17

$\text{PredSample}[-1][y] = (\text{Ref}(x\text{Cor}2, y\text{Cor}2 + y) \ll \text{Shift}0) - \text{Rounding}$, for $y=0 \dots H-1$;

$\text{PredSample}[W][y] = (\text{Ref}(x\text{Cor}3, y\text{Cor}3 + y) \ll \text{Shift}0) - \text{Rounding}$, for $y=0 \dots H-1$;

where Rec represents the reference picture. Rounding is an integer, which is equal to 2^{13} in the exemplary PROF implementation. $\text{Shift}0 = \text{Max}(2, (14 - \text{BitDepth}))$; PROF attempts to increase the precision of the gradients, unlike BIO in VTM-4.0, where the gradients are output with the same precision as input luma samples.

The gradients in PROF are calculated as below:
 $\text{Shift}1 = \text{Shift}0 - 4$.

$\text{gradientH}[x][y] = (\text{predSamples}[x+1][y] - \text{predSample}[x-1][y]) \gg \text{Shift}1$

$\text{gradientV}[x][y] = (\text{predSample}[x][y+1] - \text{predSample}[x][y-1]) \gg \text{Shift}1$

It should be noted that $\text{predSamples}[x][y]$ keeps the precision after interpolation.

b) How to Derive Δv for PROF

The derivation of Δv (denoted as $\text{dMvH}[\text{posX}][\text{posY}]$ and $\text{dMvV}[\text{posX}][\text{posY}]$ with $\text{posX}=0 \dots W-1$, $\text{posY}=0 \dots H-1$) can be described as below.

Suppose the dimensions of the current block is $\text{cbWidth} \times \text{cbHeight}$, the number of control point motion vectors is numCpMv , and the control point motion vectors are $\text{cpMvLX}[\text{cpIdx}]$, with $\text{cpIdx}=0 \dots \text{numCpMv}-1$ and X being 0 or 1 representing the two reference lists.

The variables $\log 2\text{CbW}$ and $\log 2\text{CbH}$ are derived as follows:

$\log 2\text{CbW} = \text{Log } 2(\text{cbWidth})$

$\log 2\text{CbH} = \text{Log } 2(\text{cbHeight})$

The variables mvScaleHor , mvScaleVer , dHorX and dVerX are derived as follows:

$\text{mvScaleHor} = \text{cpMvLX}[0][0] \ll 7$

$\text{mvScaleVer} = \text{cpMvLX}[0][1] \ll 7$

$\text{dHorX} = (\text{cpMvLX}[1][0] - \text{cpMvLX}[0][0]) \ll (7 - \log 2\text{CbW})$

$\text{dVerX} = (\text{cpMvLX}[1][1] - \text{cpMvLX}[0][1]) \ll (7 - \log 2\text{CbW})$

The variables dHorY and dVerY are derived as follows:
 If numCpMv is equal to 3, the following applies:

$\text{dHorY} = (\text{cpMvLX}[2][0] - \text{cpMvLX}[0][0]) \ll (7 - \log 2\text{CbH})$

$\text{dVerY} = (\text{cpMvLX}[2][1] - \text{cpMvLX}[0][1]) \ll (7 - \log 2\text{CbH})$

Otherwise (numCpMv is equal to 2), the following applies:

$\text{dHorY} = -\text{dVerX}$

$\text{dVerY} = \text{dHorX}$

The variable qHorX , qVerX , qHorY and qVerY are derived as:

$\text{qHorX} = \text{dHorX} \ll 2$;

$\text{qVerX} = \text{dVerX} \ll 2$;

$\text{qHorY} = \text{dHorY} \ll 2$;

$\text{qVerY} = \text{dVerY} \ll 2$;

18

$\text{dMvH}[0][0]$ and $\text{dMvV}[0][0]$ are calculated as:

$\text{dMvH}[0][0] = ((\text{dHorX} + \text{dHorY}) \ll 1) - ((\text{qHorX} + \text{qHorY}) \ll 1)$;

$\text{dMvV}[0][0] = ((\text{dVerX} + \text{dVerY}) \ll 1) - ((\text{qVerX} + \text{qVerY}) \ll 1)$;

$\text{dMvH}[\text{xPos}][0]$ and $\text{dMvV}[\text{xPos}][0]$ for xPos from 1 to $W-1$ are derived as:

$\text{dMvH}[\text{xPos}][0] = \text{dMvH}[\text{xPos}-1][0] + \text{qHorX}$;

$\text{dMvV}[\text{xPos}][0] = \text{dMvV}[\text{xPos}-1][0] + \text{qVerX}$;

For yPos from 1 to $H-1$, the following applies:

$\text{dMvH}[\text{xPos}][\text{yPos}] = \text{dMvH}[\text{xPos}][\text{yPos}-1] + \text{qHorY}$
 with $\text{xPos}=0 \dots W-1$

$\text{dMvV}[\text{xPos}][\text{yPos}] = \text{dMvV}[\text{xPos}][\text{yPos}-1] + \text{qVerY}$ with $\text{xPos}=0 \dots W-1$

Finally, $\text{dMvH}[\text{xPos}][\text{yPos}]$ and $\text{dMvV}[\text{xPos}][\text{yPos}]$ with $\text{posX}=0 \dots W-1$, $\text{posY}=0 \dots H-1$ are right shifted as:

$\text{dMvH}[\text{xPos}][\text{yPos}] = \text{SatShift}(\text{dMvH}[\text{xPos}][\text{yPos}], 7+2-1)$;

$\text{dMvV}[\text{xPos}][\text{yPos}] = \text{SatShift}(\text{dMvV}[\text{xPos}][\text{yPos}], 7+2-1)$;

where $\text{SatShift}(x, n)$ and $\text{Shift}(x, n)$ is defined as:

$\text{SatShift}(x, n) = \begin{cases} (x + \text{offset } 0) \gg n & \text{if } x \geq 0 \\ -((-x + \text{offset } 1) \gg n) & \text{if } x < 0 \end{cases}$

$\text{Shift}(x, n) = (x + \text{offset } 0) \gg n$

In one example, $\text{offset}0$ and/or $\text{offset}1$ are set to $(1 \ll n) \gg 1$.

c) How to Derive ΔI for PROF

For a position (posX , posY) inside a sub-block, its corresponding $\Delta v(i, j)$ is denoted as $(\text{dMvH}[\text{posX}][\text{posY}], \text{dMvV}[\text{posX}][\text{posY}])$. Its corresponding gradients are denoted as $(\text{gradientH}[\text{posX}][\text{posY}], \text{gradientV}[\text{posX}][\text{posY}])$.

Then $\Delta I(\text{posX}, \text{posY})$ is derived as follows.

$(\text{dMvH}[\text{posX}][\text{posY}], \text{dMvV}[\text{posX}][\text{posY}])$ are clipped

as:

$\text{dMvH}[\text{posX}][\text{posY}] = \text{Clip}3(-32768, 32767, \text{dMvH}[\text{posX}][\text{posY}])$;

$\text{dMvV}[\text{posX}][\text{posY}] = \text{Clip}3(-32768, 32767, \text{dMvV}[\text{posX}][\text{posY}])$;

$\Delta I(\text{posX}, \text{posY}) = \text{dMvH}[\text{posX}][\text{posY}] \times \text{gradientH}[\text{posX}][\text{posY}] + \text{dMvV}[\text{posX}][\text{posY}] \times \text{gradientV}[\text{posX}][\text{posY}]$;

$\Delta I(\text{posX}, \text{posY}) = \text{Shift}(\Delta I(\text{posX}, \text{posY}), 1+1+4)$;

$\Delta I(\text{posX}, \text{posY}) = \text{Clip}3(-(2^{13}-1), 2^{13}-1, \Delta I(\text{posX}, \text{posY}))$;

d) How to Derive I' for PROF

If the current block is not coded as bi-prediction or weighted-prediction,

$I'(\text{posX}, \text{posY}) = \text{Shift}((I(\text{posX}, \text{posY}) + \Delta I(\text{posX}, \text{posY})), \text{Shift}0)$,

$I'(\text{posX}, \text{posY}) = \text{ClipSample}(I'(\text{posX}, \text{posY}))$,

where ClipSample clips a sample value to a valid output sample value. Then $I'(\text{posX}, \text{posY})$ is output as the inter-prediction value.

Otherwise (the current block is coded as bi-prediction or weighted-prediction). I'(posX, posY) will be stored and used to generate the inter-prediction value according to other prediction values and/or weighting values.

2.6. Example Slice Header

5

	Descriptor
slice_header() {	
slice_pic_parameter_set_id	ue(v)
if(rect_slice_flag NumBricksInPic > 1)	
slice_address	u(v)
if(!rect_slice_flag && !single_brick_per_slice_flag)	
num_bricks_in_slice_minus1	ue(v)
non_reference_picture_flag	u(1)
slice_type	ue(v)
if(separate_colour_plane_flag == 1)	
colour_plane_id	u(2)
slice_pic_order_cnt_lsb	u(v)
if(nal_unit_type == GDR_NUT)	
recovery_poc_cnt	ue(v)
if(nal_unit_type == IDR_W_RADL nal_unit_type == IDR_N_LP ==	
nal_unit_type == CRA_NUT NalUnitType == GDR_NUT)	
no_output_of_prior_pics_flag	u(1)
if(output_flag_present_flag)	
pic_output_flag	u(1)
if((nal_unit_type != IDR_W_RADL && nal_unit_type != IDR_N_LP)	
sps_idr_rpl_present_flag) {	
for(i = 0; i < 2; i++) {	
if(num_ref_pic_lists_in_sps[i] > 0 && !pps_ref_pic_list_sps_idc[i]	
&&	
(i == 0 (i == 1 && rpl1_idx_present_flag))	
ref_pic_list_sps_flag[i]	u(1)
if(ref_pic_list_sps_flag[i]) {	
if(num_ref_pic_lists_in_sps[i] > 1 &&	
(i == 0 (i == 1 && rpl1_idx_present_flag))	
ref_pic_list_idx[i]	u(v)
} else	
ref_pic_list_struct(i, num_ref_pic_lists_in_sps[i])	
for(j = 0; j < NumLtrpEntries[i][RplIdx[i]]; j++) {	
if(ltrp_in_slice_header_flag[i][RplIdx[i]])	
slice_poc_lsb_ltrp[i][j]	u(v)
delta_poc_msb_present_flag[i][j]	u(1)
if(delta_poc_msb_present_flag[i][j])	
delta_poc_msb_cycle_ltrp[i][j]	ue(v)
}	
}	
if((slice_type != I && num_ref_entries [0][RplIdx[0]] > 1)	
(slice_type == B && num_ref_entries [1][RplIdx[1]] > 1)) {	
num_ref_idx_active_override_flag	u(1)
if(num_ref_idx_active_override_flag)	
for(i = 0; i < (slice_type == B ? 2 : 1); i++)	
if(num_ref_entries [i][RplIdx[i]] > 1)	
num_ref_idx_active_minus1[i]	ue(v)
}	
}	
if(partition_constraints_override_enabled_flag) {	
partition_constraints_override_flag	ue(v)
if(partition_constraints_override_flag) {	
slice_log2_diff_min_qt_min_cb_luma	ue(v)
slice_max_mtt_hierarchy_depth_luma	ue(v)
if(slice_max_mtt_hierarchy_depth_luma != 0)	
slice_log2_diff_max_bt_min_qt_luma	ue(v)
slice_log2_diff_max_tt_min_qt_luma	ue(v)
}	
if(slice_type == I && qtbt_dual_tree_intra_flag) {	
slice_log2_diff_min_qt_min_cb_chroma	ue(v)
slice_max_mtt_hierarchy_depth_chroma	ue(v)
if(slice_max_mtt_hierarchy_depth_chroma != 0)	
slice_log2_diff_max_bt_min_qt_chroma	ue(v)
slice_log2_diff_max_tt_min_qt_chroma	ue(v)
}	
}	
}	
if(slice_type != I) {	
if(sps_temporal_mvp_enabled_flag && !pps_temporal_mvp_enabled_idc)	

-continued

	Descriptor
slice_temporal_mvp_enabled_flag	u(1)
if(slice_type == B && !pps_mvd_l1_zero_idc)	
mvd_l1_zero_flag	u(1)
if(cabac_init_present_flag)	
cabac_init_flag	u(1)
if(slice_temporal_mvp_enabled_flag) {	
if(slice_type == B && !pps_collocated_from_l0_idc)	
collocated_from_l0_flag	u(1)
if((collocated_from_l0_flag && NumRefIdxActive[0] > 1)	
(!collocated_from_l0_flag && NumRefIdxActive[1] > 1))	
collocated_ref_idx	ue(v)
}	
if((pps_weighted_pred_flag && slice_type == P)	
(pps_weighted_bipred_flag && slice_type == B))	
pred_weight_table()	
if(!pps_six_minus_max_num_merge_cand_plus1)	
six_minus_max_num_merge_cand	ue(v)
if(sps_affine_enabled_flag &&	
!pps_five_minus_max_num_subblock_merge_cand_plus1)	
five_minus_max_num_subblock_merge_cand	ue(v)
if(sps_fpel_mmvd_enabled_flag)	
slice_fpel_mmvd_enabled_flag	u(1)
if(sps_bdof_dmvr_slice_present_flag)	
slice_disable_bdof_dmvr_flag	u(1)
if(slice_triangle_enabled_flag && MaxNumMergeCand >= 2 &&	
!pps_max_num_merge_cand_minus_max_num_triangle_cand_minus1	
)	
max_num_merge_cand_minus_max_num_triangle_cand	ue(v)
}	
if(sps_ibc_enabled_flag)	
slice_six_minus_max_num_ibc_merge_cand	ue(v)
if(sps_joint_cbr_enabled_flag)	
slice_joint_cbr_sign_flag	u(1)
slice_qp_delta	se(v)
if(pps_slice_chroma_qp_offsets_present_flag) {	
slice_cb_qp_offset	se(v)
slice_cr_qp_offset	se(v)
if(sps_joint_cbr_enabled_flag)	
slice_joint_cbr_qp_offset	se(v)
}	
if(sps_sao_enabled_flag) {	
slice_sao_luma_flag	u(1)
if(ChromaArrayType != 0)	
slice_sao_chroma_flag	u(1)
}	
if(sps_alf_enabled_flag) {	
slice_alf_enabled_flag	u(1)
if(slice_alf_enabled_flag) {	
slice_num_alf_aps_ids_luma	u(3)
for(i = 0; i < slice_num_alf_aps_ids_luma; i++)	
slice_alf_aps_id_luma[i]	u(3)
if(ChromaArrayType != 0)	
slice_alf_chroma_idc	u(2)
if(slice_alf_chroma_idc)	
slice_alf_aps_id_chroma	u(3)
}	
}	
if(!pps_dep_quant_enabled_flag)	
dep_quant_enabled_flag	u(1)
if(!dep_quant_enabled_flag)	
sign_data_hiding_enabled_flag	u(1)
if(deblocking_filter_override_enabled_flag)	
deblocking_filter_override_flag	u(1)
if(deblocking_filter_override_flag) {	
slice_deblocking_filter_disabled_flag	u(1)
if(!slice_deblocking_filter_disabled_flag) {	
slice_beta_offset_div2	se(v)
slice_tc_offset_div2	se(v)
}	
}	
if(sps_lmcs_enabled_flag) {	
slice_lmcs_enabled_flag	u(1)
if(slice_lmcs_enabled_flag) {	
slice_lmcs_aps_id	u(2)
if(ChromaArrayType != 0)	

	Descriptor
slice_chroma_residual_scale_flag	u(1)
}	
}	
if(sps_scaling_list_enabled_flag) {	
slice_scaling_list_present_flag	u(1)
if(slice_scaling_list_present_flag)	
slice_scaling_list_aps_id	u(3)
}	
if(entry_point_offsets_present_flag && NumEntryPoints > 0) {	
offset_len_minus1	ue(v)
for(i = 0; i < NumEntryPoints; i++)	
entry_point_offset_minus1 [i]	u(v)
}	
if(slice_header_extension_present_flag) {	
slice_header_extension_length	ue(v)
for(i = 0; i < slice_header_extension_length; i++)	
slice_header_extension_data_byte[i]	u(8)
}	
byte_alignment()	
}	

2.7. Example Sequence Parameter Set

	Descriptor
seq_parameter_set_rbsp() {	
sps_decoding_parameter_set_id	u(4)
sps_video_parameter_set_id	u(4)
sps_max_sub_layers_minus1	u(3)
sps_reserved_zero_5bits	u(5)
profile_tier_level(sps_max_sub_layers_minus1)	
gdr_enabled_flag	u(1)
sps_seq_parameter_set_id	ue(v)
chroma_format_idc	ue(v)
if(chroma_format_idc == 3)	
separate_colour_plane_flag	u(1)
pic_width_max_in_luma_samples	ue(v)
pic_height_max_in_luma_samples	ue(v)
subpics_present_flag	u(1)
if(subpics_present_flag) {	
max_subpics_minus1	u(8)
subpic_grid_col_width_minus1	u(v)
subpic_grid_row_height_minus1	u(v)
for(i = 0; i < NumSubPicGridRows; i++)	
for(j = 0; j < NumSubPicGridCols; j++)	
subpic_grid_idx[i][j]	u(v)
for(i = 0; i <= NumSubPics; i++) {	
subpic_treated_as_pic_flag[i]	u(1)
loop_filter_across_subpic_enabled_flag[i]	u(1)
}	
}	
bit_depth_luma_minus8	ue(v)
bit_depth_chroma_minus8	ue(v)
min_qp_prime_ts_minus4	ue(v)
log2_max_pic_order_cnt_lsb_minus4	ue(v)
if(sps_max_sub_layers_minus1 > 0)	
sps_sub_layer_ordering_info_present_flag	u(1)
for(i = (sps_sub_layer_ordering_info_present_flag ? 0 : sps_max_sub_layers_minus1);	
i <= sps_max_sub_layers_minus1; i++) {	
sps_max_dec_pic_buffering_minus1[i]	ue(v)
sps_max_num_reorder_pics[i]	ue(v)
sps_max_latency_increase_plus1[i]	ue(v)
}	
long_term_ref_pics_flag	u(1)
inter_layer_ref_pics_present_flag	u(1)
sps_idr_rpl_present_flag	u(1)
rpl1_same_as_rpl0_flag	u(1)
for(i = 0; i < ! rpl1_same_as_rpl0_flag ? 2 : 1; i++) {	
num_ref_pic_lists_in_sps[i]	ue(v)
for(j = 0; j < num_ref_pic_lists_in_sps[i]; j++)	
ref_pic_list_struct(i, j)	
}	
if(ChromaArrayType != 0)	

-continued

	Descriptor
qtbtt_dual_tree_intra_flag	u(1)
log2_ctu_size_minus5	u(2)
log2_min_luma_coding_block_size_minus2	ue(v)
partition_constraints_override_enabled_flag	u(1)
sps_log2_diff_min_qt_min_cb_intra_slice_luma	ue(v)
sps_log2_diff_min_qt_min_cb_inter_slice	ue(v)
sps_max_mtt_hierarchy_depth_inter_slice	ue(v)
sps_max_mtt_hierarchy_depth_intra_slice_luma	ue(v)
if(sps_max_mtt_hierarchy_depth_intra_slice_luma != 0) {	
sps_log2_diff_max_bt_min_qt_intra_slice_luma	ue(v)
sps_log2_diff_max_tt_min_qt_intra_slice_luma	ue(v)
}	
if(sps_max_mtt_hierarchy_depth_inter_slices != 0) {	
sps_log2_diff_max_bt_min_qt_inter_slice	ue(v)
sps_log2_diff_max_tt_min_qt_inter_slice	ue(v)
}	
if(qtbtt_dual_tree_intra_flag) {	
sps_log2_diff_min_qt_min_cb_intra_slice_chroma	ue(v)
sps_max_mtt_hierarchy_depth_intra_slice_chroma	ue(v)
if (sps_max_mtt_hierarchy_depth_intra_slice_chroma != 0) {	
sps_log2_diff_max_bt_min_qt_intra_slice_chroma	ue(v)
sps_log2_diff_max_tt_min_qt_intra_slice_chroma	ue(v)
}	
}	
sps_max_luma_transform_size_64_flag	u(1)
if(ChromaArrayType != 0) {	
same_qp_table_for_chroma	u(1)
for(i = 0; i < same_qp_table_for_chroma ? 1 : 3; i++) {	
num_points_in_qp_table_minus1[i]	ue(v)
for(j = 0; j <= num_points_in_qp_table_minus1[i]; j++) {	
delta_qp_in_val_minus1[i][j]	ue(v)
delta_qp_out_val[i][j]	ue(v)
}	
}	
}	
sps_weighted_pred_flag	u(1)
sps_weighted_bipred_flag	u(1)
sps_sao_enabled_flag	u(1)
sps_alf_enabled_flag	u(1)
sps_transform_skip_enabled_flag	u(1)
if(sps_transform_skip_enabled_flag)	
sps_bdpcm_enabled_flag	u(1)
sps_joint_cbr_enabled_flag	u(1)
sps_ref_wraparound_enabled_flag	u(1)
if(sps_ref_wraparound_enabled_flag)	
sps_ref_wraparound_offset_minus1	ue(v)
sps_temporal_mvp_enabled_flag	u(1)
if(sps_temporal_mvp_enabled_flag)	
sps_sbtmvp_enabled_flag	u(1)
sps_amvr_enabled_flag	u(1)
sps_bdof_enabled_flag	u(1)
sps_smvd_enabled_flag	u(1)
sps_dmvr_enabled_flag	u(1)
if(sps_bdof_enabled_flag sps_dmvr_enabled_flag)	
sps_bdof_dmvr_slice_present_flag	u(1)
sps_nmvd_enabled_flag	u(1)
sps_isp_enabled_flag	u(1)
sps_mrl_enabled_flag	u(1)
sps_mip_enabled_flag	u(1)
if(ChromaArrayType != 0)	
sps_cclm_enabled_flag	u(1)
if(sps_cclm_enabled_flag && chroma_format_idc == 1)	
sps_cclm_collocated_chroma_flag	u(1)
sps_mts_enabled_flag	u(1)
if(sps_mts_enabled_flag) {	
sps_explicit_mts_intra_enabled_flag	u(1)
sps_explicit_mts_inter_enabled_flag	u(1)
}	
sps_sbt_enabled_flag	u(1)
if(sps_sbt_enabled_flag)	
sps_sbt_max_size_64_flag	u(1)
sps_affine_enabled_flag	u(1)
if(sps_affine_enabled_flag) {	
sps_affine_type_flag	u(1)
sps_affine_amvr_enabled_flag	u(1)

	Descriptor
sps_affine_prof_enabled_flag	u(1)
}	
if(chroma_format_idc == 3)	
sps_palette_enabled_flag	u(1)
sps_bcw_enabled_flag	u(1)
sps_ibc_enabled_flag	u(1)
sps_ciip_enabled_flag	u(1)
if(sps_mmvd_enabled_flag)	
sps_fpel_mmvd_enabled_flag	u(1)
sps_triangle_enabled_flag	u(1)
sps_lmcs_enabled_flag	u(1)
sps_lfnst_enabled_flag	u(1)
sps_ladf_enabled_flag	u(1)
if (sps_ladf_enabled_flag) {	
sps_num_ladf_intervals_minus2	u(2)
sps_ladf_lowest_interval_qp_offset	se(v)
for(i = 0; i < sps_num_ladf_intervals_minus2 + 1; i++) {	
sps_ladf_qp_offset[i]	se(v)
sps_ladf_delta_threshold_minus1[i]	ue(v)
}	
}	
sps_scaling_list_enabled_flag	u(1)
hrd_parameters_present_flag	u(1)
if(general_hrd_parameters_present_flag) {	
num_units_in_tick	u(32)
time_scale	u(32)
sub_layer_cpb_parameters_present_flag	u(1)
if(sub_layer_cpb_parameters_present_flag)	
general_hrd_parameters(0, sps_max_sub_layers_minus1)	
else	
general_hrd_parameters(sps_max_sub_layers_minus1, sps_max_sub_layers_minus1)	
}	
vui_parameters_present_flag	u(1)
if(vui_parameters_present_flag)	
vui_parameters()	
sps_extension_flag	u(1)
if(sps_extension_flag)	
while(more_rbsp_data())	
sps_extension_data_flag	u(1)
rbsp_trailing_bits()	
}	

2.8 Example Picture Parameter Set

	Descriptor
pic_parameter_set_rbsp() {	
pps_pic_parameter_set_id	ue(v)
pps_seq_parameter_set_id	ue(v)
pic_width_in_luma_samples	ue(v)
pic_height_in_luma_samples	ue(v)
conformance_window_flag	u(1)
if(conformance_window_flag) {	
conf_win_left_offset	ue(v)
conf_win_right_offset	ue(v)
conf_win_top_offset	ue(v)
conf_win_bottom_offset	ue(v)
}	
output_flag_present_flag	u(1)
single_tile_in_pic_flag	u(1)
if(!single_tile_in_pic_flag) {	
uniform_tile_spacing_flag	u(1)
if(uniform_tile_spacing_flag) {	
tile_cols_width_minus1	ue(v)
tile_rows_height_minus1	ue(v)
} else {	
num_tile_columns_minus1	ue(v)
num_tile_rows_minus1	ue(v)
for(i = 0; i < num_tile_columns_minus1; i++)	
tile_column_width_minus1[i]	ue(v)
for(i = 0; i < num_tile_rows_minus1; i++)	
tile_row_height_minus1[i]	ue(v)
}	
}	

-continued

	Descriptor
brick_splitting_present_flag	u(1)
if(uniform_tile_spacing_flag && brick_splitting_present_flag)	
num_tiles_in_pic_minus1	ue(v)
for(i = 0; brick_splitting_present_flag && i <= num_tiles_in_pic_minus1 + 1; i++) {	
if(RowHeight[i] > 1)	
brick_split_flag[i]	u(1)
if(brick_split_flag[i]) {	
if(RowHeight[i] > 2)	
uniform_brick_spacing_flag[i]	u(1)
if(uniform_brick_spacing_flag[i])	
brick_height_minus1[i]	ue(v)
else {	
num_brick_rows_minus2[i]	ue(v)
for(j = 0; j <= num_brick_rows_minus2[i]; j++)	
brick_row_height_minus1[i][j]	ue(v)
}	
}	
}	
single_brick_per_slice_flag	u(1)
if(!single_brick_per_slice_flag)	
rect_slice_flag	u(1)
if(rect_slice_flag && !single_brick_per_slice_flag) {	
num_slices_in_pic_minus1	ue(v)
bottom_right_brick_idx_length_minus1	ue(v)
for(i = 0; i < num_slices_in_pic_minus1; i++) {	
bottom_right_brick_idx_delta[i]	u(v)
brick_idx_delta_sign_flag[i]	u(1)
}	
}	
loop_filter_across_bricks_enabled_flag	u(1)
if(loop_filter_across_bricks_enabled_flag)	
loop_filter_across_slices_enabled_flag	u(1)
}	
if(rect_slice_flag) {	
signalled_slice_id_flag	u(1)
if(signalled_slice_id_flag) {	
signalled_slice_id_length_minus1	ue(v)
for(i = 0; i <= num_slices_in_pic_minus1; i++)	
slice_id[i]	u(v)
}	
}	
entropy_coding_sync_enabled_flag	u(1)
if(!single_tile_in_pic_flag entropy_coding_sync_enabled_flag)	
entry_point_offsets_present_flag	u(1)
cabac_init_present_flag	u(1)
for(i = 0; i < 2; i++)	
num_ref_idx_default_active_minus1[i]	ue(v)
rpl1_idx_present_flag	u(1)
init_qp_minus26	se(v)
if(sps_transform_skip_enabled_flag)	
log2_transform_skip_max_size_minus2	ue(v)
cu_qp_delta_enabled_flag	u(1)
if(cu_qp_delta_enabled_flag)	
cu_qp_delta_subdiv	ue(v)
pps_cb_qp_offset	se(v)
pps_cr_qp_offset	se(v)
pps_joint_cbr_qp_offset	se(v)
pps_slice_chroma_qp_offsets_present_flag	u(1)
cu_chroma_qp_offset_enabled_flag	u(1)
if(cu_chroma_qp_offset_enabled_flag) {	
cu_chroma_qp_offset_subdiv	ue(v)
chroma_qp_offset_list_len_minus1	ue(v)
for(i = 0; i <= chroma_qp_offset_list_len_minus1; i++) {	
cb_qp_offset_list[i]	se(v)
cr_qp_offset_list[i]	se(v)
joint_cbr_qp_offset_list[i]	se(v)
}	
}	
pps_weighted_pred_flag	u(1)
pps_weighted_bipred_flag	u(1)
deblocking_filter_control_present_flag	u(1)
if(deblocking_filter_control_present_flag) {	
deblocking_filter_override_enabled_flag	u(1)
pps_deblocking_filter_disabled_flag	u(1)
if(!pps_deblocking_filter_disabled_flag) {	

	Descriptor
pps_beta_offset_div2	se(v)
pps_tc_offset_div2	se(v)
}	
constant_slice_header_params_enabled_flag	u(1)
if(constant_slice_header_params_enabled_flag) {	
pps_dep_quant_enabled_idc	u(2)
for(i = 0; i < 2; i++)	
pps_ref_pic_list_sps_idc[i]	u(2)
pps_temporal_mvp_enabled_idc	u(2)
pps_mv_d_11_zero_idc	u(2)
pps_collocated_from_l0_idc	u(2)
pps_six_minus_max_num_merge_cand_plus1	ue(v)
pps_five_minus_max_num_subblock_merge_cand_plus1	ue(v)
pps_max_num_merge_cand_minus_max_num_triangle_cand_minus1	ue(v)
}	
pps_loop_filter_across_virtual_boundaries_disabled_flag	u(1)
if(pps_loop_filter_across_virtual_boundaries_disabled_flag) {	
pps_num_ver_virtual_boundaries	u(2)
for(i = 0; i < pps_num_ver_virtual_boundaries; i++)	
pps_virtual_boundaries_pos_x[i]	u(13)
pps_num_hor_virtual_boundaries	u(2)
for(i = 0; i < pps_num_hor_virtual_boundaries; i++)	
pps_virtual_boundaries_pos_y[i]	u(13)
}	
slice_header_extension_present_flag	u(1)
pps_extension_flag	u(1)
if(pps_extension_flag)	
while(more_rbsp_data())	
pps_extension_data_flag	u(1)
rbsp_trailing_bits()	
}	

2.9. Example Adaptive Parameter Set

-continued

	Descriptor	35	Descriptor
adaptation_parameter_set_rbsp() {			else if(aps_params_type == SCALING_APS)
adaptation_parameter_set_id	u(5)		scaling_list_data()
aps_params_type	u(3)		aps_extension_flag
if(aps_params_type == ALF_APS)		40	if(aps_extension_flag)
alf_data()			while(more_rbsp_data())
else if(aps_params_type == LMCS_APS)			aps_extension_data_flag
lmcs_data()			rbsp_trailing_bits()
			}

	Descriptor
alf_data() {	
alf_luma_filter_signal_flag	u(1)
alf_chroma_filter_signal_flag	u(1)
if(alf_luma_filter_signal_flag) {	
alf_luma_clip_flag	u(1)
alf_luma_num_filters_signalled_minus1	ue(v)
if(alf_luma_num_filters_signalled_minus1 > 0) {	
for(filtIdx = 0; filtIdx < NumAlfFilters; filtIdx++)	
alf_luma_coeff_delta_idx[filtIdx]	u(v)
}	
alf_luma_coeff_signalled_flag	u(1)
if(alf_luma_coeff_signalled_flag) {	
for(sflIdx = 0; sflIdx <= alf_luma_num_filters_signalled_minus1;	
sflIdx++)	
alf_luma_coeff_flag[sflIdx]	u(1)
}	
for(sflIdx = 0; sflIdx <= alf_luma_num_filters_signalled_minus1;	
sflIdx++) {	
if(alf_luma_coeff_flag[sflIdx]) {	
for(j = 0; j < 12; j++) {	
alf_luma_coeff_abs[sflIdx][j]	uek(v)
if(alf_luma_coeff_abs[sflIdx][j])	
alf_luma_coeff_sign[sflIdx][j]	u(1)
}	
}	
}	

Descriptor

```
}  
}  
if( alf_luma_clip_flag ) {  
    for( sIdx = 0; sIdx <= alf_luma_num_filters_signalled_minus1;  
        sIdx++ ) {  
        if( alf_luma_coeff_flag[ sIdx ] ) {  
            for ( j = 0; j < 12; j++ )  
                alf_luma_clip_idx[ sIdx ][ j ]          ue(2)  
        }  
    }  
}  
}  
}  
if( alf_chroma_filter_signal_flag ) {  
    alf_chroma_num_alt_filters_minus1          ue(v)  
    for( altIdx = 0; altIdx <= alf_chroma_num_alt_filters_minus1;  
        altIdx++ ) {  
        alf_chroma_clip_flag[ altIdx ]          u(1)  
        for( j = 0; j < 6; j++ ) {  
            alf_chroma_coeff_abs[ altIdx ][ j ]   uek(v)  
            if( alf_chroma_coeff_abs[ altIdx ][ j ] > 0 )  
                alf_chroma_coeff_sign[ altIdx ][ j ]   u(1)  
        }  
        if( alf_chroma_clip_flag[ altIdx ] ) {  
            for( j = 0; j < 6; j++ )  
                alf_chroma_clip_idx[ altIdx ][ j ]     u(2)  
        }  
    }  
}  
}
```

		30	-continued	
Descriptor			Descriptor	
lmc_data () {			lmc_delta_abs_cw[i]	u(v)
lmc_min_bin_idx	ue(v)	35	if (lmc_delta_abs_cw[i]) > 0)	
lmc_delta_max_bin_idx	ue(v)		lmc_delta_sign_cw_flag[i]	u(1)
lmc_delta_cw_prec_minus1	ue(v)		}	
			}	
for (i = lmc_min_bin_idx; i <= LmcMaxBinIdx; i++) {				

Descriptor

```

scaling_list_data( ) {
    for( sizeId = 1; sizeId < 7; sizeId++ )
        for( matrixId = 0; matrixId < 6; matrixId ++ ) {
            if( ! ( ( ( sizeId == 1 ) && (matrixId % 3 == 0 ) ) ||
                ( ( sizeId == 6 ) && ( matrixId % 3 != 0 ) ) ) ) {
                scaling_list_pred_mode_flag[ sizeId ][ matrixId ]          u(1)
            if( scaling_list_pred_mode_flag[ sizeId ][ matrixId ] )
                scaling_list_pred_matrix_id_delta[ sizeId ][ matrixId ]    ue(v)
            else {
                nextCoef = 8
                coefNum = Min( 64, (1 << (sizeId << 1 ) ) )
                if( sizeId > 3 ) {
                    scaling_list_dc_coef_minus8[ sizeId - 4 ][ matrixId ]    se(v)
                    nextCoef = scaling_list_dc_coef_minus8[ sizeId - 4 ][ matrixId ]
                        + 8
                }
            }
            for( i = 0; i < coefNum; i++ ) {
                x = DiagScanOrder[ 3 ][ 3 ][ i ][ 0 ]
                y = DiagScanOrder[ 3 ][ 3 ][ i ][ 1 ]
                if( !( sizeId == 6 && x >= 4 && y >= 4 ) ) {

```


	Descriptor
scaling_list_delta_coef	se(v)
nextCoef = (nextCoef + scaling_list_delta_coef + 256)	
% 256	
ScalingList[sizeId][matrixId][i] = nextCoef	
}	
}	
}	
}	
}	
}	

2.10. Example Picture Header

In some embodiments, the picture header is designed to have the following properties:

1. Temporal Identifier (Id) and layer Id of picture header NAL unit are the same as temporal Id and layer Id of layer access unit that contains the picture header.
2. Picture header network abstraction layer (NAL) unit shall precede NAL unit containing the first slice of its associated picture. This established the association between a picture header and slices of picture associated with the picture header without the need of having picture header Id signalled in picture header and referred to from slice header.
3. Picture header NAL unit shall follow picture level parameter sets or higher level such as dependency parameter set (DPS), video parameter set (VPS), sequence parameter set (SPS), picture parameter set (PPS), etc. This consequently requires those parameter sets to be not repeated/present within a picture or within an access unit.
4. Picture header contains information about picture type of its associated picture. The picture type may be used to define the following (not an exhaustive list)
 - a. The picture is an IDR picture
 - b. The picture is a CRA picture
 - c. The picture is a gradual decoding refresh (GDR) picture
 - d. The picture is a non-IRAP, non-GDR picture and contains I-slices only
 - e. The picture is a non-IRAP, non-GDR picture and may contain P- and I-slices only
 - f. The picture is a non-IRAP, non-GDR picture and contains any of B-, P-, and/or I-slices
5. Move signalling of picture level syntax elements in slice header to picture header.
6. Signal non-picture level syntax elements in slice header that are typically the same for all slices of the same picture in picture header. When those syntax elements are not present in picture header, they may be signalled in slice header.

In some implementations, a mandatory picture header concept is used to be transmitted once per picture as the first video coding layer (VCL) NAL unit of a picture. It is also proposed to move syntax elements currently in the slice header to this picture header. Syntax elements that functionally only need to be transmitted once per picture could be moved to the picture header instead of being transmitted multiple times for a given picture, e.g., syntax elements in the slice header are transmitted once per slice. Moving slice header syntax elements constrained to be the same within a picture.

The syntax elements are already constrained to be the same in all slices of a picture. It is asserted that moving these fields to the picture header so they are signalled only once per picture instead of once per slice avoids unnecessary redundant transmission of bits without any change to the functionality of these syntax elements.

1. In some implementations, there is the following semantic constraint:

When present, the value of each of the slice header syntax elements slice_pic_parameter_set_id, non_reference_picture_flag, colour_plane_id, slice_pic_order_cnt_lsb, recovery_poc_cnt, no_output_of_prior_pics_flag, pic_output_flag, and slice_temporal_mvp_enabled_flag shall be the same in all slice headers of a coded picture. Thus each of these syntax elements could be moved to the picture header to avoid unnecessary redundant bits.

The recovery_poc_cnt and no_output_of_prior_pics_flag are not moved to the picture header in this contribution. Their presence in the slice header is dependent on a conditional check of the slice header nal_unit_type, so they are suggested to be studied if there is a desire to move these syntax elements to the picture header.

2. In some implementations, there are the following semantic constraints:

When present, the value of slice_lmcs_aps_id shall be the same for all slices of a picture.

When present, the value of slice_scaling_list_aps_id shall be the same for all slices of a picture. Thus each of these syntax elements could be moved to the picture header to avoid unnecessary redundant bits.

In some embodiments, the syntax elements are not currently constrained to be the same in all slices of a picture. It is suggested to assess the anticipated usage of these syntax elements to determine which can be moved into the picture header to simplify the overall VVC design as it is claimed there is a complexity impact to processing a large number of syntax elements in every slice header.

1. The following syntax elements are proposed to be moved to the picture header. There are currently no restrictions on them having different values for different slices but it is claimed there is no/minimal benefit and coding loss to transmitting them in every slice header as their anticipated usage would change at the picture level:

- a. six_minus_max_num_merge_cand
- b. five_minus_max_num_subblock_merge_cand
- c. slice_fpel_mmvd_enabled_flag
- d. slice_disable_bdof_dmvr_flag
- e. max_num_merge_cand_minus_max_num_triangle_cand
- f. slice_six_minus_max_num_ibc_merge_cand

2. The following syntax elements are proposed to be moved to the picture header. There are currently no restrictions on them having different values for different slices but it is claimed there is no/minimal benefit and coding loss to transmitting them in every slice header as their anticipated usage would change at the picture level:
 - a. partition_constraints_override_flag
 - b. slice_log_2_diff_min_qt_min_cb_luma
 - c. slice_max_mtt_hierarchy_depth_luma
 - d. slice_log_2_diff_max_bt_min_qt_luma
 - e. slice_log_2_diff_max_tt_min_qt_luma
 - f. slice_log_2_diff_min_qt_min_cb_chroma
 - g. slice_max_mtt_hierarchy_depth_chroma
 - h. slice_log_2_diff_max_bt_min_qt_chroma
 - i. slice_log_2_diff_max_tt_min_qt_chroma
 The conditional check "slice type==r" associated with some of these syntax elements has been removed with the move to the picture header.
3. The following syntax elements are proposed to be moved to the picture header. There are currently no restrictions on them having different values for different slices but it is claimed there is no/minimal benefit and coding loss to transmitting them in every slice header as their anticipated usage would change at the picture level:
 - a. mvd_11_zero_flag
 The conditional check "slice type==B" associated with some of these syntax elements has been removed with the move to the picture header.
4. The following syntax elements are proposed to be moved to the picture header. There are currently no restrictions on them having different values for different slices but it is claimed there is no/minimal benefit and coding loss to transmitting them in every slice header as their anticipated usage would change at the picture level:
 - a. dep_quant_enabled_flag
 - b. sign_data_hiding_enabled_flag

3. Drawbacks of Existing Implementations

DMVR and BIO do not involve the original signal during refining the motion vectors, which may result in coding blocks with inaccurate motion information. Also, DMVR and BIO sometimes employ the fractional motion vectors after the motion refinements while screen videos usually have integer motion vectors, which makes the current motion information more inaccurate and make the coding performance worse.

When RPR is applied in VVC, RPR (ARC) may have the following problems:

1. With RPR, the interpolation filters may be different for adjacent samples in a block, which is undesirable in SIMD (Single Instruction Multiple Data) implementation.
2. The bounding region does not consider RPR.
3. It is noted that "The conformance cropping window offset parameters are only applied at the output. All internal decoding processes are applied to the uncropped picture size." However, those parameters may be used in the decoding process when RPR is applied.
4. When deriving the reference sample position, RPR only considers the ratio between two conformance windows. But the top-left offset difference between two conformance windows should also be considered.

5. The ratio between the width/height of a reference picture and that of the current picture is constrained in VVC. However, the ratio between the width/height of the conformance window of a reference picture and that of the conformance window of the current picture is not constrained.
6. Not all the syntax elements are handled properly in the picture header.
7. In current VVC, for triangular prediction mode (TPM), geometric (GEO) prediction mode, the chroma blending weights are derived regardless of the chroma sample location type of a video sequence. For example, in TPM/GEO, if chroma weights are derived from luma weights, the luma weights may be needed to be downsampled to match the sampling of the chroma signal. The chroma downsampling are normally applied assumes the chroma sample location type 0, which is widely used in ITU-R BT.601 or ITU-R BT.709 container. However, if a different chroma sample location type is used, this could result in a misalignment between the chroma samples and the downsampled luma samples, which may reduce the coding performance.

4. Example Techniques and Embodiments

The detailed embodiments described below should be considered as examples to explain general concepts. These embodiments should not be interpreted narrowly way. Furthermore, these embodiments can be combined in any manner.

The methods described below may be also applicable to other decoder motion information derivation technologies in addition to the DMVR and BIO mentioned below.

A motion vector is denoted by (mv_x, mv_y) wherein mv_x is the horizontal component and mv_y is the vertical component.

In this disclosure, the resolution (or dimensions, or width/height, or size) of a picture may refer to the resolution (or dimensions, or width/height, or size) of the coded/decoded picture, or may refer to the resolution (or dimensions, or width/height, or size) of the conformance window in the coded/decoded picture.

Motion Compensation in RPR

1. When the resolution of the reference picture is different to the current picture, or when the width and/or height of the reference picture is larger than that of the current picture, predicted values for a group of samples (at least two samples) of a current block may be generated with the same horizontal and/or vertical interpolation filter.
 - a. In one example, the group may comprise all samples in a region of the block.
 - i. For example, a block may be divided into S MxN rectangles not overlapped with each other. Each MxN rectangle is a group. In an example as shown in FIG. 2, a 16x16 block can be divided into 16 4x4 rectangles, each of which is a group.
 - ii. For example, a row with N samples is a group. N is an integer no larger than the block width. In one example, N is 4 or 8 or the block width.
 - iii. For example, a column with N samples is a group. N is an integer no larger than the block height. In one example, N is 4 or 8 or the block height.
 - iv. M and/or N may be pre-defined or derived on-the-fly, such as based on block dimension/coded information or signaled.

- b. In one example, samples in the group may have the same MV (denoted as shared MV).
- c. In one example, samples in the group may have MVs with the same horizontal component (denoted as shared horizontal component). 5
- d. In one example, samples in the group may have MVs with the same vertical component (denoted as shared vertical component).
- e. In one example, samples in the group may have MVs with the same fractional part of the horizontal component (denoted as shared fractional horizontal component). 10
- i. For example, suppose the MV for a first sample is (MV1x, MV1y) and the MV for a second sample is (MV2x, MV2y), it should be satisfied that MV1x & (2^M-1) is equal to MV2x & (2^M-1) , where M denotes MV precision. For example, M=4. 15
- f. In one example, samples in the group may have MVs with the same fractional part of the vertical component (denoted as shared fractional vertical component). 20
- i. For example, suppose the MV for a first sample is (MV1x, MV1y) and the MV for a second sample is (MV2x, MV2y), it should be satisfied that MV1y & (2^M-1) is equal to MV2y & (2^M-1) , where M denotes MV precision. For example, M=4. 25
- g. In one example, for a sample in the group to be predicted, the motion vector, denoted by MV_b , may be firstly derived according to the resolutions of the current picture and the reference picture (e.g., $(refx_L, refy_L)$ derived in 8.5.6.3.1 in JVET-O2001-v14). Then, MV_b may be further modified (e.g., being rounded/truncated/clipped) to MV' to satisfy the requirements such as the above bullets, and MV' will be used to derive the prediction sample for the sample. 30
- i. In one example, MV' has the same integer part as MV_b , and the fractional part of the MV' is set to be the shared fractional horizontal and/or vertical component. 40
- ii. In one example, MV' is set to be the one with the shared fractional horizontal and/or vertical component, and closest to MV_b . 45
- h. The shared motion vector (and/or shared horizontal component and/or shared vertical component and/or shared fractional vertical component and/or shared fractional vertical component) may be set to be the motion vector (and/or horizontal component and/or vertical component and/or fractional vertical component and/or fractional vertical component) of a specific sample in the group. 50
- i. For example, the specific sample may be at a corner of a rectangle-shaped group, such as "A", "B", "C" and "D" shown in FIG. 3A. 55
- ii. For example, the specific sample may be at a center of a rectangle-shaped group, such as "E", "F", "G" and "H" shown in FIG. 3A. 60
- iii. For example, the specific sample may be at an end of a row-shaped or column-shaped group, such as "A" and "D" shown in FIGS. 3B and 3C. 65
- iv. For example, the specific sample may be at a middle of a row-shaped or column-shaped group, such as "B" and "C" shown in FIGS. 3B and 3C.
- v. In one example, the motion vector of the specific sample may be the MV_b mentioned in bullet g.

- i. The shared motion vector (and/or shared horizontal component and/or shared vertical component and/or shared fractional vertical component and/or shared fractional vertical component) may be set to be the motion vector (and/or horizontal component and/or vertical component and/or fractional vertical component and/or fractional vertical component) of a virtual sample located at a different position compared to all samples in this group.
- i. In one example, the virtual sample is not in the group, but it locates in the region covering all samples in the group.
 - 1) Alternatively, the virtual sample is located outside the region covering all samples in the group, e.g., next to the bottom-right position of the region.
- ii. In one example, the MV of a virtual sample is derived in the same way as a real sample but with different positions.
- iii. "V" in FIGS. 3A-3C shows three examples of virtual samples.
- j. The shared MV (and/or shared horizontal component and/or shared vertical component and/or shared fractional vertical component and/or shared fractional vertical component) may be set to be a function of MVs (and/or horizontal components and/or vertical components and/or fractional vertical components and/or fractional vertical components) of multiple samples and/or virtual samples.
- i. For example, the shared MV (and/or shared horizontal component and/or shared vertical component and/or shared fractional vertical component and/or shared fractional vertical component) may be set to be the average of MVs (and/or horizontal components and/or vertical components and/or fractional vertical components and/or fractional vertical components) of all or partial of samples in the group, or of sample "E", "F", "G", "H" in FIG. 3A, or of sample "E", "H" in FIG. 3A, or of sample "A", "B", "C", "D" in FIG. 3A, or of sample "A", "D" in FIG. 3A, or of sample "B", "C" in FIG. 3B, or of sample "A", "D" in FIG. 3B, or of sample "B", "C" in FIG. 3C, or of sample "A", "D" in FIG. 3C.
- 2. It is proposed that only integer MVs are allowed to perform the motion compensation process to derive the prediction block of a current block when the resolution of the reference picture is different to the current picture, or when the width and/or height of the reference picture is larger than that of the current picture.
 - a. In one example, the decoded motion vectors for samples to be predicted are rounded to integer MVs before being used.
 - b. In one example, the decoded motion vector for samples to be predicted are rounded to the integer MV that is closest to the decoded motion vector.
 - c. In one example, the decoded motion vector for samples to be predicted are rounded to the integer MV that is closest to the decoded motion vector in horizontal direction.
 - d. In one example, the decoded motion vector for samples to be predicted are rounded to the integer MV that is closest to the decoded motion vector in vertical direction.
- 3. The motion vectors used in the motion compensation process for samples in a current block (e.g., shared MV/shared horizontal or vertical or fractional compo-

41

ment/MV' mentioned in above bullets) may be stored in the decoded picture buffer and utilized for motion vector prediction of succeeding blocks in current/different pictures.

- a. Alternatively, the motion vectors used in the motion compensation process for samples in a current block (e.g., shared MV/shared horizontal or vertical or fractional component/MV' mentioned in above bullets) may be disallowed to be utilized for motion vector prediction of succeeding blocks in current/different pictures.
 - i. In one example, the decoded motion vectors (e.g., MV_b in above bullets) may be utilized for motion vector prediction of succeeding blocks in current/different pictures.
- b. In one example, the motion vectors used in the motion compensation process for samples in a current block may be utilized in the filtering process (e.g., deblocking filter/sample adaptive offset (SAO)/adaptive loop filter (ALF)).
 - i. Alternatively, the decoded motion vectors (e.g., MV_b in above bullets) may be utilized in the filtering process.
- c. In one example, such MV may be derived at sub-block level and may be stored for each sub-block.
4. It is proposed that the interpolation filters used in the motion compensation process to derive the prediction block of a current block may be selected depending on whether the resolution of the reference picture is different to the current picture, or whether the width and/or height of the reference picture is larger than that of the current picture.
 - a. In one example, the interpolation filters with less taps may be applied when condition A is satisfied, wherein condition A depends on the dimensions of the current picture and/or the reference picture.
 - i. In one example, condition A is the resolution of the reference picture is different to the current picture.
 - ii. In one example, condition A is the width and/or height of the reference picture is larger than that of the current picture.
 - iii. In one example, condition A is $W1 > a * W2$ and/or $H1 > b * H2$, wherein (W1, H1) represents the width and height of the reference picture and (W2, H2) represents the width and height of the current picture, a and b are two factors, e.g., $a=b=1.5$.
 - iv. In one example, condition A may also depend on whether bi-prediction is used.
 - v. In one example, 1-tap filters are applied. In other words, an integer pixel without filtering is output as the interpolation result.
 - vi. In one example, bi-linear filters are applied when the resolution of the reference picture is different to the current picture.
 - vii. In one example, 4-tap filters or 6-tap filters are applied when the resolution of the reference picture is different to the current picture, or the width and/or height of the reference picture is larger than that of the current picture.
 - 1) The 6-tap filters may also be used for the affine motion compensation.
 - 2) The 4-tap filters may also be used for interpolation for chroma samples.
 - b. In one example, padding samples are used to perform interpolation when the resolution of the reference

42

picture is different to the current picture, or the width and/or height of the reference picture is larger than that of the current picture.

- c. Whether to and/or how to apply the methods disclosed in bullet 4 may depend on the color components.
 - i. For example, the methods are only applied on the luma component.
- d. Whether to and/or how to apply the methods disclosed in bullet 4 may depend on the interpolation filtering direction.
 - i. For example, the methods are only applied on horizontal filtering.
 - ii. For example, the methods are only applied on vertical filtering.
5. It is proposed that a two-stage process for prediction block generation is applied when the resolution of the reference picture is different to the current picture, or when the width and/or height of the reference picture is larger than that of the current picture.
 - a. In the first stage, a virtual reference block is generated by up-sampling or downsampling a region in the reference picture depending on width and/or height of the current picture and the reference picture.
 - b. In the second stage, the prediction samples are generated from the virtual reference block by applying interpolation filtering, independent of width and/or height of the current picture and the reference picture.
6. It is proposed that the calculation of top-left coordinate of the bounding block for reference sample padding ($xSbInt_L$, $ySbInt_L$) as in some embodiments may be derived depending on width and/or height of the current picture and the reference picture.
 - a. In one example, the luma locations in full-sample units are modified as:

$$xInt_L = Clip3(xSbInt_L - Dx, xSbInt_L + sbWidth + Ux, xInt_L),$$

$$yInt_L = Clip3(ySbInt_L - Dy, ySbInt_L + sbHeight + Uy, yInt_L),$$
 where Dx and/or Dy and/or Ux and/or Uy may depend on width and/or height of the current picture and the reference picture.
 - b. In one example, the chroma locations in full-sample units are modified as:

$$xInt_L = Clip3(xSbInt_L - Dx, xSbInt_L + sbWidth + Ux, xInt_L)$$

$$yInt_L = Clip3(ySbInt_L - Dy, ySbInt_L + sbHeight + Uy, yInt_L)$$
 where Dx and/or Dy and/or Ux and/or Uy may depend on width and/or height of the current picture and the reference picture.
7. Instead of storing/using the motion vectors for a block based on the same reference picture resolution as current picture, it is proposed to use the real motion vectors with the resolution difference taken into consideration.
 - a. Alternatively, furthermore, when using the motion vector to generate the prediction block, there is no need to further change the motion vector according to the resolutions of the current picture and the reference picture (e.g., ($refx_L$, $refy_L$)).
- Interaction Between RPR and Other Coding Tools
8. Whether to/how to apply filtering process (e.g., deblocking filter) may depend on the resolutions of reference pictures and/or the resolution of the current picture.

- a. In one example, the boundary strength (BS) settings in the deblocking filters may take the resolution differences into consideration in addition to motion vector differences.
- i. In one example, the scaled motion vector difference according to the current and reference pictures' resolutions may be used to determine the boundary strength.
- b. In one example, the strength of deblocking filter for a boundary between block A and block B may be set differently (e.g., being increased/decreased) if the resolution of at least one reference picture of block A is different to (or smaller than or larger than) the resolution of at least one reference picture of block B compared to the case that same resolutions are utilized for the two blocks.
- c. In one example, a boundary between block A and block B is marked as to be filtered (e.g., BS is set to 2) if the resolution of at least one reference picture of block A is different to (or smaller than or larger than) the resolution of at least one reference picture of block B.
- d. In one example, the strength of deblocking filter for a boundary between block A and block B may be set differently (e.g., being increased/decreased) if the resolution of at least one reference picture of block A and/or block B is different to (or smaller than or larger than) the resolution of the current picture compared to the case same resolution is utilized of the reference picture and current picture.
- e. In one example, a boundary between two blocks is marked to be filtered (e.g., BS is set to 2) if at least one reference picture of at least one block of the two has a resolution different to that of the current picture.
9. When a sub-picture exists, a conformance bitstream may should satisfy the reference picture must have the same resolution as the current picture.
 - a. Alternatively, when a reference picture has a different resolution to the current picture, there must be no sub-picture in the current picture.
 - b. Alternatively, for a sub-picture in the current picture, it is disallowed to use a reference picture that is with different resolution as the current picture.
 - i. Alternatively, furthermore, the reference picture management may be invoked to exclude those reference pictures with different resolutions.
10. In one example, sub-pictures (e.g., how to split one picture to multiple sub-pictures) may be defined separately for pictures with different resolutions.

In one example, the corresponding sub-picture in the reference picture can be derived by scaling and/or offsetting a sub-picture of the current picture, if the reference picture has a different resolution to the current picture.
11. PROF (prediction refinement with optical flow) may be enabled when the reference picture has a resolution different to that of the current picture.
 - a. In one example, one set of MV (denoted as MV_g) may be generated for a group of samples and may be used for motion compensation as described in bullet 1. On the other hand, MV (denoted as MV_p) may be derived for each sample, and the difference (e.g., corresponds to the Δv used in PROF) between the MV_p and MV_g together with the gradients (e.g., spatial gradients of the motion compensated blocks) may be used for deriving the prediction refinement.

- b. In one example, MV_p may be with a different precision from MV_g . For example, MV_p may be with 1/N-pel ($N > 0$) precision, $N=32, 64$ etc.
- c. In one example, MV_g may be with a different precision form the internal MV precision (e.g., $1/16$ -pel).
- d. In one example, the prediction refinement is added to the prediction block to generate refined prediction block.
- e. In one example, such method may be applied in each prediction direction.
- f. In one example, such method may be applied only in uni-prediction case.
- g. In one example, such method may be applied in uni-prediction or/and bi-prediction.
- h. In one example, such method may be applied only when the reference picture has a different resolution from the current picture.
12. It is proposed that only one MV may be utilized for a block/sub-block to perform the motion compensation process to derive the prediction block of a current block when the resolution of the reference picture is different to that of the current picture.
 - a. In one example, the only MV for the block/sub-block may be defined as a function (e.g., average) of all MVs associated with each sample within the block/sub-block.
 - b. In one example, the only MV for the block/sub-block may be defined as a selected MV associated with a selected sample (e.g., center sample) within the block/sub-block.
 - c. In one example, only one MV may be utilized a 4×4 block or subblock (e.g., 4×1).
 - d. In one example, BIO may be further applied to compensate the precision loss due to a block-based motion vector.
13. A lazy mode without signalling any block-based motion vectors may be applied when the width and/or height of the reference picture is different from that of the current picture.
 - a. In one example, no motion vectors may be signaled and the motion compensation process is to approximate the case of a pure resolution change of a still image.
 - b. In one example, only a motion vector at picture/tile/brick/coding tree unit (CTU) level may be signal and related blocks may use the motion vector when resolution changes.
14. PROF may be applied to approximate motion compensation when the width and/or height of the reference picture is different to that of the current picture for blocks coded with affine prediction mode and/or non-affine prediction mode.
 - a. In one example, PROF may be enabled when the width and/or height of the reference picture and that of the current picture are different.
 - b. In one example, a set of affine motions may be generated by combining the indicated motion and resolution scaling and used by PROF.
15. Interweaved prediction (e.g., as proposed in JVET-K0102) may be applied to approximate motion compensation when the width and/or height of the reference picture is different to that of the current picture.
 - a. In one example, resolution change (zooming) is represented as an affine motion, and interweaved motion prediction may be applied.

16. LMCS and/or chroma residual scaling may be disabled when the width and/or height of the current picture is different to that of the IRAP picture in a same IRAP period.
- In one example, when luma mapping with chroma scaling (LMCS) is disabled, the slice level flags such as slice_lmcs_enabled_flag, slice_lmcs_aps_id, and slice_chroma_residual_scale_flag may be not signaled and inferred to be 0.
 - In one example, when chroma residual scaling is disabled, the slice level flags such as slice_chroma_residual_scale_flag may be not signaled and inferred to be 0.
- Constrains on RPR
17. RPR may be applied to coding blocks with block dimensions constrains.
- In one example, for an $M \times N$ coding block, with M as the block width and N as the block height, when $M * N < T$ or $M * N \leq T$ (such as $T=256$), RPR may be not used.
 - In one example, when $M < K$ (or $M \leq K$) (such as $K=16$) and/or $N < L$ (or $N \leq L$) (such as $L=16$), RPR may be not used.
18. Bitstream conformance may be added to restrict the ratio between the width and/or height of an active reference picture (or its conformance window) and that of the current picture (or its conformance window). Suppose refPicW and refPicH denote the width and height of reference picture, curPicW and curPicH denote the width and height of current picture,
- In one example, when $(\text{refPicW} + \text{curPicW})$ is equal to an integer number, the reference picture may be marked as active reference picture.
 - Alternatively, when $(\text{refPicW} + \text{curPicW})$ is equal to a fractional number, the reference picture may be marked as not available.
 - In one example, when $(\text{refPicW} + \text{curPicW})$ is equal to $(X * n)$, where X denotes a fractional number such as $X=1/2$, and n denotes an integer number such as $n=1, 2, 3, 4 \dots$, the reference picture may be marked as active reference picture.
 - In one example, when $(\text{refPicW} + \text{curPicW})$ is not equal to $(X * n)$, the reference picture may be marked as not available.
19. Whether to and/or how to enable a coding tool (e.g., bi-prediction/the whole triangular prediction mode (TPM)/blending process in TPM) for a $M \times N$ block may depend on the resolutions of reference pictures (or their conformance windows) and/or that of the current picture (or its conformance window).
- In one example, $M * N < T$ or $M * N \leq T$ (such as $T=64$).
 - In one example, $M < K$ (or $M \leq K$) (such as $K=16$) and/or $N < L$ (or $N \leq L$) (such as $L=16$).
 - In one example, the coding tool is not allowed when width/height of at least one reference picture is different to the current picture,
 - In one example, the coding tool is not allowed when width/height of at least one reference picture of the block is larger that of the current picture.
 - In one example, the coding tool is not allowed when width/height of each reference picture of the block is different to that of the current picture,
 - In one example, the coding tool is not allowed when width/height of each reference pictures is larger that of the current picture.

- Alternatively, furthermore, when the coding tool is not allowed, motion compensation may be conducted with one MV as a uni-prediction.

Conformance Window Related

20. The conformance cropping window offset parameters (e.g., conf_win_left_offset) are signaled in N-pel precision instead of 1-pel wherein N is an positive integer greater than 1.
- In one example, the actual offset may be derived as the signaled offset multiplied by N.
 - In one example, N is set to 4 or 8.
21. It is proposed that the conformance cropping window offset parameters are not only applied at the output. Certain internal decoding processes may depend on the cropped picture size (i.e., the resolution of a conformance window in a picture).
22. It is proposed that the conformance cropping window offset parameters in a first video unit (e.g., PPS) and in a second video unit may be different when the width and/or height of a picture denoted as (pic_width_in_luma_samples, pic_height_in_luma_samples) in the first video unit and second video unit are the same.
23. It is proposed that the conformance cropping window offset parameters in a first video unit (e.g., PPS) and in a second video unit should be the same in a conformance bitstream when the width and/or height of a picture denoted as (pic_width_in_luma_samples, pic_height_in_luma_samples) in the first video unit and second video unit are different.
- It is proposed that the conformance cropping window offset parameters in a first video unit (e.g., PPS) and in a second video unit should be the same in a conformance bitstream no matter the width and/or height of a picture denoted as (pic_width_in_luma_samples, pic_height_in_luma_samples) in the first video unit and second video unit are the same or not.
24. Suppose the width and height of the conformance window defined in a first video unit (e.g., PPS) are denoted as $W1$ and $H1$, respectively. The width and height of the conformance window defined in a second video unit (e.g., PPS) are denoted as $W2$ and $H2$, respectively. The top-left position of the conformance window defined in a first video unit (e.g., PPS) are denoted as $X1$ and $Y1$. The top-left position of the conformance window defined in a second video unit (e.g., PPS) are denoted as $X2$ and $Y2$. The width and height of the coded/decoded picture (e.g., pic_width_in_luma_samples and pic_height_in_luma_samples) defined in a first video unit (e.g., PPS) are denoted as $PW1$ and $PH1$, respectively. The width and height of the coded/decoded picture defined in a second video unit (e.g., PPS) are denoted as $PW2$ and $PH2$.
- In one example, $W1/W2$ should be equal to $X1/X2$ in a conformance bitstream.
 - Alternatively, $W1/X1$ should be equal to $W2/X2$ in a conformance bitstream.
 - Alternatively, $W1 * X2$ should be equal to $W2 * X1$ in a conformance bitstream.
 - In one example, $H1/H2$ should be equal to $Y1/Y2$ in a conformance bitstream.
 - Alternatively, $H1/Y1$ should be equal to $H2/Y2$ in a conformance bitstream.
 - Alternatively, $H1 * Y2$ should be equal to $H2 * Y1$ in a conformance bitstream.
 - In one example, $PW1/PW2$ should be equal to $X1/X2$ in a conformance bitstream.

47

- i. Alternatively, $PW1/X1$ should be equal to $PW2/X2$ in a conformance bitstream.
- ii. Alternatively, $PW1*X2$ should be equal to $PW2*X1$ in a conformance bitstream.
- d. In one example, $PH1/PH2$ should be equal to $Y1/Y2$ in a conformance bitstream. 5
- i. Alternatively, $PH1/Y1$ should be equal to $PH2/Y2$ in a conformance bitstream.
- ii. Alternatively, $PH1*Y2$ should be equal to $PH2*Y1$ in a conformance bitstream. 10
- e. In one example, $PW1/PW2$ should be equal to $W1/W2$ in a conformance bitstream.
- i. Alternatively, $PW1/W1$ should be equal to $PW2/W2$ in a conformance bitstream.
- ii. Alternatively, $PW1*W2$ should be equal to $PW2*W1$ in a conformance bitstream. 15
- f. In one example, $PH1/PH2$ should be equal to $H1/H2$ in a conformance bitstream.
- i. Alternatively, $PH1/H1$ should be equal to $PH2/H2$ in a conformance bitstream. 20
- ii. Alternatively, $PH1*H2$ should be equal to $PH2*H1$ in a conformance bitstream.
- g. In a conformance bitstream, if $PW1$ is greater than $PW2$, $W1$ must be greater than $W2$.
- h. In a conformance bitstream, if $PW1$ is smaller than $PW2$, $W1$ must be smaller than $W2$. 25
- i. In a conformance bitstream, $(PW1-PW2)*(W1-W2)$ must be no smaller than 0.
- j. In a conformance bitstream, if $PH1$ is greater than $PH2$, $H1$ must be greater than $H2$. 30
- k. In a conformance bitstream, if $PH1$ is smaller than $PH2$, $H1$ must be smaller than $H2$.
- l. In a conformance bitstream, $(PH1-PH2)*(H1-H2)$ must be no smaller than 0.
- m. In a conformance bitstream, $W1/W2$ must be no larger than (or smaller than) $PW1/PW2$ if $PW1 \geq PW2$. 35
- n. In a conformance bitstream, $H1/H2$ must be no larger than (or smaller than) $PH1/PH2$ if $PH1 \geq PH2$.
- 25. Suppose the width and height of the conformance window of the current picture are denoted as W and H , respectively. The width and height of the conformance window of a reference picture are denoted as W' and H' , respectively. Then at least one constraint below should be followed by a conformance bit-stream. 40
- a. $W*pw \geq W'$; pw is an integer such as 2.
- b. $W*pw > W'$; pw is an integer such as 2.
- c. $W*pw' \geq W$; pw' is an integer such as 8.
- d. $W*pw' > W$; pw' is an integer such as 8.
- e. $H*ph \geq H'$; ph is an integer such as 2. 50
- f. $H*ph > H'$; ph is an integer such as 2.
- g. $H*ph' \geq H$; ph' is an integer such as 8.
- h. $H*ph' > H$; ph' is an integer such as 8.
- i. In one example, pw is equal to pw' .
- j. In one example, ph is equal to ph' . 55
- k. In one example, pw is equal to ph .
- l. In one example, pw' is equal to ph' .
- m. In one example, the above sub-bullets may be required to be satisfied by a conformance bitstream when W and H represent the width and height of the current picture, respectively. W' and H' represent the width and height of a reference picture. 60
- 26. It is proposed that the conformance window parameters are partially signaled. 65
- a. In one example, the top-left sample in a conformance window of a picture is the same one as that in the picture.

48

- b. For example, $conf_win_left_offset$ as defined in VVC is not signaled and inferred to be zero.
- c. For example, $conf_win_top_offset$ as defined in VVC is not signaled and inferred to be zero.
- 27. It is proposed that the derivation of the position (e.g., $(refx_L, refy_L)$ as defined in VVC) of a reference sample may depend on the top-left position (e.g., $(conf_win_left_offset, conf_win_top_offset)$ as defined in VVC) of the conformance window of the current picture and/or the reference picture. FIG. 4 shows examples of the samples positions derived as in VVC (a) and in a proposed method (b). Dashed rectangles represent the conformance windows.
- a. In one example, the dependency exists only when the width and/or the height of the current picture and that of the reference picture are different.
- b. In one example, the derivation of the horizontal position (e.g., $refx_L$ as defined in VVC) of a reference sample may depend on the left position (e.g., $conf_win_left_offset$ as defined in VVC) of the conformance window of the current picture and/or the reference picture.
- i. In one example, the horizontal position (denoted as xSb') of the current sample relative to the top-left position of the conformance window in the current picture is calculated and used to derive the position of the reference sample.
- 1) For example, $xSb' = xSb - (conf_win_left_offset \ll Prec)$ is calculated and used to derive the position of the reference sample, wherein xSb represents the horizontal position of the current sample in the current picture. $conf_win_left_offset$ represents the horizontal position of the top-left sample in the conformance window of the current picture. $Prec$ presents the precision of xSb and xSb' wherein $(xSb \gg Prec)$ may show the actual horizontal coordinate of current sample relative to the current picture. For example, $Prec=0$ or $Prec=4$.
- ii. In one example, horizontal position (denoted as Rx') of the reference sample relative to the top-left position of the conformance window in the reference picture is calculated.
- 1) The calculation of Rx' may depend on xSb' , and/or a motion vector, and/or a resampling ratio.
- iii. In one example, horizontal position (denoted as Rx) of the reference sample relative in the reference picture is calculated depending on Rx' .
- 1) For example, $Rx = Rx' + (conf_win_left_offset_ref \ll Prec)$ is calculated, wherein $conf_win_left_offset_ref$ represents the horizontal position of the top-left sample in the conformance window of the reference picture. $Prec$ presents the precision of Rx and Rx' . For example, $Prec=0$ or $Prec=4$.
- iv. In one example, Rx may be calculated directly depending on xSb' , and/or a motion vector, and/or a resampling ratio. In other words, the two steps of derivation on Rx' and Rx are combined into a one-step calculation.
- v. Whether to and/or how to use the left position (e.g., $conf_win_left_offset$ as defined in VVC) of the conformance window of the current picture and/or the reference picture may depend on the color components and/or color formats.

49

- 1) For example, `conf_win_left_offset` may be revised as `conf_win_left_offset=conf_win_left_offset*SubWidthC`, wherein `SubWidthC` defines the horizontal sampling step of a color component. E.g., `SubWidthC` is equal to 1 for the luma component. `SubWidthC` is equal to 2 for a chroma component when the color format is 4:2:0 or 4:2:2. 5
- 2) For example, `conf_win_left_offset` may be revised as `conf_win_left_offset=conf_win_left_offset/SubWidthC`, wherein `SubWidthC` defines the horizontal sampling step of a color component. E.g., `SubWidthC` is equal to 1 for the luma component. `SubWidthC` is equal to 2 for a chroma component when the color format is 4:2:0 or 4:2:2. 10 15
- c. In one example, the derivation of the vertical position (e.g., `refyL` as defined in VVC) of a reference sample may depend on the top position (e.g., `conf_win_top_offset` as defined in VVC) of the conformance window of the current picture and/or the reference picture. 20
 - i. In one example, the vertical position (denoted as `ySb'`) of the current sample relative to the top-left position of the conformance window in the current picture is calculated and used to derive the position of the reference sample. 25
 - 1) For example, `ySb'=ySb-(conf_win_top_offset<<Prec)` is calculated and used to derive the position of the reference sample, wherein `ySb` represents the vertical position of the current sample in the current picture. `conf_win_top_offset` represents the vertical position of the top-left sample in the conformance window of the current picture. `Prec` presents the precision of `ySb` and `ySb'`. For example, `Prec=0` or `Prec=4`. 30 35 40
 - ii. In one example, the vertical position (denoted as `Ry'`) of the reference sample relative to the top-left position of the conformance window in the reference picture is calculated. 40
 - 1) The calculation of `Ry'` may depend on `ySb'`, and/or a motion vector, and/or a resampling ratio. 45
 - iii. In one example, the vertical position (denoted as `Ry`) of the reference sample relative in the reference picture is calculated depending on `Ry'`. 50
 - 1) For example, `Ry=Ry'+(conf_win_top_offset_ref<<Prec)` is calculated, wherein `conf_win_top_offset_ref` represents the vertical position of the top-left sample in the conformance window of the reference picture. `Prec` presents the precision of `Ry` and `Ry'`. For example, `Prec=0` or `Prec=4`. 55
 - iv. In one example, `Ry` may be calculated directly depending on `ySb'`, and/or a motion vector, and/or a resampling ratio. In other words, the two steps of derivation on `Ry'` and `Ry` are combined into a one-step calculation. 60
 - v. Whether to and/or how to use the top position (e.g., `conf_win_top_offset` as defined in VVC) of the conformance window of the current picture and/or the reference picture may depend on the color components and/or color formats. 65

50

- 1) For example, `conf_win_top_offset` may be revised as `conf_win_top_offset=conf_win_top_offset*SubHeightC`, wherein `SubHeightC` defines the vertical sampling step of a color component. E.g., `SubHeightC` is equal to 1 for the luma component. `SubHeightC` is equal to 2 for a chroma component when the color format is 4:2:0. 5
- 2) For example, `conf_win_top_offset` may be revised as `conf_win_top_offset=conf_win_top_offset/SubHeightC`, wherein `SubHeightC` defines the vertical sampling step of a color component. E.g., `SubHeightC` is equal to 1 for the luma component. `SubHeightC` is equal to 2 for a chroma component when the color format is 4:2:0. 10 15
28. It is proposed that the integer part of the horizontal coordinate of a reference sample may be clipped to `[minW, maxW]`. Suppose the width and height of the conformance window of the reference picture are denoted as `W` and `H`, respectively. The width and height of the conformance window of a reference picture are denoted as `W'` and `H'`. The top-left position of the conformance window in the reference picture are denoted as `(X0, Y0)`. 20
 - a. In one example, `minW` is equal to 0.
 - b. In one example, `minW` is equal to `X0`.
 - c. In one example, `maxW` is equal to `W-1`.
 - d. In one example, `maxW` is equal to `W'-1`.
 - e. In one example, `maxW` is equal to `X0+W'-1`.
 - f. In one example, `minW` and/or `maxW` may be modified based on color format and/or color component. 25
 - i. For example, `minW` is modified to be `minW*SubC`.
 - ii. For example, `minW` is modified to be `minW/SubC`.
 - iii. For example, `maxW` is modified to be `maxW*SubC`.
 - iv. For example, `maxW` is modified to be `maxW/SubC`.
 - v. In one example, `SubC` is equal to 1 for the luma component.
 - vi. In one example, `SubC` is equal to 2 for a chroma component when the color format is 4:2:0.
 - vii. In one example, `SubC` is equal to 2 for a chroma component when the color format is 4:2:2.
 - viii. In one example, `SubC` is equal to 1 for a chroma component when the color format is 4:4:4.
 - g. In one example, the whether to and/or how to do the clipping may depend on the dimensions of the current picture (or the conformance window in it) and the dimensions of the reference picture (or the conformance window in it). 30
 - i. In one example, the clipping is done only when the dimensions of the current picture (or the conformance window in it) and the dimensions of the reference picture (or the conformance window in it) are different. 35
29. It is proposed that the integer part of the vertical coordinate of a reference sample may be clipped to `[minH, maxH]`. Suppose the width and height of the conformance window of the reference picture are denoted as `W` and `H`, respectively. The width and height of the conformance window of a reference picture are 40 45 50 55 60 65

51

denoted as W' and H' . The top-left position of the conformance window in the reference picture are denoted as (X_0, Y_0) .

- a. In one example, $\min H$ is equal to 0.
- b. In one example, $\min H$ is equal to Y_0 . 5
- c. In one example, $\max H$ is equal to $H-1$.
- d. In one example, $\max H$ is equal to $H'-1$.
- e. In one example, $\max H$ is equal to $Y_0+H'-1$.
- f. In one example, $\min H$ and/or $\max H$ may be modified based on color format and/or color component. 10
 - i. For example, $\min H$ is modified to be $\min H * \text{SubC}$.
 - ii. For example, $\min H$ is modified to be $\min H / \text{SubC}$.
 - iii. For example, $\max H$ is modified to be $\max H * \text{SubC}$.
 - iv. For example, $\max H$ is modified to be $\max H / \text{SubC}$. 15
 - v. In one example, SubC is equal to 1 for the luma component.
 - vi. In one example, SubC is equal to 2 for a chroma component when the color format is 4:2:0. 20
 - vii. In one example, SubC is equal to 1 for a chroma component when the color format is 4:2:2.
 - viii. In one example, SubC is equal to 1 for a chroma component when the color format is 4:4:4.
- g. In one example, the whether to and/or how to do the clipping may depend on the dimensions of the current picture (or the conformance window in it) and the dimensions of the reference picture (or the conformance window in it). 25
 - i. In one example, the clipping is done only when the dimensions of the current picture (or the conformance window in it) and the dimensions of the reference picture (or the conformance window in it) are different. 30

In the following discussion, a first syntax element is asserted to be “corresponding” to a second syntax element, if the two syntax elements have an equivalent functionality but may be signaled at different video unit (e.g., VPS/SPS/PPS/slice header/picture header etc.) 35

30. It is proposed that a syntax element may be signaled in a first video unit (e.g., picture header or PPS) and no corresponding syntax element is signaled in a second video unit at a higher level (such as SPS) or a lower level (such as slice header). 40
 - a. Alternatively, a first syntax element may be signaled in the first video unit (e.g., picture header or PPS) and a corresponding second syntax element may be signaled in a second video unit at a lower level (such as slice header). 45
 - i. Alternatively, an indicator may be signaled in the second video unit to inform whether the second syntax element is signaled thereafter. 50
 - ii. In one example, the slice associated with the second video unit (such as slice header) may follow the indication of the second syntax element instead of the first one if the second one is signaled. 55
 - iii. An indicator associated with the first syntax element may be signaled in the first video unit to inform whether the second syntax element is signaled in any slice (or other video unit) associated with the first video unit. 60
 - b. Alternatively, a first syntax element may be signaled in a first video unit at a higher level (such as VPS/SPS/PPS), and a corresponding second syntax element may be signaled in the second video unit (such as picture header). 65

52

- i. Alternatively, an indicator may be signaled to inform whether the second syntax element is signaled thereafter.
- ii. In one example, the picture (which may be partitioned into slices) associated with the second video unit may follow the indication of the second syntax element instead of the first one if the second one is signaled.
- c. A first syntax element in the picture header may have an equivalent functionality as a second syntax element in the slice header as specified in section 2.6 (such as but limited to `slice_temporal_mvp_enabled_flag`, `cabac_init_flag`, `six_minus_max_num_merge_cand`, `five_minus_max_num_subblock_merge_cand`, `slice_fpel_mmvd_enabled_flag`, `slice_disable_bd-of_dmv_r_flag`, `max_num_merge_cand_minus_max_num_triangle_cand`, `slice_fpel_mmvd_enabled_flag`, `slice_six_minus_max_num_ibc_merge_cand`, `slice_joint_cbr_sign_flag`, `slice_qp_delta`, . . .) but control all slices of the picture.
- d. A first syntax element in SPS as specified in section 2.6 may have an equivalent functionality as a second syntax element in the picture header (such as but limited to `sps_bdof_dmv_r_slice_present_flag`, `sps_mmvd_enabled_flag`, `sps_isp_enabled_flag`, `sps_mrl_enabled_flag`, `sps_mip_enabled_flag`, `sps_cclm_enabled_flag`, `sps_mts_enabled_flag` . . .) but control only the associated picture (which may be partitioned into slices).
- e. A first syntax element in PPS as specified in section 2.7 may have an equivalent functionality as a second syntax element in the picture header (such as but limited to `entropy_coding_sync_enabled_flag`, `entry_point_offsets_present_flag`, `cabac_init_present_flag`, `rpl1_idx_present_flag` . . .) but control only the associated picture (which may be partitioned into slices).
31. In this disclosure (bullet 1-bullet 29), the term “conformance window” may be replaced by other terms such as “scaling window”. A scaling window may be signaled differently to the conformance window and is used to derive the scaling ratio and/or top-left offset used to derive the reference sample position for RPR.
 - a. In one example, the scaling window may be constrained by the conformance window. For example, in a conformance bit-stream, the scaling window must be contained by the conformance window.
32. Whether and/or how to signal the allowed max block size for transform-skip-coded blocks may depend on the max block size for transform-coded blocks.
 - a. Alternatively, the max block size for transform-skip-coded blocks cannot be larger than max block size for transform-coded blocks in a conformance bit-stream.
33. Whether and how to signal the indication of enabling Joint Cb-Cr Residue (JCCR) coding (such as `sps_joint_cbr_enabled_flag`) may depend the color format (such as 4:0:0, 4:2:0 etc.)
 - a. For example, the indication of enabling Joint Cb-Cr Residue (JCCR) may not be signaled if the color format is 4:0:0. An exemplary syntax design is as below:

```

if ChromaArrayType != 0
    sps_joint_cbr_enabled_flag    u(1)

```

Downsampling Filter Type for Chroma Blending Mask Generation in TPM/GEO 5

34. The type of downsampling filter used for blending weights derivation for chroma samples may be signaled at video unit level (such as SPS/VPS/PPS/Picture header/Subpicture/Slice/Slice header/Tile/Brick/CTU/VPDU level). 10

a. In one example, a high level flag may be signaled to switch between different chroma format types of content. 15

i. In one example, a high level flag may be signaled to switch between chroma format type 0 and chroma format type 2.

ii. In one example, a flag may be signaled for specifying whether the top-left downsampled luma weights in TPM/GEO prediction mode is collocated with the top-left luma weights (i.e., chroma sample location type 0). 20

iii. In one example, a flag may be signaled for specifying whether the top-left downsampled luma sample in TPM/GEO prediction mode is horizontally co-sited with the top-left luma sample but vertically shifted by 0.5 units of luma samples relatively to the top-left luma sample (i.e., chroma sample location type 2). 25

b. In one example, the type of downsampling filter may be signaled for 4:2:0 chroma format and/or 4:2:2 chroma format. 30

c. In one example, a flag may be signaled for specifying the type of chroma downsampling filter used for TPM/GEO prediction. 35

i. In one example, the flag may be signaled for whether to use downsampling filter A or downsampling filter B for the chroma weights derivation in TPM/GEO prediction mode. 40

35. The type of downsampling filter used for blending weights derivation for chroma samples may be derived at video unit level (such as SPS/VPS/PPS/Picture header/Subpicture/Slice/Slice header/Tile/Brick/CTU/VPDU level). 45

a. In one example, a look up table may be defined to specify the correspondence relationship between the chroma subsampling filter type and the chroma format types of content.

36. A specified downsampling filter may be used for TPM/GEO prediction mode in case of different chroma sample location type. 50

a. In one example, chroma weights of TPM/GEO may be subsampled from the collocated top-left luma weights in case of a certain chroma sample location type (e.g., chroma sample location type 0). 55

b. In one example, in case of a certain chroma sample location type (e.g., chroma sample location type 0 or 2), a specified X-tap filter (X is a constant such as X=6 or 5) may be used for chroma weights subsampling in TPM/GEO prediction mode. 60

37. In a video unit (e.g., SPS, PPS, picture header, slice header etc.), a first syntax element (such as a flag) may be signaled to indicate whether multi transform selection (MTS) is disabled for all blocks (slices/pictures). 65

a. A second syntax element indicating how to apply TS (such as enable MTS/disable MTS/implicit MTS/

explicit MTS) on intra-coding blocks (slices/pictures) is signaled conditionally on the first syntax element. For example, the second syntax element is signaled only when the first syntax element indicates that TS is not disabled for all blocks (slices/pictures).

b. A third syntax element indicating how to apply MTS (such as enable MTS/disable MTS/implicit MTS/explicit MTS) on inter-coding blocks (slices/pictures) is signaled conditionally on the first syntax element. For example, the third syntax element is signaled only when the first syntax element indicates that MTS is not disabled for all blocks (slices/pictures).

c. An exemplary syntax design is as below

```

...
    enable_mts_flag
    if (enable_mts_flag){
        mts_control_intra
        mts_control_inter
    }
...

```

d. The third syntax element may be signaled conditionally on whether Sub-Block Transform (SBT) is applied or not. An exemplary syntax design is as below

```

...
    if (sps_sbt_enabled_flag)
        mts_control_inter
...

```

e. An exemplary syntax design is as below

<u><i>if (sps_isp_enabled_flag)</i></u>	-
<u><i>sps_isp_non_dct2_enabled_flag</i></u>	<u><i>u(1)</i></u>
...	-
<u><i>sps_intra_mts_selection</i></u>	<u><i>ue(v)</i></u>
<u><i>sps_inter_mts_selection</i></u>	<u><i>ue(v)</i></u>

5. Additional Embodiments

In the following, text changes are shown in underlined bold italicized font.

5.1. Embodiment of Constrains on the Conformance Window

conf_win_left_offset, conf_win_right_offset, conf_win_top_offset, and conf_win_bottom_offset specify the samples of the pictures in the CVS that are output from the decoding process, in terms of a rectangular region specified in picture coordinates for output. When conformance_window_flag is equal to 0, the values of conf_win_left_offset, conf_win_right_offset, conf_win_top_offset, and conf_win_bottom_offset are inferred to be equal to 0.

The conformance cropping window contains the luma samples with horizontal picture coordinates from SubWidthC*conf_win_left_offset to pic_width_in_luma_samples-(SubWidthC*conf_win_right_offset+1) and vertical picture coordinates from SubHeightC*conf_win_top_offset to pic_height_in_luma_samples-(SubHeightC*conf_win_bottom_offset+1), inclusive.

55

The value of $\text{SubWidthC} * (\text{conf_win_left_offset} + \text{conf_win_right_offset})$ shall be less than $\text{pic_width_in_luma_samples}$, and the value of $\text{SubHeightC} * (\text{conf_win_top_offset} + \text{conf_win_bottom_offset})$ shall be less than $\text{pic_height_in_luma_samples}$.
The variables PicOutputWidthL and PicOutputHeightL are derived as follows:

$$\text{PicOutputWidthL} = \text{pic_width_in_luma_samples} - \text{SubWidthC} * (\text{conf_win_right_offset} + \text{conf_win_left_offset}) \quad (7-43)$$

$$\text{PicOutputHeightL} = \text{pic_height_in_pic_size_units} - \text{SubHeightC} * (\text{conf_win_bottom_offset} + \text{conf_win_top_offset}) \quad (7-44)$$

When ChromaArrayType is not equal to 0, the corresponding specified samples of the two chroma arrays are the samples having picture coordinates $(x/\text{SubWidthC}, y/\text{SubHeightC})$, where (x, y) are the picture coordinates of the specified luma samples.

Let $\text{refPicOutputWidthL}$ and $\text{refPicOutputHeightL}$ be the PicOutputWidthL and PicOutputHeightL , respectively, of a reference picture of a current picture referring to this PPS. It is a requirement of bitstream conformance that all of the following conditions are satisfied:
– $\text{PicOutputWidthL} * 2$ shall be greater than or equal to $\text{refPicOutputWidthL}$.
– $\text{PicOutputHeightL} * 2$ shall be greater than or equal to $\text{refPicOutputHeightL}$.
– PicOutputWidthL shall be less than or equal to $\text{refPicOutputWidthL} * 8$.
– PicOutputHeightL shall be less than or equal to $\text{refPicOutputHeightL} * 8$.

5.2. Embodiment 1 of Reference Sample Position Derivation

8.5.6.3.1 General

The variable fRefWidth is set equal to the PicOutputWidthL of the reference picture in luma samples.

The variable fRefHeight is set equal to PicOutputHeightL of the reference picture in luma samples.

The variable fRefLeftOff is set equal to the $\text{conf_win_left_offset}$ of the reference picture in luma samples.

The variable fRefTopOff is set equal to the $\text{conf_win_top_offset}$ of the reference picture in luma samples.

The motion vector mvLX is set equal to $(\text{refMvLX} - \text{mvOffset})$.

If cIdx is equal to 0, the following applies:

The scaling factors and their fixed-point representations are defined as

$$\text{hori_scale_fp} = ((\text{fRefWidth} < 14) + (\text{PicOutputWidthL} > 1)) / \text{PicOutputWidthL} \quad (8-753)$$

$$\text{vert_scale_fp} = ((\text{fRefHeight} < 14) + (\text{PicOutputHeightL} > 1)) / \text{PicOutputHeightL} \quad (8-754)$$

Let $(x_{\text{IntL}}, y_{\text{IntL}})$ be a luma location given in full-sample units and $(x_{\text{FracL}}, y_{\text{FracL}})$ be an offset given in $1/16$ -sample units. These variables are used only in this clause for specifying fractional-sample locations inside the reference sample arrays refPicLX .

56

The top-left coordinate of the bounding block for reference sample padding $(x_{\text{SbIntL}}, y_{\text{SbIntL}})$ is set equal to $(x_{\text{Sb}} + (\text{mvLX}[0] > 4), y_{\text{Sb}} + (\text{mvLX}[1] > 4))$.

For each luma sample location $(x_L = 0 \dots \text{sbWidth} - 1 + \text{brdExtSize}, y_L = 0 \dots \text{sbHeight} - 1 + \text{brdExtSize})$ inside the prediction luma sample array predSamplesLX , the corresponding prediction luma sample value $\text{predSamplesLX}[x_L][y_L]$ is derived as follows: Let $(\text{refxSb}_L, \text{refySb}_L)$ and $(\text{refx}_L, \text{refy}_L)$ be luma locations pointed to by a motion vector $(\text{refMvLX}[0], \text{refMvLX}[1])$ given in $1/16$ -sample units. The variables $\text{refxSb}_L, \text{refx}_L, \text{refySb}_L$, and refy_L are derived as follows:

$$\text{refxSb}_L = ((\text{xSb} - \text{conf_win_left_offset}) < 4) + \text{refMvLX}[0] * \text{hori_scale_fp} \quad (8-755)$$

$$\text{refx}_L = ((\text{Sign}(\text{refxSb}) * ((\text{Abs}(\text{refxSb}) + 128) > 8) + x_L * ((\text{hori_scale_fp} + 8) > 4) + 32) > 6) \quad (8-756)$$

$$\text{refySb}_L = ((\text{ySb} - \text{conf_win_top_offset}) < 4) + \text{refMvLX}[1] * \text{vert_scale_fp} \quad (8-757)$$

$$\text{refy}_L = ((\text{Sign}(\text{refySb}) * ((\text{Abs}(\text{refySb}) + 128) > 8) + y_L * ((\text{vert_scale_fp} + 8) > 4) + 32) > 6) \quad (8-758)$$

$$\begin{aligned} -\text{refx}_L &= \text{refx}_L + (\text{fRefLeftOff} < 4), \\ -\text{refy}_L &= \text{refy}_L + (\text{fRefTopOff} < 4). \end{aligned}$$

The variables $x_{\text{IntL}}, y_{\text{IntL}}, x_{\text{FracL}}$ and y_{FracL} are derived as follows:

$$x_{\text{IntL}} = \text{refx}_L > 4 \quad (8-759)$$

$$y_{\text{IntL}} = \text{refy}_L > 4 \quad (8-760)$$

$$x_{\text{FracL}} = \text{refx}_L \& 15 \quad (8-761)$$

$$y_{\text{FracL}} = \text{refy}_L \& 15 \quad (8-762)$$

If bdoFlag is equal to TRUE or $(\text{sps_affine_prof_enabled_flag}$ is equal to TRUE and $\text{inter_affine_flag}[\text{xSb}][\text{ySb}]$ is equal to TRUE), and one or more of the following conditions are true, the prediction luma sample value $\text{predSamplesLX}[x_L][y_L]$ is derived by invoking the luma integer sample fetching process as specified in clause 8.5.6.3.3 with $(x_{\text{IntL}} + (x_{\text{FracL}} > 3) - 1), y_{\text{IntL}} + (y_{\text{FracL}} > 3) - 1)$ and refPicLX as inputs.

x_L is equal to 0.

x_L is equal to $\text{sbWidth} + 1$.

y_L is equal to 0.

y_L is equal to $\text{sbHeight} + 1$.

Otherwise, the prediction luma sample value $\text{predSamplesLX}[x_L][y_L]$ is derived by invoking the luma sample 8-tap interpolation filtering process as specified in clause 8.5.6.3.2 with $(x_{\text{IntL}} - (\text{brdExtSize} > 0 ? 1 : 0), y_{\text{IntL}} - (\text{brdExtSize} > 0 ? 1 : 0)), (x_{\text{FracL}}, y_{\text{FracL}}), (x_{\text{SbIntL}}, y_{\text{SbIntL}}), \text{refPicLX}, \text{hpelIdx}, \text{sbWidth}, \text{sbHeight}$ and $(x_{\text{Sb}}, y_{\text{Sb}})$ as inputs.

Otherwise (cIdx is not equal to 0), the following applies:

Let $(x_{\text{IntC}}, y_{\text{IntC}})$ be a chroma location given in full-sample units and $(x_{\text{FracC}}, y_{\text{FracC}})$ be an offset given in $1/32$ sample units. These variables are used only in this clause for specifying general fractional-sample locations inside the reference sample arrays refPicLX .

The top-left coordinate of the bounding block for reference sample padding $(x_{\text{SbIntC}}, y_{\text{SbIntC}})$ is set

57

equal to $((xSb/SubWidthC)+(mvLX[0]>>5)$, $(ySb/SubHeightC)+(mvLX[1]>>5)$.

For each chroma sample location $(xC=0 \dots sbWidth-1, yC=0 \dots sbHeight-1)$ inside the prediction chroma sample arrays $predSamplesLX$, the corresponding prediction chroma sample value $predSamplesLX[xC][yC]$ is derived as follows:

Let $(refxSb_C, refySb_C)$ and $(refx_C, refy_C)$ be chroma locations pointed to by a motion vector $(mvLX[0], mvLX[1])$ given in $1/32$ -sample units. The variables $refxSb_C$, $refySb_C$, $refx_C$ and $refy_C$ are derived as follows:

$$refxSb_C = ((xSb - conf_win_left_offset) / SubWidthC << 5) + mvLX[0] * hori_scale_fp \quad (8-763) \quad 15$$

$$refx_C = ((Sign(refxSb_C) * ((Abs(refxSb_C) + 256) >> 9) + xC * ((hori_scale_fp + 8) >> 4)) + 16) >> 5 \quad (8-764)$$

$$refySb_C = ((ySb - conf_win_top_offset) / SubHeightC << 5) + mvLX[1] * vert_scale_fp \quad (8-765) \quad 20$$

$$refy_C = ((Sign(refySb_C) * ((Abs(refySb_C) + 256) >> 9) + yC * ((vert_scale_fp + 8) >> 4)) + 16) >> 5 \quad (8-766)$$

$$\begin{aligned} -refx_C &= refx_C + (fRefLeftOff / SubWidthC << 5), \\ -refy_C &= refy_C + (fRefTopOff / SubHeightC << 5). \end{aligned} \quad (8-767) \quad 25$$

The variables $xInt_C$, $yInt_C$, $xFrac_C$ and $yFrac_C$ are derived as follows:

$$xInt_C = refx_C >> 5 \quad (8-767) \quad 30$$

$$yInt_C = refy_C >> 5 \quad (8-768)$$

$$xFrac_C = refx_C \& 31 \quad (8-769)$$

$$yFrac_C = refy_C \& 31 \quad (8-770) \quad 35$$

5.3. Embodiment 2 of Reference Sample Position Derivation

8.5.6.3.1 General

The variable $fRefWidth$ is set equal to the $PicOutputWidthL$ of the reference picture in luma samples.

The variable $fRefHeight$ is set equal to $PicOutputHeightL$ of the reference picture in luma samples.

The variable $fRefLeftOff$ is set equal to the $conf_win_left_offset$ of the reference picture in luma samples.

The variable $fRefTopOff$ is set equal to the $conf_win_top_offset$ of the reference picture in luma samples.

The motion vector $mvLX$ is set equal to $(refMvLX - mvOffset)$.

If $cIdx$ is equal to 0, the following applies:

The scaling factors and their fixed-point representations are defined as

$$hori_scale_fp = ((fRefWidth << 14) + (PicOutputWidthL >> 1)) / PicOutputWidthL \quad (8-753) \quad 40$$

$$vert_scale_fp = ((fRefHeight << 14) + (PicOutputHeightL >> 1)) / PicOutputHeightL \quad (8-754) \quad 60$$

Let $(xIntL, yIntL)$ be a luma location given in full-sample units and $(xFracL, yFracL)$ be an offset given in $1/16$ -sample units. These variables are used only in this clause for specifying fractional-sample locations inside the reference sample arrays $refPicLX$.

58

The top-left coordinate of the bounding block for reference sample padding $(xSbInt_L, ySbInt_L)$ is set equal to $(xSb + (mvLX[0] >> 4), ySb + (mvLX[1] >> 4))$.

For each luma sample location $(x_L=0 \dots sbWidth-1 + brdExtSize, y_L=0 \dots sbHeight-1 + brdExtSize)$ inside the prediction luma sample array $predSamplesLX$, the corresponding prediction luma sample value $predSamplesLX[x_L][y_L]$ is derived as follows: Let $(refxSb_L, refySb_L)$ and $(refx_L, refy_L)$ be luma locations pointed to by a motion vector $(refMvLX[0], refMvLX[1])$ given in $1/16$ -sample units. The variables $refxSb_L$, $refx_L$, $refySb_L$, and $refy_L$ are derived as follows:

$$refxSb_L = ((xSb - conf_win_left_offset) << 4) + refMvLX[0] * hori_scale_fp \quad (8-755)$$

$$\begin{aligned} refx_L &= ((Sign(refxSb_L) * ((Abs(refxSb_L) + 128) >> 8) + x_L * \\ &\quad ((hori_scale_fp + 8) >> 4)) \\ &\quad + (fRefLeftOff << 10) + 32) >> 6 \end{aligned} \quad (8-756)$$

$$refySb_L = ((ySb - conf_win_top_offset) << 4) + refMvLX[1] * vert_scale_fp \quad (8-757)$$

$$\begin{aligned} refy_L &= ((Sign(refySb_L) * ((Abs(refySb_L) + 128) >> 8) + y_L * \\ &\quad ((vert_scale_fp + 8) >> 4)) \\ &\quad + (fRefTopOff << 10) + 32) >> 6 \end{aligned} \quad (8-758)$$

The variables $xInt_L$, $yInt_L$, $xFrac_L$ and $yFrac_L$ are derived as follows:

$$xInt_L = refx_L >> 4 \quad (8-759)$$

$$yInt_L = refy_L >> 4 \quad (8-760)$$

$$xFrac_L = refx_L \& 15 \quad (8-761)$$

$$yFrac_L = refy_L \& 15 \quad (8-762)$$

If $bdoffFlag$ is equal to TRUE or $(sps_affine_prof_enabled_flag$ is equal to TRUE and $inter_affine_flag[xSb][ySb]$ is equal to TRUE), and one or more of the following conditions are true, the prediction luma sample value $predSamplesLX[x_L][y_L]$ is derived by invoking the luma integer sample fetching process as specified in clause 8.5.6.3.3 with $(xInt_L + (xFrac_L >> 3) - 1)$, $yInt_L + (yFrac_L >> 3) - 1$ and $refPicLX$ as inputs.

x_L is equal to 0.

x_L is equal to $sbWidth + 1$.

y_L is equal to 0.

y_L is equal to $sbHeight + 1$.

Otherwise, the prediction luma sample value $predSamplesLX[x_L][y_L]$ is derived by invoking the luma sample 8-tap interpolation filtering process as specified in clause 8.5.6.3.2 with $(xIntL - (brdExtSize > 0 ? 1 : 0))$, $yIntL - (brdExtSize > 0 ? 1 : 0)$, $(xFracL, yFracL)$, $(xSbInt_L, ySbInt_L)$, $refPicLX$, $hpelIdx$, $sbWidth$, $sbHeight$ and (xSb, ySb) as inputs.

Otherwise ($cIdx$ is not equal to 0), the following applies:

Let $(xIntC, yIntC)$ be a chroma location given in full-sample units and $(xFracC, yFracC)$ be an offset given in $1/32$ sample units. These variables are used only in this clause for specifying general fractional-sample locations inside the reference sample arrays $refPicLX$.

The top-left coordinate of the bounding block for reference sample padding $(xSbIntC, ySbIntC)$ is set

59

equal to $((xSb/SubWidthC)+(mvLX[0]>>5)$, $(ySb/SubHeightC)+(mvLX[1]>>5)$.
 For each chroma sample location $(xC=0 \dots sbWidth-1, yC=0 \dots sbHeight-1)$ inside the prediction chroma sample arrays $predSamplesLX$, the corresponding prediction chroma sample value $predSamplesLX[xC][yC]$ is derived as follows:
 Let $(refxSb_C, refySb_C)$ and $(refx_C, refy_C)$ be chroma locations pointed to by a motion vector $(mvLX[0], mvLX[1])$ given in $1/32$ -sample units. The variables $refxSb_C$, $refySb_C$, $refx_C$ and $refy_C$ are derived as follows:

$$refxSb_C = ((xSb - conf_win_left_offset) / SubWidthC < 5) + mvLX[0] * hori_scale_fp \quad (8-763) \quad 15$$

$$refx_C = ((Sign(refxSb_C) * ((Abs(refxSb_C) + 256) >> 9) + xC * ((hori_scale_fp + 8) >> 4) + 16) >> 5) \quad (8-764)$$

$$refySb_C = ((ySb - conf_win_top_offset) / SubHeightC < 5) + mvLX[1] * vert_scale_fp \quad (8-765) \quad 20$$

$$refy_C = ((Sign(refySb_C) * ((Abs(refySb_C) + 256) >> 9) + yC * ((vert_scale_fp + 8) >> 4) + 16) >> 5) \quad (8-766)$$

$$-refx_C = refx_C + (fRefLeftOff / SubWidthC < 5).$$

$$-refy_C = refy_C + (fRefTopOff / SubHeightC < 5).$$

The variables $xInt_C$, $yInt_C$, $xFrac_C$ and $yFrac_C$ are derived as follows:

$$xInt_C = refx_C >> 5 \quad (8-767) \quad 30$$

$$yInt_C = refy_C >> 5 \quad (8-768)$$

$$xFrac_C = refx_C \& 31 \quad (8-769)$$

$$yFrac_C = refy_C \& 31 \quad (8-770) \quad 35$$

5.4. Embodiment 3 of Reference Sample Position Derivation

8.5.6.3.1 General

The variable $fRefWidth$ is set equal to the $PicOutputWidthL$ of the reference picture in luma samples.

The variable $fRefHeight$ is set equal to $PicOutputHeightL$ of the reference picture in luma samples.

The variable $fRefLeftOff$ is set equal to the $conf_win_left_offset$ of the reference picture in luma samples.

The variable $fRefTopOff$ is set equal to the $conf_win_top_offset$ of the reference picture in luma samples.

The motion vector $mvLX$ is set equal to $(refMvLX - mvOffset)$.

If $cIdx$ is equal to 0, the following applies:

The scaling factors and their fixed-point representations are defined as

$$hori_scale_fp = ((fRefWidth < 14) + (PicOutputWidthL >> 1)) / PicOutputWidthL \quad (8-753)$$

$$vert_scale_fp = ((fRefHeight < 14) + (PicOutputHeightL >> 1)) / PicOutputHeightL \quad (8-754) \quad 60$$

Let $(xInt_L, yInt_L)$ be a luma location given in full-sample units and $(xFrac_L, yFrac_L)$ be an offset given in $1/16$ -sample units. These variables are used only in this clause for specifying fractional-sample locations inside the reference sample arrays $refPicLX$.

60

The top-left coordinate of the bounding block for reference sample padding $(xSbInt_L, ySbInt_L)$ is set equal to $(xSb + (mvLX[0] >> 4), ySb + (mvLX[1] >> 4))$.

For each luma sample location $(x_L=0 \dots sbWidth-1 + brdExtSize, y_L=0 \dots sbHeight-1 + brdExtSize)$ inside the prediction luma sample array $predSamplesLX$, the corresponding prediction luma sample value $predSamplesLX[x_L][y_L]$ is derived as follows:
 Let $(refxSb_L, refySb_L)$ and $(refx_L, refy_L)$ be luma locations pointed to by a motion vector $(refMvLX[0], refMvLX[1])$ given in $1/16$ -sample units. The variables $refxSb_L$, $refx_L$, $refySb_L$, and $refy_L$ are derived as follows:

$$refxSb_L = ((xSb - conf_win_left_offset) < 4) + refMvLX[0] * hori_scale_fp \quad (8-755)$$

$$refx_L = ((Sign(refxSb_L) * ((Abs(refxSb_L) + 128) >> 8) + x_L * ((hori_scale_fp + 8) >> 4)) + (fRefLeftOff < 10) + 32) >> 6 \quad (8-756)$$

$$refySb_L = ((ySb - conf_win_top_offset) < 4) + refMvLX[1] * vert_scale_fp \quad (8-757)$$

$$refy_L = ((Sign(refySb_L) * ((Abs(refySb_L) + 128) >> 8) + y_L * ((vert_scale_fp + 8) >> 4)) + (fRefTopOff < 10) + 32) >> 6 \quad (8-758)$$

The variables $xInt_L$, $yInt_L$, $xFrac_L$ and $yFrac_L$ are derived as follows:

$$xInt_L = refx_L >> 4 \quad (8-759)$$

$$yInt_L = refy_L >> 4 \quad (8-760)$$

$$xFrac_L = refx_L \& 15 \quad (8-761)$$

$$yFrac_L = refy_L \& 15 \quad (8-762)$$

If $bdoffFlag$ is equal to TRUE or $(sps_affine_prof_enabled_flag$ is equal to TRUE and $inter_affine_flag[xSb][ySb]$ is equal to TRUE), and one or more of the following conditions are true, the prediction luma sample value $predSamplesLX[x_L][y_L]$ is derived by invoking the luma integer sample fetching process as specified in clause 8.5.6.3.3 with $(xInt_L + (xFrac_L >> 3) - 1, yInt_L + (yFrac_L >> 3) - 1)$ and $refPicLX$ as inputs.

x_L is equal to 0.

x_L is equal to $sbWidth + 1$.

y_L is equal to 0.

y_L is equal to $sbHeight + 1$.

Otherwise, the prediction luma sample value $predSamplesLX[x_L][y_L]$ is derived by invoking the luma sample 8-tap interpolation filtering process as specified in clause 8.5.6.3.2 with $(xInt_L - (brdExtSize > 0 ? 1 : 0), yInt_L - (brdExtSize > 0 ? 1 : 0))$, $(xFrac_L, yFrac_L)$, $(xSbInt_L, ySbInt_L)$, $refPicLX$, $hpelIdx$, $sbWidth$, $sbHeight$ and (xSb, ySb) as inputs.

Otherwise ($cIdx$ is not equal to 0), the following applies:

Let $(xInt_C, yInt_C)$ be a chroma location given in full-sample units and $(xFrac_C, yFrac_C)$ be an offset given in $1/32$ sample units. These variables are used only in this clause for specifying general fractional-sample locations inside the reference sample arrays $refPicLX$.

The top-left coordinate of the bounding block for reference sample padding $(xSbInt_C, ySbInt_C)$ is set

61

equal to $((xSb/SubWidthC)+(mvLX[0]>>5))$, $(ySb/SubHeightC)+(mvLX[1]>>5))$.

For each chroma sample location $(xC=0 \dots sbWidth-1, yC=0 \dots sbHeight-1)$ inside the prediction chroma sample arrays $predSamplesLX$, the corresponding prediction chroma sample value $predSamplesLX[xC][yC]$ is derived as follows:

Let $(refxSb_C, refySb_C)$ and $(refx_C, refy_C)$ be chroma locations pointed to by a motion vector $(mvLX[0], mvLX[1])$ given in $1/32$ -sample units. The variables $refxSb_C$, $refySb_C$, $refx_C$ and $refy_C$ are derived as follows:

$$refxSb_C = ((xSb - conf_win_left_offset)/SubWidthC << 5) + mvLX[0] * hori_scale_fp \quad (8-763)$$

$$refx_C = ((Sign(refxSb_C) * ((Abs(refxSb_C) + 256) >> 9) + xC * ((hori_scale_fp + 8) >> 4)) + (fRefLeftOff/ SubWidthC << 10) + 16) >> 5 \quad (8-764)$$

$$refySb_C = ((ySb - conf_win_top_offset)/SubHeightC << 5) + mvLX[1] * vert_scale_fp \quad (8-765)$$

$$refy_C = ((Sign(refySb_C) * ((Abs(refySb_C) + 256) >> 9) + yC * ((vert_scale_fp + 8) >> 4)) + (fRefTopOff/ SubHeightC << 10) + 16) >> 5 \quad (8-766)$$

The variables $xInt_C$, $yInt_C$, $xFrac_C$ and $yFrac_C$ are derived as follows:

$$xInt_C = refx_C >> 5 \quad (8-767)$$

$$yInt_C = refy_C >> 5 \quad (8-768)$$

$$xFrac_C = refx_C \& 31 \quad (8-769)$$

$$yFrac_C = refy_C \& 31 \quad (8-770)$$

5.5. Embodiment 1 of Reference Sample Position Clipping

8.5.6.3.1 General

Inputs to this process are:

a luma location (xSb, ySb) specifying the top-left sample of the current coding subblock relative to the top-left luma sample of the current picture,

a variable $sbWidth$ specifying the width of the current coding subblock,

a variable $sbHeight$ specifying the height of the current coding subblock,

a motion vector offset $mvOffset$,

a refined motion vector $refMvLX$,

the selected reference picture sample array $refPicLX$,

the half sample interpolation filter index $hpellIdx$,

the bi-directional optical flow flag $bdoFflag$,

a variable $cIdx$ specifying the colour component index of the current block.

Outputs of this process are:

an $(sbWidth + brdExtSize) \times (sbHeight + brdExtSize)$ array $predSamplesLX$ of prediction sample values.

The prediction block border extension size $brdExtSize$ is derived as follows:

$$brdExtSize = (bdoFflag | (inter_affine_flag[xSb][ySb] \& \& sps_affine_prof_enabled_flag)) ? 2 : 0 \quad (8-752)$$

The variable $fRefWidth$ is set equal to the $PicOutputWidthL$ of the reference picture in luma samples.

62

The variable $fRefHeight$ is set equal to $PicOutputHeightL$ of the reference picture in luma samples.

The motion vector $mvLX$ is set equal to $(refMvLX - mvOffset)$.

If $cIdx$ is equal to 0, the following applies:

The scaling factors and their fixed-point representations are defined as

$$hori_scale_fp = ((RefWidth << 14) + (PicOutputWidthL >> 1)) / PicOutputWidthL \quad (8-753)$$

$$vert_scale_fp = ((RefHeight << 14) + (PicOutputHeightL >> 1)) / PicOutputHeightL \quad (8-754)$$

Let $(xInt_L, yInt_L)$ be a luma location given in full-sample units and $(xFrac_L, yFrac_L)$ be an offset given in $1/16$ -sample units. These variables are used only in this clause for specifying fractional-sample locations inside the reference sample arrays $refPicLX$.

The top-left coordinate of the bounding block for reference sample padding $(xSbInt_L, ySbInt_L)$ is set equal to $(xSb + (mvLX[0] >> 4), ySb + (mvLX[1] >> 4))$.

For each luma sample location $(x_L=0 \dots sbWidth-1 + brdExtSize, y_L=0 \dots sbHeight-1 + brdExtSize)$ inside the prediction luma sample array $predSamplesLX$, the corresponding prediction luma sample value $predSamplesLX[x_L][y_L]$ is derived as follows:

Let $(refxSb_L, refySb_L)$ and $(refx_L, refy_L)$ be luma locations pointed to by a motion vector $(refMvLX[0], refMvLX[1])$ given in $1/16$ -sample units. The variables $refxSb_L$, $refx_L$, $refySb_L$, and $refy_L$ are derived as follows:

$$refxSb_L = ((xSb < 4) + refMvLX[0]) * hori_scale_fp \quad (8-755)$$

$$refx_L = ((Sign(refxSb_L) * ((Abs(refxSb_L) + 128) >> 8) + x_L * ((hori_scale_fp + 8) >> 4)) + 32) >> 6 \quad (8-756)$$

$$refySb_L = ((ySb < 4) + refMvLX[1]) * vert_scale_fp \quad (8-757)$$

$$refy_L = ((Sign(refySb_L) * ((Abs(refySb_L) + 128) >> 8) + y_L * ((vert_scale_fp + 8) >> 4)) + 32) >> 6 \quad (8-758)$$

The variables $xInt_L$, $yInt_L$, $xFrac_L$ and $yFrac_L$ are derived as follows:

$$xInt_L = Clip3(0, fRefWidth - 1, refx_L >> 4) \quad (8-759)$$

$$yInt_L = Clip3(0, fRefHeight - 1, refy_L >> 4) \quad (8-760)$$

$$xFrac_L = refx_L \& 15 \quad (8-761)$$

$$yFrac_L = refy_L \& 15 \quad (8-762)$$

If $bdoFflag$ is equal to TRUE or $(sps_affine_prof_enabled_flag$ is equal to TRUE and $inter_affine_flag[xSb][ySb]$ is equal to TRUE), and one or more of the following conditions are true, the prediction luma sample value $predSamplesLX[x_L][y_L]$ is derived by invoking the luma integer sample fetching process as specified in clause 8.5.6.3.3 with $(xInt_L + (xFrac_L >> 3) - 1)$, $yInt_L + (yFrac_L >> 3) - 1$ and $refPicLX$ as inputs.

x_L is equal to 0.

x_L is equal to $sbWidth + 1$.

y_L is equal to 0.

y_L is equal to $sbHeight + 1$.

Otherwise, the prediction luma sample value $predSamplesLX[x_L][y_L]$ is derived by invoking the luma sample 8-tap interpolation filtering process as specified in clause 8.5.6.3.2 with $(xInt_L - (brdExtSize > 0 ? 1 : 0))$, $yInt_L - (brdExtSize > 0 ? 1 : 0)$, $(xFrac_L$,

63

yFracL), (xSbInt_L, ySbInt_L), refPicLX, hpelldfx, sbWidth, sbHeight and (xSb, ySb) as inputs.

Otherwise (cldx is not equal to 0), the following applies:

Let (xIntC, yIntC) be a chroma location given in full-sample units and (xFracC, yFracC) be an offset given in 1/32 sample units. These variables are used only in this clause for specifying general fractional-sample locations inside the reference sample arrays refPicLX.

The top-left coordinate of the bounding block for reference sample padding (xSbIntC, ySbIntC) is set equal to ((xSb/SubWidthC)+(mvLX[0]>>5), (ySb/SubHeightC)+(mvLX[1]>>5)).

For each chroma sample location (xC=0 . . . sbWidth-1, yC=0 . . . sbHeight-1) inside the prediction chroma sample arrays predSamplesLX, the corresponding prediction chroma sample value predSamplesLX[xC][yC] is derived as follows:

Let (refxSb_C, refySb_C) and (refx_C, refy_C) be chroma locations pointed to by a motion vector (mvLX[0], mvLX[1]) given in 1/32-sample units. The variables refxSb_C, refySb_C, refx_C and refy_C are derived as follows:

$$\text{refxSb}_C = ((xSb/\text{SubWidthC} < 5) + mvLX[0]) * \text{hori_scale_fp} \quad (8-763)$$

$$\text{refx}_C = ((\text{Sign}(\text{refxSb}_C) * ((\text{Abs}(\text{refxSb}_C) + 256) >> 9) + xC * ((\text{hori_scale_fp} + 8) >> 4)) + 16) >> 5 \quad (8-764)$$

$$\text{refySb}_C = ((ySb/\text{SubHeightC} < 5) + mvLX[1]) * \text{vert_scale_fp} \quad (8-765)$$

$$\text{refy}_C = ((\text{Sign}(\text{refySb}_C) * ((\text{Abs}(\text{refySb}_C) + 256) >> 9) + yC * ((\text{vert_scale_fp} + 8) >> 4)) + 16) >> 5 \quad (8-766)$$

The variables xInt_C, yInt_C, xFrac_C and yFrac_C are derived as follows:

$$xInt_C = \text{Clip3}(0, fRefWidth / \text{SubWidthC} - 1, \text{refxC} >> 5) \quad (8-767)$$

$$yInt_C = \text{Clip3}(0, fRefHeight / \text{SubHeightC} - 1, \text{refyC} >> 5) \quad (8-768)$$

$$xFrac_C = \text{refy}_C \ \& \ 31 \quad (8-769)$$

$$yFrac_C = \text{refxC} \ \& \ 31 \quad (8-770)$$

The prediction sample value predSamplesLX[xC][yC] is derived by invoking the process specified in clause 8.5.6.3.4 with (xIntC, yIntC), (xFracC, yFracC), (xSbIntC, ySbIntC), sbWidth, sbHeight and refPicLX as inputs.

5.6. Embodiment 2 of Reference Sample Position Clipping

8.5.6.3.1 General

Inputs to this process are:

a luma location (xSb, ySb) specifying the top-left sample of the current coding subblock relative to the top-left luma sample of the current picture,

a variable sbWidth specifying the width of the current coding subblock,

a variable sbHeight specifying the height of the current coding subblock,

a motion vector offset mvOffset,

a refined motion vector refMvLX,

the selected reference picture sample array refPicLX,

the half sample interpolation filter index hpelldfx,

64

the bi-directional optical flow flag bdofFlag,

a variable cldx specifying the colour component index of the current block.

Outputs of this process are:

an (sbWidth+brdExtSize)×(sbHeight+brdExtSize) array predSamplesLX of prediction sample values.

The prediction block border extension size brdExtSize is derived as follows:

$$\text{brdExtSize} = (\text{bdofFlag} * (\text{inter_affine_flag}[xSb][ySb] \ \& \ \& \ \text{sps_affine_prof_enabled_flag})) ? 2 : 0 \quad (8-752)$$

The variable fRefWidth is set equal to the PicOutputWidthL of the reference picture in luma samples.

The variable fRefHeight is set equal to PicOutputHeightL of the reference picture in luma samples.

The variable fRefLeftOff is set equal to the conf win left offset of the reference picture in luma samples.

The variable fRefTopOff is set equal to the conf win top offset of the reference picture in luma samples.

The motion vector mvLX is set equal to (refMvLX-mvOffset).

If cldx is equal to 0, the following applies:

The scaling factors and their fixed-point representations are defined as

$$\text{hori_scale_fp} = ((\text{RefWidth} < 14) + (\text{PicOutputWidthL} > 1)) / \text{PicOutputWidthL} \quad (8-753)$$

$$\text{vert_scale_fp} = ((\text{RefHeight} < 14) + (\text{PicOutputHeightL} > 1)) / \text{PicOutputHeightL} \quad (8-754)$$

Let (xIntL, yIntL) be a luma location given in full-sample units and (xFracL, yFracL) be an offset given in 1/16-sample units. These variables are used only in this clause for specifying fractional-sample locations inside the reference sample arrays refPicLX.

The top-left coordinate of the bounding block for reference sample padding (xSbIntL, ySbIntL) is set equal to (xSb+(mvLX[0]>>4), ySb+(mvLX[1]>>4)).

For each luma sample location (x_L=0 . . . sbWidth-1+brdExtSize, y_L=0 . . . sbHeight-1+brdExtSize) inside the prediction luma sample array predSamplesLX, the corresponding prediction luma sample value predSamplesLX[x_L][y_L] is derived as follows:

Let (refxSb_L, refySb_L) and (refx_L, refy_L) be luma locations pointed to by a motion vector (refMvLX[0], refMvLX[1]) given in 1/16-sample units. The variables refxSb_L, refx_L, refySb_L, and refy_L are derived as follows:

$$\text{refxSb}_L = ((xSb < 4) + \text{refMvLX}[0]) * \text{hori_scale_fp} \quad (8-755)$$

$$\text{refx}_L = ((\text{Sign}(\text{refxSb}_L) * ((\text{Abs}(\text{refxSb}_L) + 128) >> 8) + x_L * ((\text{hori_scale_fp} + 8) >> 4)) + 32) >> 6 \quad (8-756)$$

$$\text{refySb}_L = ((ySb < 4) + \text{refMvLX}[1]) * \text{vert_scale_fp} \quad (8-757)$$

$$\text{refy}_L = ((\text{Sign}(\text{refySb}_L) * ((\text{Abs}(\text{refySb}_L) + 128) >> 8) + y_L * ((\text{vert_scale_fp} + 8) >> 4)) + 32) >> 6 \quad (8-758)$$

The variables xInt_L, yInt_L, xFrac_L and yFrac_L are derived as follows:

$$xInt_L = \text{Clip3}(fRefLeftOff, fRefLeftOff + fRefWidth - 1, \text{refx}_L >> 4) \quad (8-759)$$

65

$$y_{\text{Int}_L} = \text{Clip3}(\text{fRefTopOff}, \text{fRefTopOff} + \text{fRefHeight} - 1, \text{refy}_L \gg 4) \quad (8-760)$$

$$x_{\text{Frac}_L} = \text{refx}_L \& 15 \quad (8-761)$$

$$y_{\text{Frac}_L} = \text{refy}_L \& 15 \quad (8-762)$$

If `bdoFlag` is equal to `TRUE` or (`sps_affine_prof_enabled_flag` is equal to `TRUE` and `inter_affine_flag` [`xSb`][`ySb`] is equal to `TRUE`), and one or more of the following conditions are true, the prediction luma sample value `predSamplesLX[xL][yL]` is derived by invoking the luma integer sample fetching process as specified in clause 8.5.6.3.3 with (`xIntL` + (`xFracL` >> 3) - 1), `yIntL` + (`yFracL` >> 3) - 1) and `refPicLX` as inputs.

`xL` is equal to 0.

`xL` is equal to `sbWidth` + 1.

`yL` is equal to 0.

`yL` is equal to `sbHeight` + 1.

Otherwise, the prediction luma sample value `predSamplesLX[xL][yL]` is derived by invoking the luma sample 8-tap interpolation filtering process as specified in clause 8.5.6.3.2 with (`xIntL` - (`brdExtSize` > 0 ? 1 : 0)), `yIntL` - (`brdExtSize` > 0 ? 1 : 0)), (`xFracL`, `yFracL`), (`xSbIntL`, `ySbIntL`), `refPicLX`, `hpfldfx`, `sbWidth`, `sbHeight` and (`xSb`, `ySb`) as inputs.

Otherwise (`cldx` is not equal to 0), the following applies:

Let (`xIntC`, `yIntC`) be a chroma location given in full-sample units and (`xFracC`, `yFracC`) be an offset given in 1/32 sample units. These variables are used only in this clause for specifying general fractional-sample locations inside the reference sample arrays `refPicLX`.

The top-left coordinate of the bounding block for reference sample padding (`xSbIntC`, `ySbIntC`) is set equal to (`(xSb/SubWidthC) + (mvLX[0] >> 5)`), (`ySb/SubHeightC`) + (`mvLX[1] >> 5`)).

For each chroma sample location (`xC` = 0 . . . `sbWidth` - 1, `yC` = 0 . . . `sbHeight` - 1) inside the prediction chroma sample arrays `predSamplesLX`, the corresponding prediction chroma sample value `predSamplesLX[xC][yC]` is derived as follows:

Let (`refxSbC`, `refySbC`) and (`refxC`, `refyC`) be chroma locations pointed to by a motion vector (`mvLX[0]`, `mvLX[1]`) given in 1/32-sample units. The variables `refxSbC`, `refySbC`, `refxC` and `refyC` are derived as follows:

$$\text{refxSb}_C = ((xSb/SubWidthC < 5) + mvLX[0]) * \text{hori_scale_fp} \quad (8-763)$$

$$\text{refxC} = ((\text{Sign}(\text{refxSb}_C) * ((\text{Abs}(\text{refxSb}_C) + 256) \gg 9) + xC * ((\text{hori_scale_fp} + 8) \gg 4)) + 16) \gg 5 \quad (8-764)$$

$$\text{refySb}_C = ((ySb/SubHeightC < 5) + mvLX[1]) * \text{vert_scale_fp} \quad (8-765)$$

$$\text{refyC} = ((\text{Sign}(\text{refySb}_C) * ((\text{Abs}(\text{refySb}_C) + 256) \gg 9) + yC * ((\text{vert_scale_fp} + 8) \gg 4)) + 16) \gg 5 \quad (8-766)$$

The variables `xIntC`, `yIntC`, `xFracC` and `yFracC` are derived as follows:

$$x_{\text{Int}_C} = \text{Clip3}(\text{fRefLeftOff} / \text{SubWidthC}, (\text{fRefWidth} + \text{fRefLeftOff}) / \text{SubWidthC} - 1, \text{refxC} \gg 5) \quad (8-767)$$

$$y_{\text{Int}_C} = \text{Clip3}(\text{fRefTopOff} / \text{SubHeightC}, (\text{fRefHeight} + \text{fRefTopOff}) / \text{SubHeightC} - 1, \text{refyC} \gg 5) \quad (8-768)$$

66

$$x_{\text{Frac}_C} = \text{refxC} \& 31 \quad (8-769)$$

$$y_{\text{Frac}_C} = \text{refyC} \& 31 \quad (8-770)$$

The prediction sample value `predSamplesLX[xC][yC]` is derived by invoking the process specified in clause 8.5.6.3.4 with (`xIntC`, `yIntC`), (`xFracC`, `yFracC`), (`xSbIntC`, `ySbIntC`), `sbWidth`, `sbHeight` and `refPicLX` as inputs.

6. Example Implementations of the Disclosed Technology

FIG. 5 is a block diagram of a video processing apparatus 500. The apparatus 500 may be used to implement one or more of the methods described herein. The apparatus 500 may be embodied in a smartphone, tablet, computer, Internet of Things (IoT) receiver, and so on. The apparatus 500 may include one or more processors 502, one or more memories 504 and video processing hardware 506. The processor(s) 502 may be configured to implement one or more methods described in the present disclosure. The memory (memories) 504 may be used for storing data and code used for implementing the methods and techniques described herein. The video processing hardware 506 may be used to implement, in hardware circuitry, some techniques described in the present disclosure, and may be partly or completely be a part of the processors 502 (e.g., graphics processor core (GPU) or other signal processing circuitry).

In the present disclosure, the term “video processing” or coding may refer to video encoding, video decoding, video compression or video decompression. For example, video compression algorithms may be applied during conversion from pixel representation of a video to a corresponding bitstream representation or vice versa. The bitstream representation of a current video block may, for example, correspond to bits that are either co-located or spread in different places within the bitstream, as is defined by the syntax. For example, a macroblock may be encoded in terms of transformed and coded error residual values and also using bits in headers and other fields in the bitstream.

It will be appreciated that the disclosed methods and techniques will benefit video encoder and/or decoder embodiments incorporated within video processing devices such as smartphones, laptops, desktops, and similar devices by allowing the use of the techniques disclosed in the present disclosure.

FIG. 6 is a flowchart for an example method 600 of video processing. The method 600 includes, at 610, performing a conversion between a current video block and a coded representation of the current video block, wherein, during the conversion, if a resolution and/or a size of a reference picture is different from a resolution and/or a size of the current video block, a same interpolation filter is applied to a group of adjacent or non-adjacent samples predicted using the current video block.

Some embodiments may be described using the following clause-based format.

1. A method of video processing, comprising:

performing a conversion between a current video block and a coded representation of the current video block, wherein, during the conversion, if a resolution and/or a size of a reference picture is different from a resolution and/or a size of the current video block, a same interpolation filter is applied to a group of adjacent or non-adjacent samples predicted using the current video block.

67

2. The method of clause 1, wherein the same interpolation filter is a vertical interpolation filter.
3. The method of clause 1, wherein the same interpolation filter is a horizontal interpolation filter.
4. The method of clause 1, wherein the group of adjacent or non-adjacent samples include all samples located in a region of the current video block. 5
5. The method of clause 4, wherein the current video blocks is divided into multiple rectangles each of size $M \times N$. 10
6. The method of clause 5, wherein M and/or N are pre-determined.
7. The method of clause 5, wherein M and/or N are derived from dimensions of the current video block.
8. The method of clause 5, wherein M and/or N are signaled in the coded representation of the current video block. 15
9. The method of clause 1, wherein the group of samples share a same motion vector.
10. The method of clause 9, wherein the group of samples share a same horizontal component and/or a same fractional part of a horizontal component. 20
11. The method of clause 9, wherein the group of samples share a same vertical component and/or a same fractional part of a vertical component. 25
12. The method of one or more of clauses 9-11, wherein the same motion vector or components thereof satisfy one or more rules based at least on one of: the resolution of the reference picture, the size of the reference picture, the resolution of the current video block, the size of the current video block, or a precision value. 30
13. The method of one or more of clauses 9-11, wherein the same motion vector or components thereof corresponds to a motion information of a sample located in the current video block. 35
14. The method of one or more of clauses 9-11, wherein the same motion vector or components thereof are set to a motion information of a virtual sample located inside or outside the group.
15. A method of video processing, comprising: 40
 - performing a conversion between a current video block and a coded representation of the current video block, wherein, during the conversion, if a resolution and/or a size of a reference picture is different from a resolution and/or a size of the current video block, wherein blocks predicted using the current video block are only allowed to use integer-valued motion information related to the current block. 45
16. The method of clause 15, wherein the integer-valued motion information is derived by rounding an original motion information of the current video block. 50
17. The method of clause 15, wherein the original motion information of the current video block is in a horizontal direction and/or a vertical direction.
18. A method of video processing, comprising: 55
 - performing a conversion between a current video block and a coded representation of the current video block, wherein, during the conversion, if a resolution and/or a size of a reference picture is different from a resolution and/or a size of the current video block, an interpolation filter is applied to derive blocks predicted using the current video block, and wherein the interpolation filter is selected based on a rule. 60
19. The method of clause 18, whether the rule is related to the resolution and/or the size of the reference picture relative to the resolution and/or the size of the current video block. 65

68

20. The method of clause 18, wherein the interpolation filter is a vertical interpolation filter.
21. The method of clause 18, wherein the interpolation filter is a horizontal interpolation filter.
22. The method of clause 18, wherein the interpolation filter is one of: a 1-tap filter, a bilinear filter, a 4-tap filter, or a 6-tap filter.
23. The method of clause 22, wherein the interpolation filter is used as part of other steps of the conversion.
24. The method of clause 18, wherein the interpolation filter includes a use of padding samples.
25. The method of clause 18, wherein a use of the interpolation filter depends on a color component of a sample of the current video block.
26. A method of video processing, comprising:
 - performing a conversion between a current video block and a coded representation of the current video block, wherein, during the conversion, if a resolution and/or a size of a reference picture is different from a resolution and/or a size of the current video block, selectively applying a deblocking filter, wherein a strength of the deblocking filter set in accordance with a rule related to the resolution and/or the size of the reference picture relative to the resolution and/or the size of the current video block.
27. The method of clause 27, wherein the strength of the deblocking filter varies from one video block to another.
28. A method of video processing, comprising:
 - performing a conversion between a current video block and a coded representation of the current video block, wherein, during the conversion, if a sub-picture of the current video block exists, a conformance bitstream satisfies a rule related to the resolution and/or the size of the reference picture relative to the resolution and/or the size of the current video block.
29. The method of clause 28, further comprising:
 - splitting the current video block into one or more sub-pictures, wherein the splitting depends at least on the resolution of the current video block.
30. A method of video processing, comprising:
 - performing a conversion between a current video block and a coded representation of the current video block, wherein, during the conversion, a reference picture of the current video block is resampled in accordance with a rule based on dimensions of the current video block.
31. A method of video processing, comprising:
 - performing a conversion between a current video block and a coded representation of the current video block, wherein, during the conversion, a use of a coding tool on the current video block is selectively enabled or disabled depending on a resolution/size of a reference picture of the current video block relative to a resolution/size of the current video block.
32. The method of one or more of the aforementioned clauses, wherein the group of samples are located in a conformance window.
33. The method of clause 32, wherein the conformance window is rectangular in shape.
34. The method of any one or more of the aforementioned clauses, wherein the resolution pertains to a resolution of the coded/decoded video block or a resolution of the conformance window in the coded/decoded video block.

35. The method of any one or more of the aforementioned clauses, wherein the size pertains to a size of the coded/decoded video block or a size of the conformance window in the coded/decoded video block.
36. The method of any one or more of the aforementioned clauses, wherein the dimensions pertain to dimensions of the coded/decoded video block or dimensions of the conformance window in the coded/decoded video block.
37. The method of clause 32, wherein the conformance window is defined by a set of conformance cropping window parameters.
38. The method of clause 37, wherein at least a portion of set of conformance cropping window parameters are implicitly or explicitly signaled in the coded representation.
39. The method of any one or more of the aforementioned clauses, wherein the set of conformance cropping window parameters is disallowed to be signaled in the coded representation.
40. The method of any one or more of the aforementioned clauses, wherein a position of the reference sample is derived with respect to a top-left sample of the current video block in the conformance window.
41. A method of video processing, comprising:
performing a conversion between multiple video blocks and coded representations of the multiple video blocks, wherein, during the conversion, a first conformance window is defined for a first video block and a second conformance window for a second video block, and wherein a ratio of a width and/or a height of the first conformance window to the second conformance window is in accordance with a rule based at least on a conformance bitstream.
42. A video decoding apparatus comprising a processor configured to implement a method recited in one or more of clauses 1 to 41.
43. A video encoding apparatus comprising a processor configured to implement a method recited in one or more of clauses 1 to 41.
44. A computer program product having computer code stored thereon, the code, when executed by a processor, causes the processor to implement a method recited in any of clauses 1 to 41.
45. A method, apparatus or system described in the present disclosure.

FIG. 7 is a block diagram showing an example video processing system 700 in which various techniques disclosed herein may be implemented. Various implementations may include some or all of the components of the system 700. The system 700 may include input 702 for receiving video content. The video content may be received in a raw or uncompressed format, e.g., 8 or 10 bit multi-component pixel values, or may be in a compressed or encoded format. The input 702 may represent a network interface, a peripheral bus interface, or a storage interface. Examples of network interface include wired interfaces such as Ethernet, passive optical network (PON), etc. and wireless interfaces such as wireless fidelity (Wi-Fi) or cellular interfaces.

The system 700 may include a coding component 704 that may implement the various coding or encoding methods described in the present disclosure. The coding component 704 may reduce the average bitrate of video from the input 702 to the output of the coding component 704 to produce a coded representation of the video. The coding techniques

are therefore sometimes called video compression or video transcoding techniques. The output of the coding component 704 may be either stored, or transmitted via a communication connected, as represented by the component 706. The stored or communicated bitstream (or coded) representation of the video received at the input 702 may be used by the component 708 for generating pixel values or displayable video that is sent to a display interface 710. The process of generating user-viewable video from the bitstream representation is sometimes called video decompression. Furthermore, while certain video processing operations are referred to as “coding” operations or tools, it will be appreciated that the coding tools or operations are used at an encoder and corresponding decoding tools or operations that reverse the results of the coding will be performed by a decoder.

Examples of a peripheral bus interface or a display interface may include universal serial bus (USB) or high definition multimedia interface (HDMI) or Displayport, and so on. Examples of storage interfaces include serial advanced technology attachment (SATA), peripheral component interconnect (PCI), integrated drive electronics (IDE) interface, and the like. The techniques described in the present disclosure may be embodied in various electronic devices such as mobile phones, laptops, smartphones or other devices that are capable of performing digital data processing and/or video display.

FIG. 8 is a block diagram that illustrates an example video coding system 100 that may utilize the techniques of this disclosure.

As shown in FIG. 8, video coding system 100 may include a source device 110 and a destination device 120. Source device 110 generates encoded video data which may be referred to as a video encoding device. Destination device 120 may decode the encoded video data generated by source device 110 which may be referred to as a video decoding device.

Source device 110 may include a video source 112, a video encoder 114, and an input/output (I/O) interface 116.

Video source 112 may include a source such as a video capture device, an interface to receive video data from a video content provider, and/or a computer graphics system for generating video data, or a combination of such sources. The video data may comprise one or more pictures. Video encoder 114 encodes the video data from video source 112 to generate a bitstream. The bitstream may include a sequence of bits that form a coded representation of the video data. The bitstream may include coded pictures and associated data. The coded picture is a coded representation of a picture. The associated data may include sequence parameter sets, picture parameter sets, and other syntax structures. I/O interface 116 may include a modulator/demodulator (modem) and/or a transmitter. The encoded video data may be transmitted directly to destination device 120 via I/O interface 116 through network 130a. The encoded video data may also be stored onto a storage medium/server 130b for access by destination device 120.

Destination device 120 may include an I/O interface 126, a video decoder 124, and a display device 122.

I/O interface 126 may include a receiver and/or a modem. I/O interface 126 may acquire encoded video data from the source device 110 or the storage medium/server 130b. Video decoder 124 may decode the encoded video data. Display device 122 may display the decoded video data to a user. Display device 122 may be integrated with the destination device 120, or may be external to destination device 120 which be configured to interface with an external display device.

71

Video encoder **114** and video decoder **124** may operate according to a video compression standard, such as the High Efficiency Video Coding (HEVC) standard, Versatile Video Coding (VVC) standard and other current and/or further standards.

FIG. 9 is a block diagram illustrating an example of video encoder **200**, which may be video encoder **114** in the system **100** illustrated in FIG. 8.

Video encoder **200** may be configured to perform any or all of the techniques of this disclosure. In the example of FIG. 9, video encoder **200** includes a plurality of functional components. The techniques described in this disclosure may be shared among the various components of video encoder **200**. In some examples, a processor may be configured to perform any or all of the techniques described in this disclosure.

The functional components of video encoder **200** may include a partition unit **201**, a prediction unit **202** which may include a mode select unit **203**, a motion estimation unit **204**, a motion compensation unit **205** and an intra prediction unit **206**, a residual generation unit **207**, a transform unit **208**, a quantization unit **209**, an inverse quantization unit **210**, an inverse transform unit **211**, a reconstruction unit **212**, a buffer **213**, and an entropy encoding unit **214**.

In other examples, video encoder **200** may include more, fewer, or different functional components. In an example, prediction unit **202** may include an intra block copy (IBC) unit. The IBC unit may perform prediction in an IBC mode in which at least one reference picture is a picture where the current video block is located.

Furthermore, some components, such as motion estimation unit **204** and motion compensation unit **205** may be highly integrated, but are represented in the example of FIG. 5 separately for purposes of explanation.

Partition unit **201** may partition a picture into one or more video blocks. Video encoder **200** and video decoder **300** may support various video block sizes.

Mode select unit **203** may select one of the coding modes, intra or inter, e.g., based on error results, and provide the resulting intra- or inter-coded block to a residual generation unit **207** to generate residual block data and to a reconstruction unit **212** to reconstruct the encoded block for use as a reference picture. In some example, Mode select unit **203** may select a combination of intra and inter prediction (CIIP) mode in which the prediction is based on an inter prediction signal and an intra prediction signal. Mode select unit **203** may also select a resolution for a motion vector (e.g., a sub-pixel or integer pixel precision) for the block in the case of inter-prediction.

To perform inter prediction on a current video block, motion estimation unit **204** may generate motion information for the current video block by comparing one or more reference frames from buffer **213** to the current video block. Motion compensation unit **205** may determine a predicted video block for the current video block based on the motion information and decoded samples of pictures from buffer **213** other than the picture associated with the current video block.

Motion estimation unit **204** and motion compensation unit **205** may perform different operations for a current video block, for example, depending on whether the current video block is in an I slice, a P slice, or a B slice.

In some examples, motion estimation unit **204** may perform uni-directional prediction for the current video block, and motion estimation unit **204** may search reference pictures of list 0 or list 1 for a reference video block for the current video block. Motion estimation unit **204** may then

72

generate a reference index that indicates the reference picture in list 0 or list 1 that contains the reference video block and a motion vector that indicates a spatial displacement between the current video block and the reference video block. Motion estimation unit **204** may output the reference index, a prediction direction indicator, and the motion vector as the motion information of the current video block. Motion compensation unit **205** may generate the predicted video block of the current block based on the reference video block indicated by the motion information of the current video block.

In other examples, motion estimation unit **204** may perform bi-directional prediction for the current video block, motion estimation unit **204** may search the reference pictures in list 0 for a reference video block for the current video block and may also search the reference pictures in list 1 for another reference video block for the current video block. Motion estimation unit **204** may then generate reference indexes that indicate the reference pictures in list 0 and list 1 containing the reference video blocks and motion vectors that indicate spatial displacements between the reference video blocks and the current video block. Motion estimation unit **204** may output the reference indexes and the motion vectors of the current video block as the motion information of the current video block. Motion compensation unit **205** may generate the predicted video block of the current video block based on the reference video blocks indicated by the motion information of the current video block.

In some examples, motion estimation unit **204** may output a full set of motion information for decoding processing of a decoder.

In some examples, motion estimation unit **204** may not output a full set of motion information for the current video. Rather, motion estimation unit **204** may signal the motion information of the current video block with reference to the motion information of another video block. For example, motion estimation unit **204** may determine that the motion information of the current video block is sufficiently similar to the motion information of a neighboring video block.

In one example, motion estimation unit **204** may indicate, in a syntax structure associated with the current video block, a value that indicates to the video decoder **300** that the current video block has the same motion information as the another video block.

In another example, motion estimation unit **204** may identify, in a syntax structure associated with the current video block, another video block and a motion vector difference (MVD). The motion vector difference indicates a difference between the motion vector of the current video block and the motion vector of the indicated video block. The video decoder **300** may use the motion vector of the indicated video block and the motion vector difference to determine the motion vector of the current video block.

As discussed above, video encoder **200** may predictively signal the motion vector. Two examples of predictive signaling techniques that may be implemented by video encoder **200** include advanced motion vector prediction (AMVP) and merge mode signaling.

Intra prediction unit **206** may perform intra prediction on the current video block. When intra prediction unit **206** performs intra prediction on the current video block, intra prediction unit **206** may generate prediction data for the current video block based on decoded samples of other video blocks in the same picture. The prediction data for the current video block may include a predicted video block and various syntax elements.

73

Residual generation unit **207** may generate residual data for the current video block by subtracting (e.g., indicated by the minus sign) the predicted video block(s) of the current video block from the current video block. The residual data of the current video block may include residual video blocks that correspond to different sample components of the samples in the current video block.

In other examples, there may be no residual data for the current video block for the current video block, for example in a skip mode, and residual generation unit **207** may not perform the subtracting operation.

Transform processing unit **208** may generate one or more transform coefficient video blocks for the current video block by applying one or more transforms to a residual video block associated with the current video block.

After transform processing unit **208** generates a transform coefficient video block associated with the current video block, quantization unit **209** may quantize the transform coefficient video block associated with the current video block based on one or more quantization parameter (QP) values associated with the current video block.

Inverse quantization unit **210** and inverse transform unit **211** may apply inverse quantization and inverse transforms to the transform coefficient video block, respectively, to reconstruct a residual video block from the transform coefficient video block. Reconstruction unit **212** may add the reconstructed residual video block to corresponding samples from one or more predicted video blocks generated by the prediction unit **202** to produce a reconstructed video block associated with the current block for storage in the buffer **213**.

After reconstruction unit **212** reconstructs the video block, loop filtering operation may be performed reduce video blocking artifacts in the video block.

Entropy encoding unit **214** may receive data from other functional components of the video encoder **200**. When entropy encoding unit **214** receives the data, entropy encoding unit **214** may perform one or more entropy encoding operations to generate entropy encoded data and output a bitstream that includes the entropy encoded data.

FIG. **10** is a block diagram illustrating an example of video decoder **300** which may be video decoder **124** in the system **100** illustrated in FIG. **8**.

The video decoder **300** may be configured to perform any or all of the techniques of this disclosure. In the example of FIG. **10**, the video decoder **300** includes a plurality of functional components. The techniques described in this disclosure may be shared among the various components of the video decoder **300**. In some examples, a processor may be configured to perform any or all of the techniques described in this disclosure.

In the example of FIG. **10**, video decoder **300** includes an entropy decoding unit **301**, a motion compensation unit **302**, an intra prediction unit **303**, an inverse quantization unit **304**, an inverse transformation unit **305**, and a reconstruction unit **306** and a buffer **307**. Video decoder **300** may, in some examples, perform a decoding pass generally reciprocal to the encoding pass described with respect to video encoder **200** (e.g., FIG. **9**).

Entropy decoding unit **301** may retrieve an encoded bitstream. The encoded bitstream may include entropy coded video data (e.g., encoded blocks of video data). Entropy decoding unit **301** may decode the entropy coded video data, and from the entropy decoded video data, motion compensation unit **302** may determine motion information including motion vectors, motion vector precision, reference picture list indexes, and other motion information. Motion

74

compensation unit **302** may, for example, determine such information by performing the AMVP and merge mode.

Motion compensation unit **302** may produce motion compensated blocks, possibly performing interpolation based on interpolation filters. Identifiers for interpolation filters to be used with sub-pixel precision may be included in the syntax elements.

Motion compensation unit **302** may use interpolation filters as used by video encoder **20** during encoding of the video block to calculate interpolated values for sub-integer pixels of a reference block. Motion compensation unit **302** may determine the interpolation filters used by video encoder **200** according to received syntax information and use the interpolation filters to produce predictive blocks.

Motion compensation unit **302** may use some of the syntax information to determine sizes of blocks used to encode frame(s) and/or slice(s) of the encoded video sequence, partition information that describes how each macroblock of a picture of the encoded video sequence is partitioned, modes indicating how each partition is encoded, one or more reference frames (and reference frame lists) for each inter-encoded block, and other information to decode the encoded video sequence.

Intra prediction unit **303** may use intra prediction modes for example received in the bitstream to form a prediction block from spatially adjacent blocks. Inverse quantization unit **303** inverse quantizes, i.e., de-quantizes, the quantized video block coefficients provided in the bitstream and decoded by entropy decoding unit **301**. Inverse transform unit **303** applies an inverse transform.

Reconstruction unit **306** may sum the residual blocks with the corresponding prediction blocks generated by motion compensation unit **302** or intra-prediction unit **303** to form decoded blocks. If desired, a deblocking filter may also be applied to filter the decoded blocks in order to remove blockiness artifacts. The decoded video blocks are then stored in buffer **307**, which provides reference blocks for subsequent motion compensation.

FIG. **11** is a flowchart representation of a method of video processing in accordance with the present technology. The method **1100** includes, at operation **1110**, performing a conversion between a current picture of a video and a coded representation of the video. The current picture comprises a first window having a $W \times H$ sample dimension, where W and H are integers. A reference picture associated with the current picture comprises a second window having a $W' \times H'$ sample dimension, where W' and H' are integers. The samples in the first window and the second window are subject to a same processing rule during the conversion, and W , H , W' and H' satisfy a constraint.

In some embodiments, the first window and the second window are scaling windows. The same processing rule specifies that samples in the scaling windows are subject to a resampling operation. In some embodiments, the first window and the second window are conformance windows. The same processing rule specifies that samples outside the conformance windows are discarded when outputting the video.

In some embodiments, the constraint specifies that $W'/a \leq W \leq W' \times b$, a and b being positive integers. In some embodiments, a is equal to 2. In some embodiments, b is equal to 8. In some embodiments, the constraint specifies that $H'/c \leq H \leq H' \times d$, c and d being positive integers. In some embodiments, c is equal to 2. In some embodiments, d is equal to 8. In some embodiments, $a=c$. In some embodiments, $b=d$.

75

In some embodiments, the dimension of the first scaling window or the second scaling window is signaled in the coded representation and is used to determine a set of scaling ratios. In some embodiments, the dimension of the first scaling window or the second scaling window is signaled in the coded representation and is used to determine a top-left offset for a reference sample in applying Reference Picture Resampling (RPR) during the conversion.

FIG. 12 is a flowchart representation of a method of video processing in accordance with the present technology. The method 1200 includes, at operation 1210, performing a conversion between a current picture of a video and a coded representation of the video. During the conversion, offset values for determining a window of the current picture are signaled in the coded representation in N-pel precision, N being a positive integer greater than 1.

In some embodiments, the window comprises a scaling window in which samples are subject to a resampling operation. In some embodiments, the window comprises a conformance window outside which samples are discarded when outputting the video. In some embodiments, the offset values signaled in the coded representation are multiplied by N in determining the conformance window of the current picture. In some embodiments, N is equal to 4 or 8. In some embodiments, the offset values are used in one or more coding processes during the conversion.

FIG. 13 is a flowchart representation of a method of video processing in accordance with the present technology. The method 1300 includes, at operation 1310, performing a conversion between a video comprising a first video unit and a second video unit and a coded representation of the video. A first set of offset values of a first window of the first video unit and a second set of offset values of a second window of the second video unit are determined according to a rule associated with a dimension of the first video unit and a dimension of the second video unit. Samples in the first window and the second window are subject to a same processing rule during the conversion.

In some embodiments, the first window and the second window are scaling windows, and the same processing rule specifies that samples in the scaling windows are subject to a resampling operation. In some embodiments, the first window and the second window are conformance windows, and the same processing rule specifies that samples outside the conformance windows are discarded when outputting the video.

In some embodiments, the first video unit or the second video unit comprises a picture of the video. In some embodiments, the rule specifies that, in case the dimension of the first video unit is same as the dimension of the second video unit, the first set of offset values is different than the second set of offset values. In some embodiments, the rule specifies that, in case the dimension of the first video unit is different than the dimension of the second video unit, the first set of offset values is same as the second set of offset values. In some embodiments, the rule specifies that the first set of offset values is same as the second set of offset values regardless of whether the dimension of the first video unit is same as the dimension of the second video unit.

In some embodiments, the first conformance window has a dimension of $W1 \times H1$ and a top-left position of the first conformance window is denoted as $(X1, Y1)$, the second conformance window has a dimension of $W2 \times H2$ and a top-left position of the second conformance window is denoted as $(X2, Y2)$, and the dimension of the first video unit is denoted as $PW1 \times PH1$ and the dimension of the second video unit is denoted as $PW2 \times PH2$. In some embodiments,

76

$W1/W2 = X1/X2$. In some embodiments, $W1/X1 = W2/X2$. In some embodiments, $W1 \times X2 = W2 \times X1$. In some embodiments, $H1/H2 = Y1/Y2$. In some embodiments, $H1/Y1 = H2/Y2$. In some embodiments, $H1 \times Y2 = H2 \times Y1$. In some embodiments, $PW1/PW2 = X1/X2$. In some embodiments, $PW1/X1 = PW2/X2$. In some embodiments, $PW1 \times X2 = PW2 \times X1$. In some embodiments, $PH1/PH2 = Y1/Y2$. In some embodiments, $PH1/Y1 = PH2/Y2$. In some embodiments, $PH1 \times Y2 = PH2 \times Y1$. In some embodiments, $PW1/PW2 = W1/W2$. In some embodiments, $PW1/W1 = PW2/W2$. In some embodiments, $PW1 \times W2 = PW2 \times W1$. In some embodiments, $PH1/PH2 = H1/H2$. In some embodiments, $PH1/H1 = PH2/H2$. In some embodiments, $PH1 \times H2 = PH2 \times H1$. In some embodiments, in case $PW1 > PW2$, $W1 > W2$. In some embodiments, in case $PW1 < PW2$, $W1 < W2$. In some embodiments, $(PW1 - PW2) \times (W1 - W2) > 0$. In some embodiments, in case $PH1 > PH2$, $H1 > H2$. In some embodiments, in case $PH1 < PH2$, $H1 < H2$. In some embodiments, $(PH1 - PH2) \times (H1 - H2) \geq 0$. In some embodiments, in case $PW1 \geq PW2$, $(W1/W2) \leq (PW1/PW2)$. In some embodiments, in case $PW1 \geq PW2$, $(W1/W2) \geq (PW1/PW2)$. In some embodiments, in case $PH1 \geq PH2$, $(H1/H2) \leq (PH1/PH2)$. In some embodiments, in case $PH1 \geq PH2$, $(H1/H2) \geq (PH1/PH2)$.

FIG. 14 is a flowchart representation of a method of video processing in accordance with the present technology. The method 1400 includes, at operation 1410, determining, for a conversion between a current picture of a video and a coded representation of the video, a position of a reference sample in a reference picture that is associated with the current picture based on a top-left position of a window of a picture, where the picture includes at least the current picture or the reference picture. The window is subject to a processing rule during the conversion. The method 1400 also includes, at operation 1420, performing the conversion based on the determining.

In some embodiments, the window comprises a scaling window, and the processing rule specifies that samples in the scaling window are subject to a resampling operation. In some embodiments, the window comprises a conformance window, and the processing rule specifies that samples outside the conformance window are disposable when outputting the video.

In some embodiments, the top-left position of the window is determined based on a left offset value and a top offset value associated with the window. In some embodiments, a horizontal position of a current sample relative to the top-left position of the window xSb' is determined based on a horizontal position of the current sample in the current picture xSb and the left offset of the window of the current picture. In some embodiments, the horizontal position of the current sample relative to the top-left position of the window xSb' is further determined based on a sample precision.

In some embodiments, the horizontal position of the current sample relative to the top-left position of the window xSb' is used to determine a position of a reference sample. In some embodiments, a horizontal position of a reference sample relative to the reference picture is determined based on xSb' and the left offset of the window of the reference picture. In some embodiments, the sample precision indicates a shift of four bits.

In some embodiments, a vertical position of a current sample relative to the top-left position of the window ySb' is determined based on a vertical position of the current sample in the current picture ySb and the top offset of the window of the current picture. In some embodiments, the vertical position of the current sample relative to the top-left position

of the window ySb' is further determined based on a sample precision. In some embodiments, the vertical position of the current sample relative to the top-left position of the window ySb' is used to determine a position of a reference sample. In some embodiments, a vertical position of a reference sample relative to the reference picture is determined based on ySb' and the top offset of the window of the reference picture. In some embodiments, the sample precision indicates a shift of four bits.

In some embodiments, usage of the top-left position of the window is based on a color component of the video. In some embodiments, the color component comprises a luma component.

FIG. 15 is a flowchart representation of a method of video processing in accordance with the present technology. The method 1500 includes, at operation 1510, performing a conversion between a current picture of a video and a coded representation of the video. The current picture comprises a first window having a $W \times H$ sample dimension, where W and H are integers. A reference picture associated with the current picture comprises a second window having a $W' \times H'$ sample dimension, where W' and H' are integers and a top-left position of the second window is denoted as $(X0, Y0)$. Samples in the first window and the second window are subject to a same processing rule during the conversion. An integer part of a coordinate of a reference sample is constrained in a range that is determined in part based on the first window or the second window.

In some embodiments, the integer part of a horizontal coordinate of the reference sample is constrained in the range of $[\min W, \max W]$. In some embodiments, $\min W$ is equal to 0. In some embodiments, $\min W$ is equal to $X0$. In some embodiments, $\max W$ is equal to $W-1$. In some embodiments, $\max W$ is equal to $W'-1$. In some embodiments, $\max W$ is equal to $X0+W'-1$. In some embodiments, at least one of $\min W$ or $\max W$ is modified based on a color component and/or a color format of the video. In some embodiments, $\min W$ is modified to be $\min W \times \text{subC}$, subC being an integer. In some embodiments, $\min W$ is modified to be $\min W / \text{subC}$, subC being an integer. In some embodiments, $\max W$ is modified to be $\max W \times \text{subC}$, subC being an integer. In some embodiments, $\max W$ is modified to be $\max W / \text{subC}$, subC being an integer. In some embodiments, subC is 1 in case the color component comprises a luma component. In some embodiments, subC is 2 in case the color component comprises a chroma component and the color format is 4:2:0 or 4:2:2. In some embodiments, subC is 1 in case the color component comprises a chroma component and the color format is 4:4:4.

In some embodiments, the integer part of a vertical coordinate of the reference sample is constrained in the range of $[\min H, \max H]$. In some embodiments, $\min H$ is equal to 0. In some embodiments, $\min H$ is equal to $Y0$. In some embodiments, $\max H$ is equal to $H-1$. In some embodiments, $\max H$ is equal to $H'-1$. In some embodiments, $\max H$ is equal to $Y0+H'-1$. In some embodiments, at least one of $\min H$ or $\max H$ is modified based on a color component and/or a color format of the video. In some embodiments, $\min H$ is modified to be $\min H \times \text{subC}$, subC being an integer. In some embodiments, $\min H$ is modified to be $\min H / \text{subC}$, subC being an integer. In some embodiments, $\max H$ is modified to be $\max H \times \text{subC}$, subC being an integer. In some embodiments, $\max H$ is modified to be $\max H / \text{subC}$, subC being an integer. In some embodiments, subC is 1 in case the color component comprises a luma component. In some

embodiments, subC is 2 in case the color component comprises a chroma component and the color format is 4:2:2. In some embodiments, subC is 1 in case the color component comprises a chroma component and the color format is 4:4:4.

In some embodiments, applicability of the constraint for the integer part of the coordinate of the reference sample is determined based on a dimension of the current picture and a dimension of the reference picture. In some embodiments, the constraint for the integer part of the coordinate of the reference sample is based on a dimension of the first window and a dimension of the second window. In some embodiments, the constraint is applicable in case (1) the dimension of the current picture is different than the dimension of the reference picture, or (2) the dimension of the first window is different than the dimension of the second window. In some embodiments, the first window and the second window are scaling windows, and the same processing rule specifies that samples in the scaling windows are subject to a resampling operation. In some embodiments, the first window and the second window are conformance windows, and the same processing rule specifies that samples outside the conformance windows are discarded when outputting the video.

In some embodiments, the conversion includes encoding the video into the coded representation. In some embodiments, the conversion includes decoding the coded representation into the video.

Some embodiments of the disclosed technology include making a decision or determination to enable a video processing tool or mode. In an example, when the video processing tool or mode is enabled, the encoder will use or implement the tool or mode in the processing of a block of video, but may not necessarily modify the resulting bitstream based on the usage of the tool or mode. That is, a conversion from the block of video to the bitstream representation of the video will use the video processing tool or mode when it is enabled based on the decision or determination. In another example, when the video processing tool or mode is enabled, the decoder will process the bitstream with the knowledge that the bitstream has been modified based on the video processing tool or mode. That is, a conversion from the bitstream representation of the video to the block of video will be performed using the video processing tool or mode that was enabled based on the decision or determination.

Some embodiments of the disclosed technology include making a decision or determination to disable a video processing tool or mode. In an example, when the video processing tool or mode is disabled, the encoder will not use the tool or mode in the conversion of the block of video to the bitstream representation of the video. In another example, when the video processing tool or mode is disabled, the decoder will process the bitstream with the knowledge that the bitstream has not been modified using the video processing tool or mode that was enabled based on the decision or determination.

The disclosed and other solutions, examples, embodiments, modules and the functional operations described in this disclosure can be implemented in digital electronic circuitry, or in computer software, firmware, or hardware, including the structures disclosed in this disclosure and their structural equivalents, or in combinations of one or more of them. The disclosed and other embodiments can be implemented as one or more computer program products, i.e., one or more modules of computer program instructions encoded on a computer readable medium for execution by, or to control the operation of, data processing apparatus. The computer readable medium can be a machine-readable storage device, a machine-readable storage substrate, a memory

device, a composition of matter effecting a machine-readable propagated signal, or a combination of one or more them. The term "data processing apparatus" encompasses all apparatus, devices, and machines for processing data, including by way of example a programmable processor, a computer, or multiple processors or computers. The apparatus can include, in addition to hardware, code that creates an execution environment for the computer program in question, e.g., code that constitutes processor firmware, a protocol stack, a database management system, an operating system, or a combination of one or more of them. A propagated signal is an artificially generated signal, e.g., a machine-generated electrical, optical, or electromagnetic signal, that is generated to encode information for transmission to suitable receiver apparatus.

A computer program (also known as a program, software, software application, script, or code) can be written in any form of programming language, including compiled or interpreted languages, and it can be deployed in any form, including as a stand-alone program or as a module, component, subroutine, or other unit suitable for use in a computing environment. A computer program does not necessarily correspond to a file in a file system. A program can be stored in a portion of a file that holds other programs or data (e.g., one or more scripts stored in a markup language document), in a single file dedicated to the program in question, or in multiple coordinated files (e.g., files that store one or more modules, sub programs, or portions of code). A computer program can be deployed to be executed on one computer or on multiple computers that are located at one site or distributed across multiple sites and interconnected by a communication network.

The processes and logic flows described in this disclosure can be performed by one or more programmable processors executing one or more computer programs to perform functions by operating on input data and generating output. The processes and logic flows can also be performed by, and apparatus can also be implemented as, special purpose logic circuitry, e.g., a field programmable gate array (FPGA) or an application specific integrated circuit (ASIC).

Processors suitable for the execution of a computer program include, by way of example, both general and special purpose microprocessors, and any one or more processors of any kind of digital computer. Generally, a processor will receive instructions and data from a read only memory or a random-access memory or both. The essential elements of a computer are a processor for performing instructions and one or more memory devices for storing instructions and data. Generally, a computer will also include, or be operatively coupled to receive data from or transfer data to, or both, one or more mass storage devices for storing data, e.g., magnetic, magneto optical disks, or optical disks. However, a computer need not have such devices. Computer readable media suitable for storing computer program instructions and data include all forms of non-volatile memory, media and memory devices, including by way of example semiconductor memory devices, e.g., erasable programmable read-only memory (EPROM), electrically erasable programmable read-only memory (EEPROM), and flash memory devices; magnetic disks, e.g., internal hard disks or removable disks; magneto optical disks; and compact disc, read-only memory (CD ROM) and digital versatile disc read-only memory (DVD-ROM) disks. The processor and the memory can be supplemented by, or incorporated in, special purpose logic circuitry.

While the present disclosure contains many specifics, these should not be construed as limitations on the scope of

any subject matter or of what may be claimed, but rather as descriptions of features that may be specific to particular embodiments of particular techniques. Certain features that are described in the present disclosure in the context of separate embodiments can also be implemented in combination in a single embodiment. Conversely, various features that are described in the context of a single embodiment can also be implemented in multiple embodiments separately or in any suitable subcombination. Moreover, although features may be described above as acting in certain combinations and even initially claimed as such, one or more features from a claimed combination can in some cases be excised from the combination, and the claimed combination may be directed to a subcombination or variation of a subcombination.

Similarly, while operations are depicted in the drawings in a particular order, this should not be understood as requiring that such operations be performed in the particular order shown or in sequential order, or that all illustrated operations be performed, to achieve desirable results. Moreover, the separation of various system components in the embodiments described in the present disclosure should not be understood as requiring such separation in all embodiments.

Only a few implementations and examples are described and other implementations, enhancements and variations can be made based on what is described and illustrated in the present disclosure.

What is claimed is:

1. A method of processing video data, comprising:

determining, for a conversion between a video unit of a video and a bitstream of the video, a first syntax element and a second syntax element; and

performing the conversion based on the determining,

wherein the first syntax element and the second syntax element have an equivalent functionality,

wherein the first syntax element is included at a first level in the video unit in the bitstream, and the second syntax element is included at a second level in the video unit different from the first level,

wherein the first level is a picture header and the second level is a slice header, a sequence parameter set or a picture parameter set, or wherein the first level is a slice header and the second level is a sequence parameter set or a picture parameter set,

wherein the first syntax element specifying whether a use of an explicit scaling list is enabled is included in the slice header, and the second syntax element specifying whether the use of the explicit scaling list is enabled is included in the sequence parameter set, and

wherein the first syntax element is not included in the slice header when the second syntax element is not included in the sequence parameter set.

2. The method of claim 1, wherein the first syntax element specifying whether a coding tool comprising luma mapping and chroma scaling is enabled is included in the slice header, and the second syntax element specifying whether the coding tool is enabled is included in the sequence parameter set, and

wherein the first syntax element is not included in the slice header when the second syntax element is not included in the sequence parameter set.

3. The method of claim 1, wherein the first syntax element specifying whether adaptive loop filter (ALF) is enabled is included in the slice header, and the second syntax element specifying whether ALF is enabled is included in the sequence parameter set, and

81

wherein the first syntax element is not included in the slice header when the second syntax element is not included in the sequence parameter set.

4. The method of claim 1, wherein the first syntax element specifying whether mmvd-fullpel-only is enabled is included in the picture header, and the second syntax element specifying whether mmvd-fullpel-only is enabled is included in the sequence parameter set, and

wherein the first syntax element is not included in the picture header when the second syntax element is not included in the sequence parameter set.

5. The method of claim 1, wherein whether a third syntax element is included in the sequence parameter set depends on a color format of the sequence parameter set, and wherein the third syntax element specifies whether a joint coding of chroma residuals is enabled.

6. The method of claim 5, wherein the third syntax element is not included in the sequence parameter set when the color format of the sequence parameter set is 4:0:0, and the third syntax element is included in the sequence parameter set when the color format of the sequence parameter set is 4:2:0, 4:2:2, or 4:4:4.

7. The method of claim 5, wherein a joint_cbr_sign_flag is included at a level different from the sequence parameter set in the bitstream when the third syntax element specifies the joint coding of chroma residuals is enabled, and

wherein the joint_cbr_sign flag is not included in the bitstream when the third syntax element specifies the joint coding of chroma residuals is not enabled.

8. The method of claim 1, wherein a sps_mts_enabled_flag specifying whether sps_explicit_mts_intra_enabled_flag and sps_explicit_mts_inter_enabled_flag are present is included in the sequence parameter set, and

wherein a fourth syntax element is included in the bitstream to indicate whether multiple transform selection MTS is disabled for all pictures.

9. The method of claim 1, wherein the conversion includes encoding the video unit into the bitstream.

10. The method of claim 1, wherein the conversion includes decoding the video unit from the bitstream.

11. An apparatus for processing video data comprising a processor and a non-transitory memory with instructions thereon, wherein the instructions upon execution by the processor, cause the processor to:

determine, for a conversion between a video unit of a video and a bitstream of the video, a first syntax element and a second syntax element; and

perform the conversion based on the determining,

wherein the first syntax element and the second syntax element have an equivalent functionality,

wherein the first syntax element is included at a first level in the video unit in the bitstream, and the second syntax element is included at a second level in the video unit different from the first level,

wherein the first level is a picture header and the second level is a slice header, a sequence parameter set or a picture parameter set, or wherein the first level is a slice header and the second level is a sequence parameter set or a picture parameter set,

wherein the first syntax element specifying whether a use of an explicit scaling list is enabled is included in the slice header, and the second syntax element specifying whether the use of the explicit scaling list is enabled is included in the sequence parameter set, and

wherein the first syntax element is not included in the slice header when the second syntax element is not included in the sequence parameter set.

82

12. The apparatus of claim 11, wherein the first syntax element specifying whether a coding tool comprising luma mapping and chroma scaling is enabled is included in the slice header, and the second syntax element specifying whether the coding tool is enabled is included in the sequence parameter set, and

wherein the second syntax element is not included in the sequence parameter set when the first syntax element is not included in the slice header.

13. A non-transitory computer-readable storage medium storing instructions that cause a processor to:

determine, for a conversion between a video unit of a video and a bitstream of the video, a first syntax element and a second syntax element;

perform the conversion based on the determining,

wherein the first syntax element and the second syntax element have an equivalent functionality,

wherein the first syntax element is included at a first level in the video unit in the bitstream, and the second syntax element is included at a second level in the video unit different from the first level,

wherein the first level is a picture header and the second level is a slice header, a sequence parameter set or a picture parameter set, or wherein the first level is a slice header and the second level is a sequence parameter set or a picture parameter set,

wherein the first syntax element specifying whether a use of an explicit scaling list is enabled is included in the slice header, and the second syntax element specifying whether the use of the explicit scaling list is enabled is included in the sequence parameter set, and

wherein the first syntax element is not included in the slice header when the second syntax element is not included in the sequence parameter set.

14. The non-transitory computer-readable storage medium of claim 13, wherein the first syntax element specifying whether a coding tool comprising luma mapping and chroma scaling is enabled is included in the slice header, and the second syntax element specifying whether the coding tool is enabled is included in the sequence parameter set, and

wherein the second syntax element is not included in the sequence parameter set when the first syntax element is not included in the slice header.

15. A method for storing a bitstream of a video, comprising:

determining, for a video unit of the video, a first syntax element and a second syntax element;

generating the bitstream based on the determining; and

storing the bitstream in a non-transitory computer-readable recording medium,

wherein the first syntax element and the second syntax element have an equivalent functionality,

wherein the first syntax element is included at a first level in the video unit in the bitstream, and the second syntax element is included at a second level in the video unit different from the first level,

wherein the first level is a picture header and the second level is a slice header, a sequence parameter set or a picture parameter set, or wherein the first level is a slice header and the second level is a sequence parameter set or a picture parameter set,

wherein the first syntax element specifying whether a use of an explicit scaling list is enabled is included in the slice header, and the second syntax element specifying whether the use of the explicit scaling list is enabled is included in the sequence parameter set, and

83

wherein the first syntax element is not included in the slice header when the second syntax element is not included in the sequence parameter set.

16. The non-transitory computer-readable recording medium of claim 15, wherein the first syntax element specifying whether a coding tool comprising luma mapping and chroma scaling is enabled is included in the slice header, and the second syntax element specifying whether the coding tool is enabled is included in the sequence parameter set, and

wherein the first syntax element is not included in the slice header when the second syntax element is not included in the sequence parameter set.

17. The apparatus of claim 11, wherein the first syntax element specifying whether adaptive loop filter (ALF) is enabled is included in the slice header, and the second syntax element specifying whether ALF is enabled is included in the sequence parameter set, and

wherein the first syntax element is not included in the slice header when the second syntax element is not included in the sequence parameter set.

18. The apparatus of claim 11, wherein the first syntax element specifying whether mmvd-fullpel-only is enabled is included in the picture header, and the second syntax ele-

84

ment specifying whether mmvd-fullpel-only is enabled is included in the sequence parameter set, and

wherein the first syntax element is not included in the picture header when the second syntax element is not included in the sequence parameter set.

19. The non-transitory computer-readable storage medium of claim 13, wherein the first syntax element specifying whether adaptive loop filter (ALF) is enabled is included in the slice header, and the second syntax element specifying whether ALF is enabled is included in the sequence parameter set, and

wherein the first syntax element is not included in the slice header when the second syntax element is not included in the sequence parameter set.

20. The non-transitory computer-readable storage medium of claim 13, wherein the first syntax element specifying whether mmvd-fullpel-only is enabled is included in the picture header, and the second syntax element specifying whether mmvd-fullpel-only is enabled is included in the sequence parameter set, and

wherein the first syntax element is not included in the picture header when the second syntax element is not included in the sequence parameter set.

* * * * *